



CUDA Tile C++

Practical Automatic Parallelization for the GPU

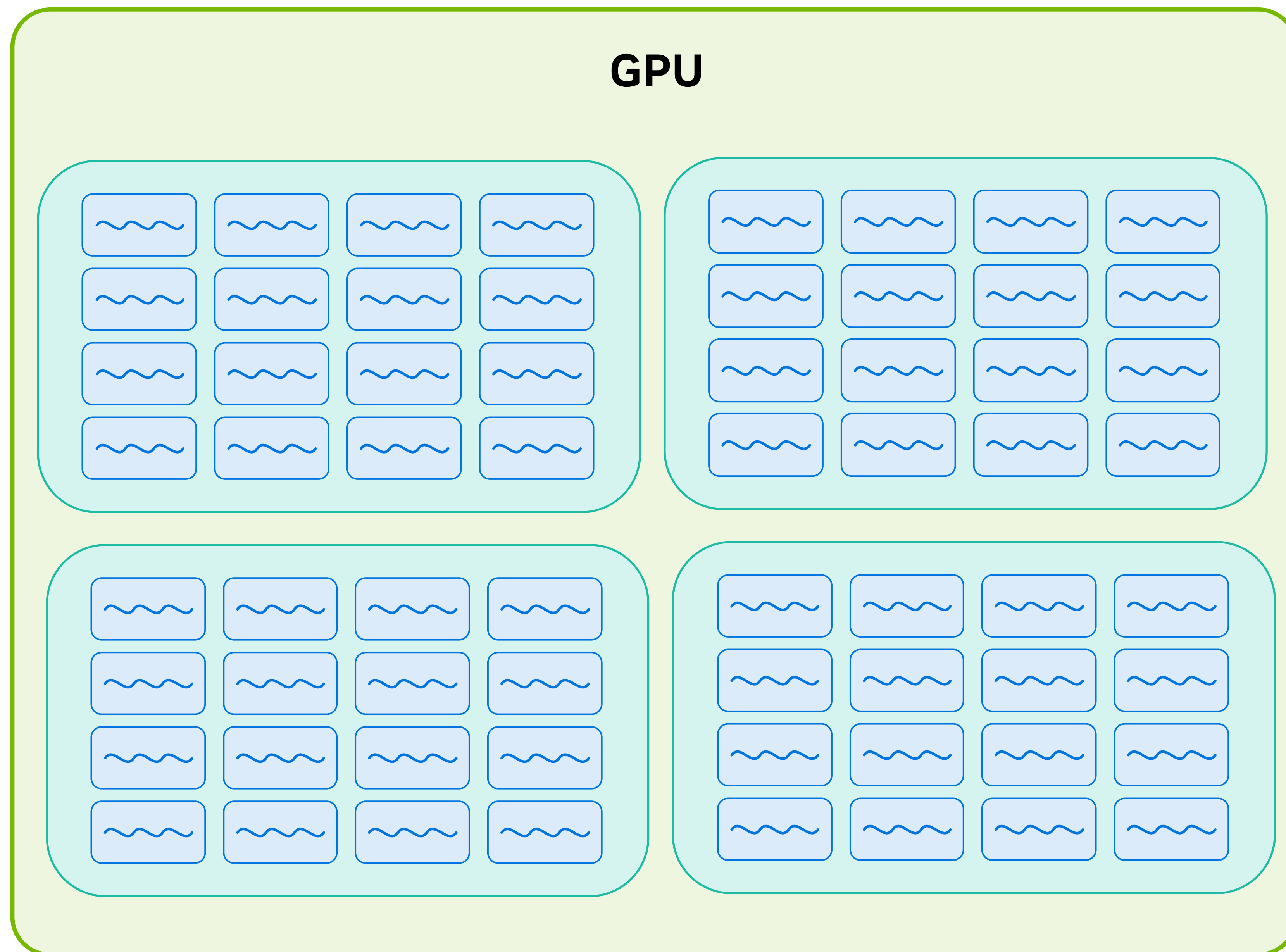
EZRA STEIN



20
26



GPUs are Powerful Systems



Data parallel applications:

- Graphics
- Scientific Computing
- Deep Learning

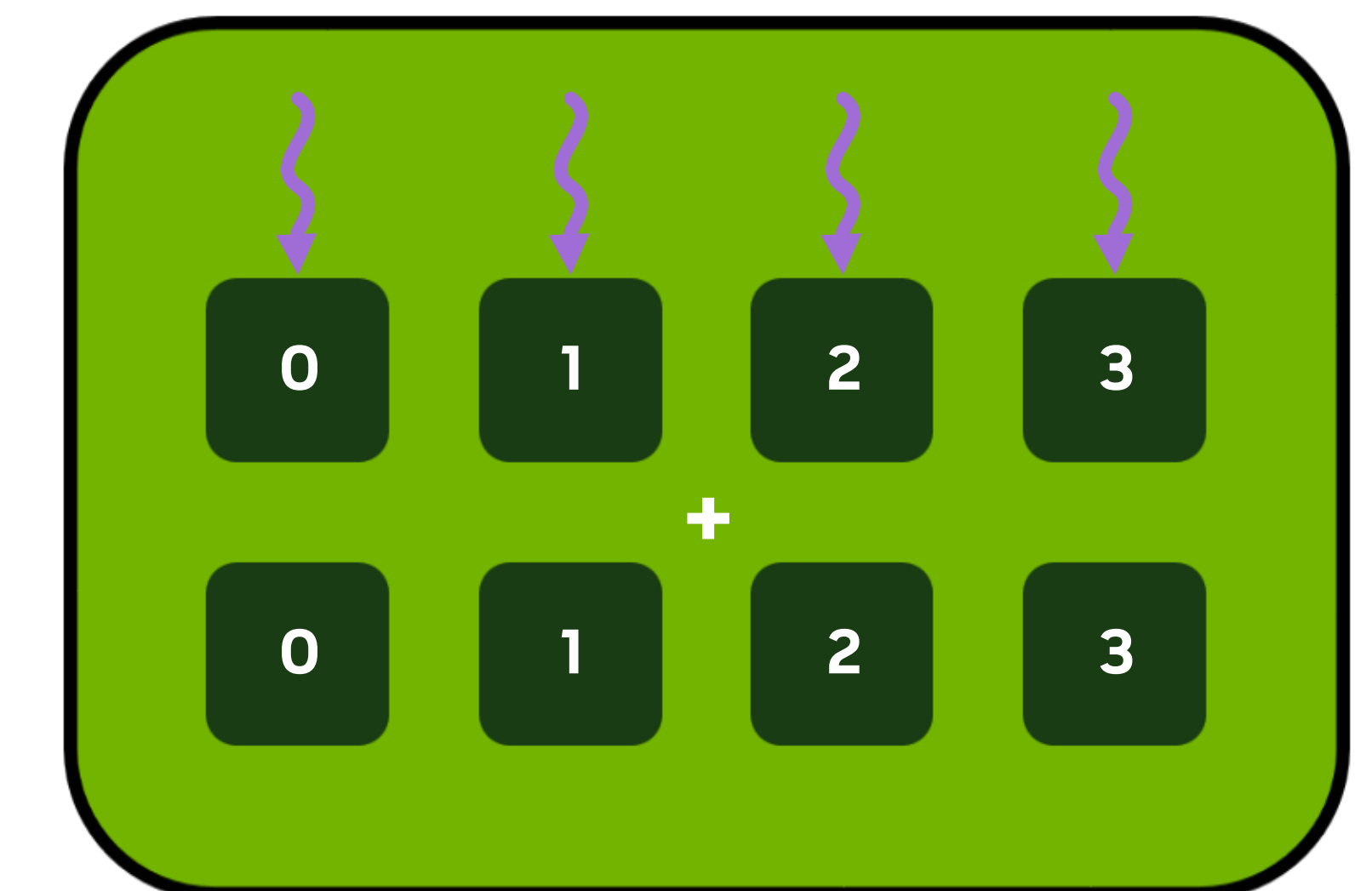
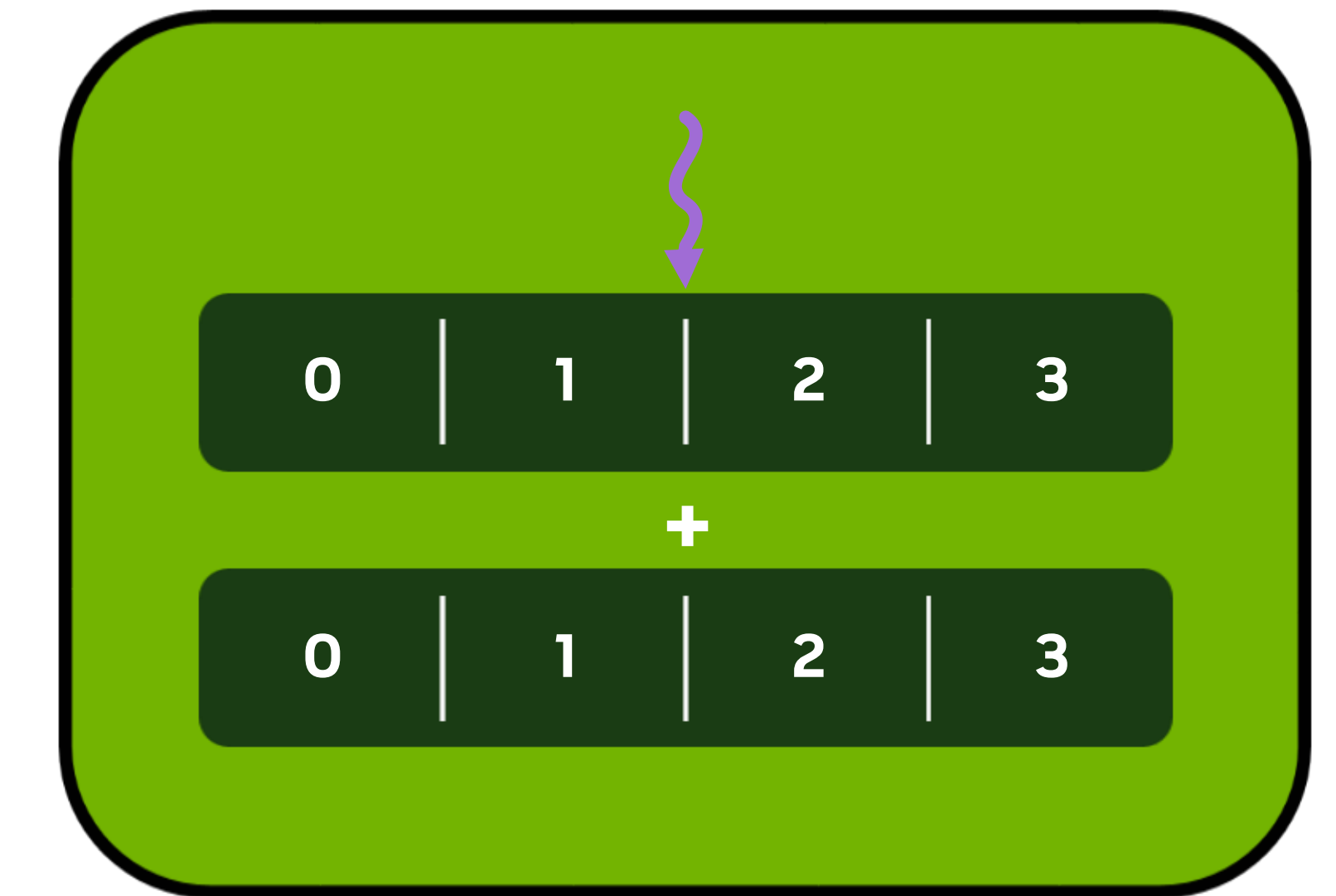
With Great Power Comes Great Responsibility...

The programmer is responsible for:

- Assigning data to hardware threads
- Avoiding race conditions
- Managing data movement and caches
- Ensuring hardware resources are 100% utilized

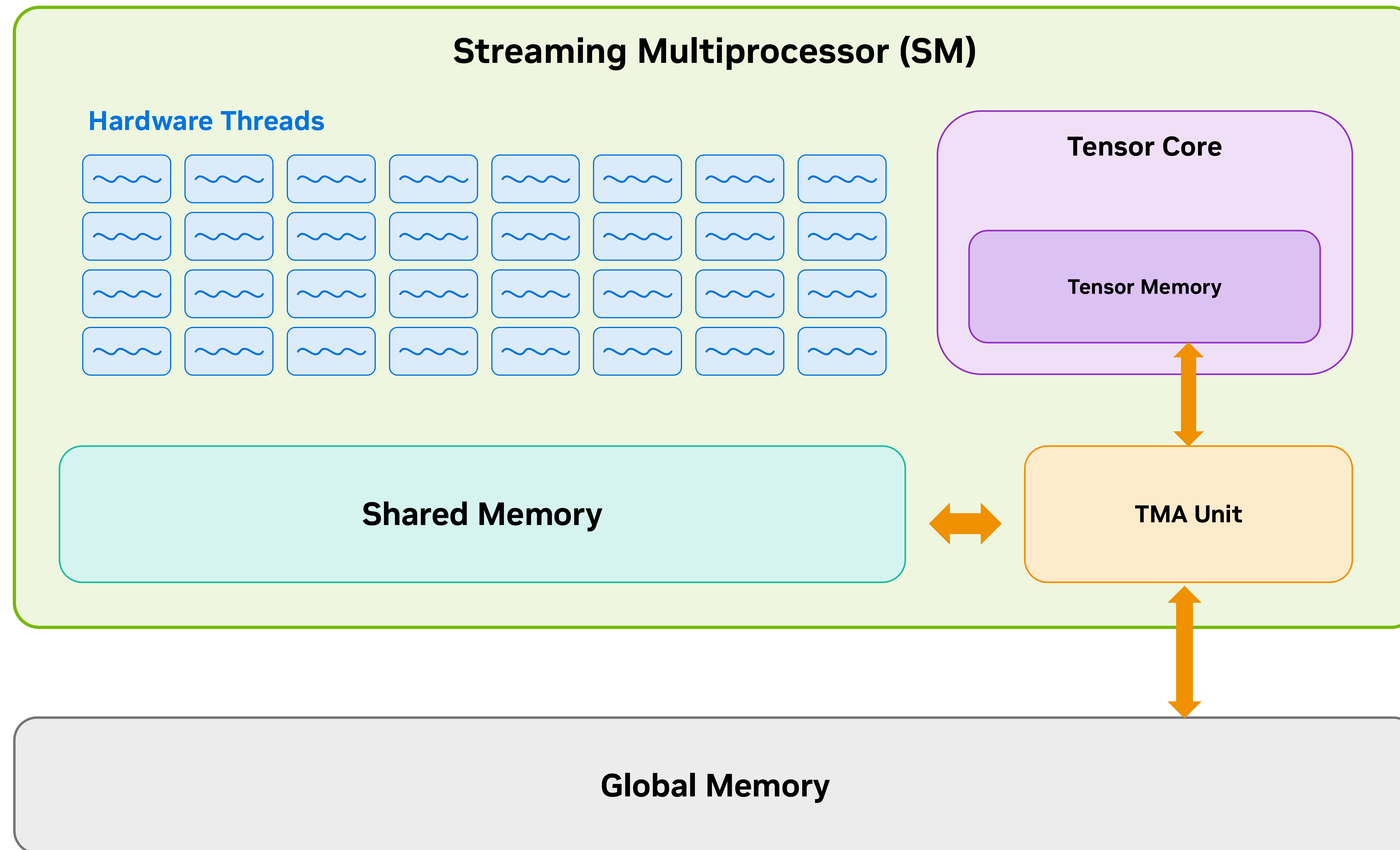
CUDA Tile C++ Moves these Responsibilities to the Compiler

- The programmer writes single threaded code using SIMD style operations on arrays
- The compiler schedules the application across the hardware resources



Unlock the power of the GPU without managing low level hardware details

Evolution of a GPU Core



- Hardware threads
- Global memory
- Shared memory
- Tensor Cores (2017)
- TMA (2022)
- Tensor Memory (2024)

Matrix Multiply in Practice

```
template<size_t TILE_SIZE>
__global__ void mma_simt_tensor_cores_ws(__half const* __restrict__ lhsPtr, __half const* __restrict__ rhsPtr,
    __grid_constant__ const CUTensorMap lhsTensorMap, __grid_constant__ const CUTensorMap rhsTensorMap,
    size_t m, size_t n, size_t k, __half* __restrict__ outPtr) {
    namespace ctd = cuda::std;
    auto lhs = ctd::mdspan(lhsPtr, ctd::extents{m, k});
    auto rhs = ctd::mdspan(rhsPtr, ctd::extents{k, n});
    auto out = ctd::mdspan(outPtr, ctd::extents{m, n});

    constexpr size_t QUEUE_SIZE = 2;
    alignas(128) __shared__ uint64_t fullBarrier[QUEUE_SIZE];
    alignas(128) __shared__ uint64_t emptyBarrier[QUEUE_SIZE];
    if (threadIdx.x == 0) {
        for (size_t idx = 0; idx != QUEUE_SIZE; ++idx) {
            cuda::ptx::mbarrier_init(&fullBarrier[idx], 5 * 32);
            cuda::ptx::mbarrier_init(&emptyBarrier[idx], 5 * 32);
        }
    }
    __syncthreads();

    alignas(128) __shared__ __half lhsShared[QUEUE_SIZE][TILE_SIZE][TILE_SIZE];
    alignas(128) __shared__ __half rhsShared[QUEUE_SIZE][TILE_SIZE][TILE_SIZE];
    size_t numTiles = k / TILE_SIZE;
    int32_t warpId = threadIdx.x / 32;

    if (warpId == 4) {
        size_t queueIdx = 0;
        for (size_t tileIdx = 0; tileIdx != numTiles; ++tileIdx) {
            uint64_t phase = cuda::ptx::mbarrier_arrive(&emptyBarrier[queueIdx]);
            while (!cuda::ptx::mbarrier_try_wait(&emptyBarrier[queueIdx], phase)) { /* spin */ };
            if (threadIdx.x % 32 == 0) {
                int32_t lhsCoord[2] = {(int32_t)TILE_SIZE * (int32_t)tileIdx, (int32_t)TILE_SIZE * (int32_t)blockIdx.x};
                cuda::ptx::cp_async_bulk_tensor(cuda::ptx::space_shared_t{}, cuda::ptx::space_global_t{},
                    &lhsShared[queueIdx][0][0], &lhsTensorMap, lhsCoord, &fullBarrier[queueIdx]);
                cuda::ptx::mbarrier_expect_tx(cuda::ptx::sem_relaxed_t{}, cuda::ptx::scope_cta_t{},
                    cuda::ptx::space_shared_t{}, &fullBarrier[queueIdx], TILE_SIZE * TILE_SIZE * sizeof(__half));
                int32_t rhsCoord[2] = {(int32_t)TILE_SIZE * (int32_t)blockIdx.y, (int32_t)TILE_SIZE * (int32_t)tileIdx};
                cuda::ptx::cp_async_bulk_tensor(cuda::ptx::space_shared_t{}, cuda::ptx::space_global_t{},
                    &rhsShared[queueIdx][0][0], &rhsTensorMap, rhsCoord, &fullBarrier[queueIdx]);
                cuda::ptx::mbarrier_expect_tx(cuda::ptx::sem_relaxed_t{}, cuda::ptx::scope_cta_t{},
                    cuda::ptx::space_shared_t{}, &fullBarrier[queueIdx], TILE_SIZE * TILE_SIZE * sizeof(__half));
            }
            cuda::ptx::mbarrier_arrive(&fullBarrier[queueIdx]);
            queueIdx = (queueIdx + 1) % QUEUE_SIZE;
        }
    } else {
        for (size_t idx = 0; idx != QUEUE_SIZE; ++idx) { cuda::ptx::mbarrier_arrive(&emptyBarrier[idx]); }

        uint32_t accFragment[4] = {0, 0, 0, 0};
        int32_t warpIdx = warpId / 2;
        int32_t warpIdY = warpId % 2;
        auto cornerX = blockIdx.x * TILE_SIZE + warpIdx * TILE_SIZE / 2;
        auto cornerY = blockIdx.y * TILE_SIZE + warpIdY * TILE_SIZE / 2;
        size_t queueIdx = 0;

        for (size_t tileIdx = 0; tileIdx != numTiles; ++tileIdx) {
            uint64_t phase = cuda::ptx::mbarrier_arrive(&fullBarrier[queueIdx]);
            while (!cuda::ptx::mbarrier_try_wait(&fullBarrier[queueIdx], phase)) { /* spin */ };
            uint32_t lhsFragment[8];
```

```
uint32_t rhsFragment[8];
for (size_t subTileIdx = 0; subTileIdx != 2; ++subTileIdx) {
    __half* lhsCorner = &lhsShared[queueIdx][0][0]
        + (TILE_SIZE * TILE_SIZE / 2) * warpIdx + subTileIdx * TILE_SIZE / 2;
    __half* rhsCorner = &rhsShared[queueIdx][0][0]
        + (TILE_SIZE * TILE_SIZE / 2) * subTileIdx + warpIdY * TILE_SIZE / 2;
    asm volatile (
        "mma.load.a.sync.aligned.row.m16n16k16.shared::cta.f16 {%0, %1, %2, %3, %4, %5, %6, %7}, [%20], %22;"
        "mma.load.b.sync.aligned.row.m16n16k16.shared::cta.f16 {%8, %9, %10, %11, %12, %13, %14, %15}, [%21], %22;"
        "mma.mma.sync.aligned.row.row.m16n16k16.f16.f16 {%16, %17, %18, %19}, "
        "{%0, %1, %2, %3, %4, %5, %6, %7}, {%8, %9, %10, %11, %12, %13, %14, %15}, {%16, %17, %18, %19};"
        : "=r"(lhsFragment[0]), "=r"(lhsFragment[1]), "=r"(lhsFragment[2]), "=r"(lhsFragment[3]),
          "=r"(lhsFragment[4]), "=r"(lhsFragment[5]), "=r"(lhsFragment[6]), "=r"(lhsFragment[7]),
          "=r"(rhsFragment[0]), "=r"(rhsFragment[1]), "=r"(rhsFragment[2]), "=r"(rhsFragment[3]),
          "=r"(rhsFragment[4]), "=r"(rhsFragment[5]), "=r"(rhsFragment[6]), "=r"(rhsFragment[7]),
          "+r"(accFragment[0]), "+r"(accFragment[1]), "+r"(accFragment[2]), "+r"(accFragment[3])
        : "1"(lhsCorner), "1"(rhsCorner), "r"(int32_t(TILE_SIZE)) : "memory");
    }
    cuda::ptx::mbarrier_arrive(&emptyBarrier[queueIdx]);
    queueIdx = (queueIdx + 1) % QUEUE_SIZE;
}

__half* gOut = &out(cornerX, cornerY);
asm volatile (
    "mma.store.d.sync.aligned.row.m16n16k16.global.f16 [%4], {%0, %1, %2, %3}, %5;"
    :: "r"(accFragment[0]), "r"(accFragment[1]), "r"(accFragment[2]), "r"(accFragment[3]),
      "1"(gOut), "r"(int32_t(n)) : "memory");
}
}

void launch_mma_simt_tensor_cores_ws(cudaStream_t stream, __half const* lhs, __half const* rhs,
    size_t m, size_t n, size_t k, __half* out) {
    constexpr size_t TILE_SIZE = 2 * 16;
    CUTensorMap lhsTensorMap, rhsTensorMap;
    {
        uint32_t lhsRank = 2;
        uint64_t lhsSize[2] = {k, m};
        uint64_t lhsStride[1] = {k * sizeof(__half)};
        uint32_t lhsTileSize[2] = {TILE_SIZE, TILE_SIZE};
        uint32_t lhsElemStride[2] = {1, 1};
        checkDriverError(cuTensorMapEncodeTiled(&lhsTensorMap, CU_TENSOR_MAP_DATA_TYPE_FLOAT16, lhsRank, (void*)lhs,
            &lhsSize[0], &lhsStride[0], &lhsTileSize[0], &lhsElemStride[0], CU_TENSOR_MAP_INTERLEAVE_NONE,
            CU_TENSOR_MAP_SWIZZLE_NONE, CU_TENSOR_MAP_L2_PROMOTION_NONE, CU_TENSOR_MAP_FLOAT_OOB_FILL_NONE));
    }
    {
        uint32_t rhsRank = 2;
        uint64_t rhsSize[2] = {n, k};
        uint64_t rhsStride[1] = {n * sizeof(__half)};
        uint32_t rhsTileSize[2] = {TILE_SIZE, TILE_SIZE};
        uint32_t rhsElemStride[2] = {1, 1};
        checkDriverError(cuTensorMapEncodeTiled(&rhsTensorMap, CU_TENSOR_MAP_DATA_TYPE_FLOAT16, rhsRank, (void*)rhs,
            &rhsSize[0], &rhsStride[0], &rhsTileSize[0], &rhsElemStride[0], CU_TENSOR_MAP_INTERLEAVE_NONE,
            CU_TENSOR_MAP_SWIZZLE_NONE, CU_TENSOR_MAP_L2_PROMOTION_NONE, CU_TENSOR_MAP_FLOAT_OOB_FILL_NONE));
    }

    dim3 numBlocks(1 + (m - 1) / TILE_SIZE, 1 + (n - 1) / TILE_SIZE);
    mma_simt_tensor_cores_ws<TILE_SIZE><<<numBlocks, 32 * 5, 0, stream>>>(&lhs, &rhs, &lhsTensorMap, &rhsTensorMap,
        m, n, k, out);
}
```

Matrix Multiply in Practice

```
template<size_t TILE_SIZE>
__global__ void mma_simt_tensor_cores_ws(__half const* __restrict__ lhsPtr, __half const* __restrict__ rhsPtr,
    __grid_constant__ const CUtensorMap lhsTensorMap, __grid_constant__ const CUtensorMap rhsTensorMap,
    size_t m, size_t n, size_t k, __half* __restrict__ outPtr) {
    namespace ctd = cuda::std;
    auto lhs = ctd::mdspan(lhsPtr, ctd::extents{m, k});
    auto rhs = ctd::mdspan(rhsPtr, ctd::extents{k, n});
    auto out = ctd::mdspan(outPtr, ctd::extents{m, n});

    constexpr size_t QUEUE_SIZE = 2;
    alignas(128) __shared__ uint64_t fullBarrier[QUEUE_SIZE];
    alignas(128) __shared__ uint64_t emptyBarrier[QUEUE_SIZE];
    if (threadIdx.x == 0) {
        for (size_t idx = 0; idx != QUEUE_SIZE; ++idx) {
            cuda::ptx::mbarrier_init(&fullBarrier[idx], 5 * 32);
            cuda::ptx::mbarrier_init(&emptyBarrier[idx], 5 * 32);
        }
    }
    __syncthreads();

    alignas(128) __shared__ __half lhsShared[QUEUE_SIZE][TILE_SIZE][TILE_SIZE];
    alignas(128) __shared__ __half rhsShared[QUEUE_SIZE][TILE_SIZE][TILE_SIZE];
    size_t numTiles = k / TILE_SIZE;
    int32_t warpId = threadIdx.x / 32;

    if (warpId == 4) {
        size_t queueIdx = 0;
        for (size_t tileIdx = 0; tileIdx != numTiles; ++tileIdx) {
            uint64_t phase = cuda::ptx::mbarrier_arrive(&emptyBarrier[queueIdx]);
            while (!cuda::ptx::mbarrier_try_wait(&emptyBarrier[queueIdx], phase)) { /* spin */ };
            if (threadIdx.x % 32 == 0) {
                int32_t lhsCoord[2] = {(int32_t)TILE_SIZE * (int32_t)tileIdx, (int32_t)TILE_SIZE * (int32_t)blockIdx.x};
                cuda::ptx::cp_async_bulk_tensor(cuda::ptx::space_shared_t{}, cuda::ptx::space_global_t{},
                    &lhsShared[queueIdx][0][0], &lhsTensorMap, lhsCoord, &fullBarrier[queueIdx]);
                cuda::ptx::mbarrier_expect_tx(cuda::ptx::sem_relaxed_t{}, cuda::ptx::scope_cta_t{},
                    cuda::ptx::space_shared_t{}, &fullBarrier[queueIdx], TILE_SIZE * TILE_SIZE * sizeof(__half));
                int32_t rhsCoord[2] = {(int32_t)TILE_SIZE * (int32_t)blockIdx.y, (int32_t)TILE_SIZE * (int32_t)tileIdx};
                cuda::ptx::cp_async_bulk_tensor(cuda::ptx::space_shared_t{}, cuda::ptx::space_global_t{},
                    &rhsShared[queueIdx][0][0], &rhsTensorMap, rhsCoord, &fullBarrier[queueIdx]);
                cuda::ptx::mbarrier_expect_tx(cuda::ptx::sem_relaxed_t{}, cuda::ptx::scope_cta_t{},
                    cuda::ptx::space_shared_t{}, &fullBarrier[queueIdx], TILE_SIZE * TILE_SIZE * sizeof(__half));
            }
            cuda::ptx::mbarrier_arrive(&fullBarrier[queueIdx]);
            queueIdx = (queueIdx + 1) % QUEUE_SIZE;
        }
    } else {
        for (size_t idx = 0; idx != QUEUE_SIZE; ++idx) { cuda::ptx::mbarrier_arrive(&emptyBarrier[idx]); }

        uint32_t accFragment[4] = {0, 0, 0, 0};
        int32_t warpIdx = warpId / 2;
        int32_t warpIdY = warpId % 2;
        auto cornerX = blockIdx.x * TILE_SIZE + warpIdx * TILE_SIZE / 2;
        auto cornerY = blockIdx.y * TILE_SIZE + warpIdY * TILE_SIZE / 2;
        size_t queueIdx = 0;

        for (size_t tileIdx = 0; tileIdx != numTiles; ++tileIdx) {
            uint64_t phase = cuda::ptx::mbarrier_arrive(&fullBarrier[queueIdx]);
            while (!cuda::ptx::mbarrier_try_wait(&fullBarrier[queueIdx], phase)) { /* spin */ };
            uint32_t lhsFragment[8];
```

[illegible]

Matrix Multiply in Practice

```
template<size_t TILE_SIZE>
__global__ void mma_simt_tensor_cores_ws(__half const* __restrict__ lhsPtr, __half const* __restrict__ rhsPtr,
    __grid_constant__ const CUTensorMap lhsTensorMap, __grid_constant__ const CUTensorMap rhsTensorMap,
    size_t m, size_t n, size_t k, __half* __restrict__ outPtr) {
    namespace ctd = cuda::std;
    auto lhs = ctd::mdspan(lhsPtr, ctd::extents{m, k});
    auto rhs = ctd::mdspan(rhsPtr, ctd::extents{k, n});
    auto out = ctd::mdspan(outPtr, ctd::extents{m, n});

    constexpr size_t QUEUE_SIZE = 2;
    alignas(128) __shared__ uint64_t fullBarrier[QUEUE_SIZE];
    alignas(128) __shared__ uint64_t emptyBarrier[QUEUE_SIZE];
    if (threadIdx.x == 0) {
        for (size_t queueIdx = 0; queueIdx < QUEUE_SIZE; ++queueIdx) {
            fullBarrier[queueIdx] = 0;
            emptyBarrier[queueIdx] = 0;
        }
    }
    __syncthreads();

    if (warpId == 4) {
        size_t queueIdx = 0;
        for (size_t tileIdx = 0; tileIdx != numTiles; ++tileIdx) {
            uint64_t phase = cuda::ptx::mbarrier_arrive(&emptyBarrier[queueIdx]);
            while (!cuda::ptx::mbarrier_try_wait(&emptyBarrier[queueIdx], phase)) { /* spin */ }
            if (threadIdx.x % 32 == 0) {
                int32_t lhsCoord[2] = {(int32_t)TILE_SIZE * (int32_t)tileIdx, (int32_t)TILE_SIZE * (int32_t)blockIdx.x};
                cuda::ptx::cp_async_bulk_tensor(cuda::ptx::space_shared_t{}, cuda::ptx::space_global_t{},
                    &lhsShared[queueIdx][0][0], &lhsTensorMap, lhsCoord, &fullBarrier[queueIdx]);
                cuda::ptx::mbarrier_expect_tx(cuda::ptx::sem_relaxed_t{}, cuda::ptx::scope_cta_t{},
                    cuda::ptx::space_shared_t{}, &fullBarrier[queueIdx], TILE_SIZE * TILE_SIZE * sizeof(__half));
                int32_t rhsCoord[2] = {(int32_t)TILE_SIZE * (int32_t)blockIdx.y, (int32_t)TILE_SIZE * (int32_t)tileIdx};
                cuda::ptx::cp_async_bulk_tensor(cuda::ptx::space_shared_t{}, cuda::ptx::space_global_t{},
                    &rhsShared[queueIdx][0][0], &rhsTensorMap, rhsCoord, &fullBarrier[queueIdx]);
                cuda::ptx::mbarrier_expect_tx(cuda::ptx::sem_relaxed_t{}, cuda::ptx::scope_cta_t{},
                    cuda::ptx::space_shared_t{}, &fullBarrier[queueIdx], TILE_SIZE * TILE_SIZE * sizeof(__half));
            }
            cuda::ptx::mbarrier_arrive(&fullBarrier[queueIdx]);
            queueIdx = (queueIdx + 1) % QUEUE_SIZE;
        }
    } else {
        for (size_t idx = 0; idx != QUEUE_SIZE; ++idx) { cuda::ptx::mbarrier_arrive(&emptyBarrier[idx]); }

        uint32_t accFragment[4] = {0, 0, 0, 0};
        int32_t warpIdx = warpId / 2;
        int32_t warpIdY = warpId % 2;
        auto cornerX = blockIdx.x * TILE_SIZE + warpIdx * TILE_SIZE / 2;
        auto cornerY = blockIdx.y * TILE_SIZE + warpIdY * TILE_SIZE / 2;
        size_t queueIdx = 0;

        for (size_t tileIdx = 0; tileIdx != numTiles; ++tileIdx) {
            uint64_t phase = cuda::ptx::mbarrier_arrive(&fullBarrier[queueIdx]);
            while (!cuda::ptx::mbarrier_try_wait(&fullBarrier[queueIdx], phase)) { /* spin */ }
            uint32_t lhsFragment[8];
            uint32_t rhsFragment[8];
            for (size_t subTileIdx = 0; subTileIdx != 2; ++subTileIdx) {
                __half* lhsCorner = &lhsShared[queueIdx][0][0]
                    + (TILE_SIZE * TILE_SIZE / 2) * warpIdx + subTileIdx * TILE_SIZE / 2;
                __half* rhsCorner = &rhsShared[queueIdx][0][0]
                    + (TILE_SIZE * TILE_SIZE / 2) * subTileIdx + warpIdx * TILE_SIZE / 2;

                asm volatile (
                    "mma.load.a.sync.aligned.row.m16n16k16.shared::cta.f16 {%0, %1, %2, %3, %4, %5, %6, %7}, [%20], %22;"
                    "mma.load.b.sync.aligned.row.m16n16k16.shared::cta.f16 {%8, %9, %10, %11, %12, %13, %14, %15}, [%21], %22;"
                    "mma.mma.sync.aligned.row.m16n16k16.f16.f16 {%16, %17, %18, %19}, "
                    "{%0, %1, %2, %3, %4, %5, %6, %7}, {%8, %9, %10, %11, %12, %13, %14, %15}, {%16, %17, %18, %19};"
                    : "=r"(lhsFragment[0]), "=r"(lhsFragment[1]), "=r"(lhsFragment[2]), "=r"(lhsFragment[3]),
                      "=r"(lhsFragment[4]), "=r"(lhsFragment[5]), "=r"(lhsFragment[6]), "=r"(lhsFragment[7]),
                      "=r"(rhsFragment[0]), "=r"(rhsFragment[1]), "=r"(rhsFragment[2]), "=r"(rhsFragment[3]),
                      "=r"(rhsFragment[4]), "=r"(rhsFragment[5]), "=r"(rhsFragment[6]), "=r"(rhsFragment[7]),
                      "+r"(accFragment[0]), "+r"(accFragment[1]), "+r"(accFragment[2]), "+r"(accFragment[3])
                    : "1"(lhsCorner), "1"(rhsCorner), "r"(int32_t(TILE_SIZE)) : "memory");

                cuda::ptx::mbarrier_arrive(&emptyBarrier[queueIdx]);
                queueIdx = (queueIdx + 1) % QUEUE_SIZE;
            }

            __half* gOut = &out(cornerX, cornerY);
            asm volatile (
                "mma.store.d.sync.aligned.row.m16n16k16.global.f16 [%4], {%0, %1, %2, %3}, %5;"
                :: "r"(accFragment[0]), "r"(accFragment[1]), "r"(accFragment[2]), "r"(accFragment[3]),
                  "1"(gOut), "r"(int32_t(n)) : "memory");
        }
    }
}

void launch_mma_simt_tensor_cores_ws(cudaStream_t stream, __half const* lhs, __half const* rhs,
    size_t m, size_t n, size_t k, __half* out) {
    constexpr size_t TILE_SIZE = 2 * 16;
    CUTensorMap lhsTensorMap, rhsTensorMap;
    {
        uint32_t lhsRank = 2;
        uint64_t lhsSize[2] = {k, m};
        uint64_t lhsStride[1] = {k * sizeof(__half)};
        uint32_t lhsTileSize[2] = {TILE_SIZE, TILE_SIZE};
        uint32_t lhsElemStride[2] = {1, 1};
        checkDriverError(cuTensorMapEncodeTiled(&lhsTensorMap, CU_TENSOR_MAP_DATA_TYPE_FLOAT16, lhsRank, (void*)lhs,
            &lhsSize[0], &lhsStride[0], &lhsTileSize[0], &lhsElemStride[0], CU_TENSOR_MAP_INTERLEAVE_NONE,
            CU_TENSOR_MAP_SWIZZLE_NONE, CU_TENSOR_MAP_L2_PROMOTION_NONE, CU_TENSOR_MAP_FLOAT_OOB_FILL_NONE));
    }
    {
        uint32_t rhsRank = 2;
        uint64_t rhsSize[2] = {n, k};
        uint64_t rhsStride[1] = {n * sizeof(__half)};
        uint32_t rhsTileSize[2] = {TILE_SIZE, TILE_SIZE};
        uint32_t rhsElemStride[2] = {1, 1};
        checkDriverError(cuTensorMapEncodeTiled(&rhsTensorMap, CU_TENSOR_MAP_DATA_TYPE_FLOAT16, rhsRank, (void*)rhs,
            &rhsSize[0], &rhsStride[0], &rhsTileSize[0], &rhsElemStride[0], CU_TENSOR_MAP_INTERLEAVE_NONE,
            CU_TENSOR_MAP_SWIZZLE_NONE, CU_TENSOR_MAP_L2_PROMOTION_NONE, CU_TENSOR_MAP_FLOAT_OOB_FILL_NONE));
    }

    dim3 numBlocks(1 + (m - 1) / TILE_SIZE, 1 + (n - 1) / TILE_SIZE);
    mma_simt_tensor_cores_ws<TILE_SIZE><<<numBlocks, 32 * 5, 0, stream>>>(&lhs, &rhs, &lhsTensorMap, &rhsTensorMap,
        m, n, k, out);
}
```

Tensor Core Instructions

```
uint32_t rhsFragment[8];
for (size_t subTileIdx = 0; subTileIdx != 2; ++subTileIdx) {
    __half* lhsCorner = &lhsShared[queueIdx][0][0]
        + (TILE_SIZE * TILE_SIZE / 2) * warpIdx + subTileIdx * TILE_SIZE / 2;
    __half* rhsCorner = &rhsShared[queueIdx][0][0]
        + (TILE_SIZE * TILE_SIZE / 2) * subTileIdx + warpIdx * TILE_SIZE / 2;

    asm volatile (
        "mma.load.a.sync.aligned.row.m16n16k16.shared::cta.f16 {%0, %1, %2, %3, %4, %5, %6, %7}, [%20], %22;"
        "mma.load.b.sync.aligned.row.m16n16k16.shared::cta.f16 {%8, %9, %10, %11, %12, %13, %14, %15}, [%21], %22;"
        "mma.mma.sync.aligned.row.m16n16k16.f16.f16 {%16, %17, %18, %19}, "
        "{%0, %1, %2, %3, %4, %5, %6, %7}, {%8, %9, %10, %11, %12, %13, %14, %15}, {%16, %17, %18, %19};"
        : "=r"(lhsFragment[0]), "=r"(lhsFragment[1]), "=r"(lhsFragment[2]), "=r"(lhsFragment[3]),
          "=r"(lhsFragment[4]), "=r"(lhsFragment[5]), "=r"(lhsFragment[6]), "=r"(lhsFragment[7]),
          "=r"(rhsFragment[0]), "=r"(rhsFragment[1]), "=r"(rhsFragment[2]), "=r"(rhsFragment[3]),
          "=r"(rhsFragment[4]), "=r"(rhsFragment[5]), "=r"(rhsFragment[6]), "=r"(rhsFragment[7]),
          "+r"(accFragment[0]), "+r"(accFragment[1]), "+r"(accFragment[2]), "+r"(accFragment[3])
        : "1"(lhsCorner), "1"(rhsCorner), "r"(int32_t(TILE_SIZE)) : "memory");

    cuda::ptx::mbarrier_arrive(&emptyBarrier[queueIdx]);
    queueIdx = (queueIdx + 1) % QUEUE_SIZE;
}

__half* gOut = &out(cornerX, cornerY);
asm volatile (
    "mma.store.d.sync.aligned.row.m16n16k16.global.f16 [%4], {%0, %1, %2, %3}, %5;"
    :: "r"(accFragment[0]), "r"(accFragment[1]), "r"(accFragment[2]), "r"(accFragment[3]),
      "1"(gOut), "r"(int32_t(n)) : "memory");
}

void launch_mma_simt_tensor_cores_ws(cudaStream_t stream, __half const* lhs, __half const* rhs,
    size_t m, size_t n, size_t k, __half* out) {
    constexpr size_t TILE_SIZE = 2 * 16;
    CUTensorMap lhsTensorMap, rhsTensorMap;
    {
        uint32_t lhsRank = 2;
        uint64_t lhsSize[2] = {k, m};
        uint64_t lhsStride[1] = {k * sizeof(__half)};
        uint32_t lhsTileSize[2] = {TILE_SIZE, TILE_SIZE};
        uint32_t lhsElemStride[2] = {1, 1};
        checkDriverError(cuTensorMapEncodeTiled(&lhsTensorMap, CU_TENSOR_MAP_DATA_TYPE_FLOAT16, lhsRank, (void*)lhs,
            &lhsSize[0], &lhsStride[0], &lhsTileSize[0], &lhsElemStride[0], CU_TENSOR_MAP_INTERLEAVE_NONE,
            CU_TENSOR_MAP_SWIZZLE_NONE, CU_TENSOR_MAP_L2_PROMOTION_NONE, CU_TENSOR_MAP_FLOAT_OOB_FILL_NONE));
    }
    {
        uint32_t rhsRank = 2;
        uint64_t rhsSize[2] = {n, k};
        uint64_t rhsStride[1] = {n * sizeof(__half)};
        uint32_t rhsTileSize[2] = {TILE_SIZE, TILE_SIZE};
        uint32_t rhsElemStride[2] = {1, 1};
        checkDriverError(cuTensorMapEncodeTiled(&rhsTensorMap, CU_TENSOR_MAP_DATA_TYPE_FLOAT16, rhsRank, (void*)rhs,
            &rhsSize[0], &rhsStride[0], &rhsTileSize[0], &rhsElemStride[0], CU_TENSOR_MAP_INTERLEAVE_NONE,
            CU_TENSOR_MAP_SWIZZLE_NONE, CU_TENSOR_MAP_L2_PROMOTION_NONE, CU_TENSOR_MAP_FLOAT_OOB_FILL_NONE));
    }

    dim3 numBlocks(1 + (m - 1) / TILE_SIZE, 1 + (n - 1) / TILE_SIZE);
    mma_simt_tensor_cores_ws<TILE_SIZE><<<numBlocks, 32 * 5, 0, stream>>>(&lhs, &rhs, &lhsTensorMap, &rhsTensorMap,
        m, n, k, out);
}
```

Matrix Multiply in Practice

```

template<size_t TILE_SIZE>
__global__ void mma_simt_tensor_cores_ws(__half const* __restrict__ lhsPtr, __half const* __restrict__ rhsPtr,
    __grid_constant__ const CUTensorMap lhsTensorMap, __grid_constant__ const CUTensorMap rhsTensorMap,
    size_t m, size_t n, size_t k, __half* __restrict__ outPtr) {
    namespace ctd = cuda::std;
    auto lhs = ctd::mdspan(lhsPtr, ctd::extents{m, k});
    auto rhs = ctd::mdspan(rhsPtr, ctd::extents{k, n});
    auto out = ctd::mdspan(outPtr, ctd::extents{m, n});

    constexpr size_t QUEUE_SIZE = 2;
    alignas(128) __shared__ uint64_t fullBarrier[QUEUE_SIZE];
    alignas(128) __shared__ uint64_t emptyBarrier[QUEUE_SIZE];
    if (threadIdx.x == 0) {
        for (size_t idx = 0; idx != QUEUE_SIZE; ++idx) {
            cuda::ptx::mbarrier_init(&fullBarrier[idx], 5 * 32);
            cuda::ptx::mbarrier_init(&emptyBarrier[idx], 5 * 32);
        }
    }
    __syncthreads();

    alignas(128) __shared__ __half lhsShared[QUEUE_SIZE][TILE_SIZE][TILE_SIZE];
    alignas(128) __shared__ __half rhsShared[QUEUE_SIZE][TILE_SIZE][TILE_SIZE];
    size_t numTiles = k / TILE_SIZE;
    int32_t warpId = threadIdx.x / 32;

    if (warpId == 4) {
        size_t queueIdx = 0;
        for (size_t tileIdx = 0; tileIdx != numTiles; ++tileIdx) {
            uint64_t phase = cuda::ptx::mbarrier_arrive(&emptyBarrier[queueIdx]);
            while (!cuda::ptx::mbarrier_try_wait(&emptyBarrier[queueIdx], phase)) { /* spin */ };
            if (threadIdx.x % 32 == 0) {
                int32_t lhsCoord[2] = {(int32_t)TILE_SIZE * (int32_t)tileIdx, (int32_t)TILE_SIZE * (int32_t)blockIdx.x};
                cuda::ptx::cp_async_bulk_tensor(cuda::ptx::space_shared_t{}, cuda::ptx::space_global_t{},
                    &lhsShared[queueIdx][0][0], &lhsTensorMap, lhsCoord, &fullBarrier[queueIdx]);
                cuda::ptx::mbarrier_expect_tx(cuda::ptx::sem_relaxed_t{}, cuda::ptx::scope_cta_t{},
                    cuda::ptx::space_shared_t{}, &fullBarrier[queueIdx], TILE_SIZE * TILE_SIZE * sizeof(__half));
                int32_t rhsCoord[2] = {(int32_t)TILE_SIZE * (int32_t)blockIdx.y, (int32_t)TILE_SIZE * (int32_t)tileIdx};
                cuda::ptx::cp_async_bulk_tensor(cuda::ptx::space_shared_t{}, cuda::ptx::space_global_t{},
                    &rhsShared[queueIdx][0][0], &rhsTensorMap, rhsCoord, &fullBarrier[queueIdx]);
                cuda::ptx::mbarrier_expect_tx(cuda::ptx::sem_relaxed_t{}, cuda::ptx::scope_cta_t{},
                    cuda::ptx::space_shared_t{}, &fullBarrier[queueIdx], TILE_SIZE * TILE_SIZE * sizeof(__half));
            }
            cuda::ptx::mbarrier_arrive(&fullBarrier[queueIdx]);
            queueIdx = (queueIdx + 1) % QUEUE_SIZE;
        }
    } else {
        for (size_t idx = 0; idx != QUEUE_SIZE; ++idx) { cuda::ptx::mbarrier_arrive(&emptyBarrier[idx]); }

        uint32_t accFragment[4] = {0, 0, 0, 0};
        int32_t warpIdx = warpId / 2;
        int32_t warpIdY = warpId % 2;
        auto cornerX = blockIdx.x * TILE_SIZE + warpIdx * TILE_SIZE / 2;
        auto cornerY = blockIdx.y * TILE_SIZE + warpIdY * TILE_SIZE / 2;
        size_t queueIdx = 0;

        for (size_t tileIdx = 0; tileIdx != numTiles; ++tileIdx) {
            uint64_t phase = cuda::ptx::mbarrier_arrive(&fullBarrier[queueIdx]);
            while (!cuda::ptx::mbarrier_try_wait(&fullBarrier[queueIdx], phase)) { /* spin */ };
            uint32_t lhsFragment[8];

```

```
uint32_t rhsFragment[8];
for (size_t subTileIdx = 0; subTileIdx != 2; ++subTileIdx) {
    __half* lhsCorner = &lhsShared[queueIdx][0][0]
        + (TILE_SIZE * TILE_SIZE / 2) * warpIdX + subTileIdx * TILE_SIZE / 2;
    __half* rhsCorner = &rshsShared[queueIdx][0][0]
        + (TILE_SIZE * TILE_SIZE / 2) * subTileIdx + warpIdY * TILE_SIZE / 2;
    asm volatile (
        "wmma.load.a.sync.aligned.row.m16n16k16.shared::cta.f16 {%0, %1, %2, %3, %4, %5, %6, %7}, [%20], %22;"
        "wmma.load.b.sync.aligned.row.m16n16k16.shared::cta.f16 {%8, %9, %10, %11, %12, %13, %14, %15}, [%21], %22;"
        "wmma.mma.sync.aligned.row.row.m16n16k16.f16.f16 {%16, %17, %18, %19}, ",
        "{%0, %1, %2, %3, %4, %5, %6, %7}, {%8, %9, %10, %11, %12, %13, %14, %15}, {%16, %17, %18, %19};"
        : "=r"(lhsFragment[0]), "=r"(lhsFragment[1]), "=r"(lhsFragment[2]), "=r"(lhsFragment[3]),
          "=r"(lhsFragment[4]), "=r"(lhsFragment[5]), "=r"(lhsFragment[6]), "=r"(lhsFragment[7]),
          "=r"(rhsFragment[0]), "=r"(rhsFragment[1]), "=r"(rhsFragment[2]), "=r"(rhsFragment[3]),
          "=r"(rhsFragment[4]), "=r"(rhsFragment[5]), "=r"(rhsFragment[6]), "=r"(rhsFragment[7]),
          "+r"(accFragment[0]), "+r"(accFragment[1]), "+r"(accFragment[2]), "+r"(accFragment[3])
        : "l"(lhsCorner), "l"(rhsCorner), "r"(int32_t(TILE_SIZE)) : "memory");
    }
    cuda::ptx::mbarrier_arrive(&emptyBarrier[queueIdx]);
    queueIdx = (queueIdx + 1) % QUEUE_SIZE;
}

__half* gOut = &out(cornerX, cornerY);
asm volatile (
    "wmma.store.d.sync.aligned.row.m16n16k16.global.f16 [%4], {%0, %1, %2, %3}, %5;"
    :: "r"(accFragment[0]), "r"(accFragment[1]), "r"(accFragment[2]), "r"(accFragment[3]),
      "l"(gOut), "r"(int32_t(n)) : "memory");
}

void launch_mma(
    constexpr size_t m, constexpr size_t n, constexpr size_t k,
    CUtensorMap lhsTensorMap, CUtensorMap rhsTensorMap, void* lhs, void* rhs,
    unsigned int lhsRank, unsigned int rhsRank, void* lws, void* rws,
    CUtensorMap lhsTensorMapEnc, CUtensorMap rhsTensorMapEnc,
    CU_TENSOR_MAP_SWIZZLE_NONE, CU_TENSOR_MAP_L2_PROMOTION_NONE, CU_TENSOR_MAP_FLOAT_OOB_FILL_NONE));

{
    uint32_t rhsRank = 2;
    uint64_t rhsSize[2] = {n, k};
    uint64_t rhsStride[1] = {n * sizeof(__half)};
    uint32_t rhsTileSize[2] = {TILE_SIZE, TILE_SIZE};
    uint32_t rhsElemStride[2] = {1, 1};
    checkDriverError(cuTensorMapEncodeTiled(&rhsTensorMap, CU_TENSOR_MAP_DATA_TYPE_FLOAT16, rhsRank, (void*)rhs,
        &rhsSize[0], &rhsStride[0], &rhsTileSize[0], &rhsElemStride[0], CU_TENSOR_MAP_INTERLEAVE_NONE,
        CU_TENSOR_MAP_SWIZZLE_NONE, CU_TENSOR_MAP_L2_PROMOTION_NONE, CU_TENSOR_MAP_FLOAT_OOB_FILL_NONE));
}

dim3 numBlocks((m - 1) / TILE_SIZE + 1, (n - 1) / TILE_SIZE + 1, 1);
mma_simt_tensor_cores_ws<TILE_SIZE><<<numBlocks, 32 * 5, 0, stream>>>(lhs, rhs, lhsTensorMap, rhsTensorMap,
    m, n, k, out);
}
```

Matrix Multiply in Practice

```
template<size_t TILE_SIZE>
__global__ void mma_simt_tensor_cores_ws(__half const* __restrict__ lhsPtr, __half const* __restrict__ rhsPtr,
    __grid_constant__ const CUTensorMap lhsTensorMap, __grid_constant__ const CUTensorMap rhsTensorMap,
    size_t m, size_t n, size_t k, __half* __restrict__ outPtr) {
    namespace ctd = cuda::std;
    auto lhs = ctd::mdspan(lhsPtr, ctd::extents{m, k});
    auto rhs = ctd::mdspan(rhsPtr, ctd::extents{k, n});
    auto out = ctd::mdspan(outPtr, ctd::extents{m, n});

    constexpr size_t QUEUE_SIZE = 2;
    alignas(128) __shared__ uint64_t fullBarrier[QUEUE_SIZE];
    alignas(128) __shared__ uint64_t emptyBarrier[QUEUE_SIZE];
    if (threadIdx.x == 0) {
        for (size_t idx = 0; idx != QUEUE_SIZE; ++idx) {
            cuda::ptx::mbarrier_init(&fullBarrier[idx], 5 * 32);
            cuda::ptx::mbarrier_init(&emptyBarrier[idx], 5 * 32);
        }
    }
    __syncthreads();

    alignas(128) __shared__ __half lhsShared[QUEUE_SIZE][TILE_SIZE][TILE_SIZE];
    alignas(128) __shared__ __half rhsShared[QUEUE_SIZE][TILE_SIZE][TILE_SIZE];
    size_t numTiles = k / TILE_SIZE;
    int32_t warpId = threadIdx.x / 32;

    if (warpId == 4) {
        size_t queueIdx = 0;
        for (size_t tileIdx = 0; tileIdx != numTiles; ++tileIdx) {
            uint64_t phase = cuda::ptx::mbarrier_arrive(&emptyBarrier[queueIdx]);
            while (!cuda::ptx::mbarrier_try_wait(&emptyBarrier[queueIdx], phase)) { /* spin */ };
            if (threadIdx.x % 32 == 0) {
                int32_t lhsCoord[2] = {(int32_t)TILE_SIZE * (int32_t)tileIdx, (int32_t)TILE_SIZE * (int32_t)blockIdx.x};
                cuda::ptx::cp_async_bulk_tensor(cuda::ptx::space_shared_t{}, cuda::ptx::space_global_t{},
                    &lhsShared[queueIdx][0][0], &lhsTensorMap, lhsCoord, &fullBarrier[queueIdx]);
                cuda::ptx::mbarrier_expect_tx(cuda::ptx::sem_relaxed_t{}, cuda::ptx::scope_cta_t{},
                    cuda::ptx::space_shared_t{}, &fullBarrier[queueIdx], TILE_SIZE * TILE_SIZE * sizeof(__half));
                int32_t rhsCoord[2] = {(int32_t)TILE_SIZE * (int32_t)blockIdx.y, (int32_t)TILE_SIZE * (int32_t)tileIdx};
                cuda::ptx::cp_async_bulk_tensor(cuda::ptx::space_shared_t{}, cuda::ptx::space_global_t{},
                    &rhsShared[queueIdx][0][0], &rhsTensorMap, rhsCoord, &fullBarrier[queueIdx]);
                cuda::ptx::mbarrier_expect_tx(cuda::ptx::sem_relaxed_t{}, cuda::ptx::scope_cta_t{},
                    cuda::ptx::space_shared_t{}, &fullBarrier[queueIdx], TILE_SIZE * TILE_SIZE * sizeof(__half));
            }
            cuda::ptx::mbarrier_wait(&fullBarrier[queueIdx], phase);
        }
    }
    else {
        for (size_t tileIdx = 0; tileIdx != numTiles; ++tileIdx) {
            uint64_t phase = cuda::ptx::mbarrier_arrive(&fullBarrier[queueIdx]);
            while (!cuda::ptx::mbarrier_try_wait(&fullBarrier[queueIdx], phase)) { /* spin */ };
            uint32_t lhsFragment[8];
            int32_t warpIdx = warpId / 2;
            int32_t warpIdY = warpId % 2;
            auto cornerX = blockIdx.x * TILE_SIZE + warpIdx * TILE_SIZE / 2;
            auto cornerY = blockIdx.y * TILE_SIZE + warpIdY * TILE_SIZE / 2;
            size_t queueIdx = 0;

            for (size_t subTileIdx = 0; subTileIdx != 2; ++subTileIdx) {
                __half* lhsCorner = &lhsShared[queueIdx][0][0]
                    + (TILE_SIZE * TILE_SIZE / 2) * warpIdx + subTileIdx * TILE_SIZE / 2;
                __half* rhsCorner = &rhsShared[queueIdx][0][0]
                    + (TILE_SIZE * TILE_SIZE / 2) * subTileIdx + warpIdY * TILE_SIZE / 2;
                asm volatile (
                    "mma.load.a.sync.aligned.row.m16n16k16.shared::cta.f16 {%0, %1, %2, %3, %4, %5, %6, %7}, [%20], %22;"
                    "mma.load.b.sync.aligned.row.m16n16k16.shared::cta.f16 {%8, %9, %10, %11, %12, %13, %14, %15}, [%21], %22;"
                    "mma.mma.sync.aligned.row.row.m16n16k16.f16.f16 {%16, %17, %18, %19}, "
                    "{%0, %1, %2, %3, %4, %5, %6, %7}, {%8, %9, %10, %11, %12, %13, %14, %15}, {%16, %17, %18, %19};"
                    : "=r"(lhsFragment[0]), "=r"(lhsFragment[1]), "=r"(lhsFragment[2]), "=r"(lhsFragment[3]),
                      "=r"(lhsFragment[4]), "=r"(lhsFragment[5]), "=r"(lhsFragment[6]), "=r"(lhsFragment[7]),
                      "=r"(rhsFragment[0]), "=r"(rhsFragment[1]), "=r"(rhsFragment[2]), "=r"(rhsFragment[3]),
                      "=r"(rhsFragment[4]), "=r"(rhsFragment[5]), "=r"(rhsFragment[6]), "=r"(rhsFragment[7]),
                      "+r"(accFragment[0]), "+r"(accFragment[1]), "+r"(accFragment[2]), "+r"(accFragment[3])
                    : "1"(lhsCorner), "1"(rhsCorner), "r"(int32_t)(TILE_SIZE)) : "memory");
                cuda::ptx::mbarrier_arrive(&emptyBarrier[queueIdx]);
                queueIdx = (queueIdx + 1) % QUEUE_SIZE;
            }

            __half* gOut = &out(cornerX, cornerY);
            asm volatile (
                "mma.store.d.sync.aligned.row.m16n16k16.global.f16 [%4], {%0, %1, %2, %3}, %5;"
                :: "r"(accFragment[0]), "r"(accFragment[1]), "r"(accFragment[2]), "r"(accFragment[3]),
                  "1"(gOut), "r"(int32_t)(n)) : "memory");
        }
    }
}
```

TMA Descriptors

```
uint32_t rhsFragment[8];
for (size_t subTileIdx = 0; subTileIdx != 2; ++subTileIdx) {
    __half* lhsCorner = &lhsShared[queueIdx][0][0]
        + (TILE_SIZE * TILE_SIZE / 2) * warpIdx + subTileIdx * TILE_SIZE / 2;
    __half* rhsCorner = &rhsShared[queueIdx][0][0]
        + (TILE_SIZE * TILE_SIZE / 2) * subTileIdx + warpIdY * TILE_SIZE / 2;
    asm volatile (
        "mma.load.a.sync.aligned.row.m16n16k16.shared::cta.f16 {%0, %1, %2, %3, %4, %5, %6, %7}, [%20], %22;"
        "mma.load.b.sync.aligned.row.m16n16k16.shared::cta.f16 {%8, %9, %10, %11, %12, %13, %14, %15}, [%21], %22;"
        "mma.mma.sync.aligned.row.row.m16n16k16.f16.f16 {%16, %17, %18, %19}, "
        "{%0, %1, %2, %3, %4, %5, %6, %7}, {%8, %9, %10, %11, %12, %13, %14, %15}, {%16, %17, %18, %19};"
        : "=r"(lhsFragment[0]), "=r"(lhsFragment[1]), "=r"(lhsFragment[2]), "=r"(lhsFragment[3]),
          "=r"(lhsFragment[4]), "=r"(lhsFragment[5]), "=r"(lhsFragment[6]), "=r"(lhsFragment[7]),
          "=r"(rhsFragment[0]), "=r"(rhsFragment[1]), "=r"(rhsFragment[2]), "=r"(rhsFragment[3]),
          "=r"(rhsFragment[4]), "=r"(rhsFragment[5]), "=r"(rhsFragment[6]), "=r"(rhsFragment[7]),
          "+r"(accFragment[0]), "+r"(accFragment[1]), "+r"(accFragment[2]), "+r"(accFragment[3])
        : "1"(lhsCorner), "1"(rhsCorner), "r"(int32_t)(TILE_SIZE)) : "memory");
    cuda::ptx::mbarrier_arrive(&emptyBarrier[queueIdx]);
    queueIdx = (queueIdx + 1) % QUEUE_SIZE;
}

__half* gOut = &out(cornerX, cornerY);
asm volatile (
    "mma.store.d.sync.aligned.row.m16n16k16.global.f16 [%4], {%0, %1, %2, %3}, %5;"
    :: "r"(accFragment[0]), "r"(accFragment[1]), "r"(accFragment[2]), "r"(accFragment[3]),
      "1"(gOut), "r"(int32_t)(n)) : "memory");
}

void launch_mma_simt_tensor_cores_ws(cudaStream_t stream, __half const* lhs, __half const* rhs,
    size_t m, size_t n, size_t k, __half* out) {
    constexpr size_t TILE_SIZE = 2 * 16;
    CUTensorMap lhsTensorMap, rhsTensorMap;
    {
        uint32_t lhsRank = 2;
        uint64_t lhsSize[2] = {k, m};
        uint64_t lhsStride[1] = {k * sizeof(__half)};
        uint32_t lhsTileSize[2] = {TILE_SIZE, TILE_SIZE};
        uint32_t lhsElemStride[2] = {1, 1};
        checkDriverError(cuTensorMapEncodeTiled(&lhsTensorMap, CU_TENSOR_MAP_DATA_TYPE_FLOAT16, lhsRank, (void*)lhs,
            &lhsSize[0], &lhsStride[0], &lhsTileSize[0], &lhsElemStride[0], CU_TENSOR_MAP_INTERLEAVE_NONE,
            CU_TENSOR_MAP_SWIZZLE_NONE, CU_TENSOR_MAP_L2_PROMOTION_NONE, CU_TENSOR_MAP_FLOAT_OOB_FILL_NONE));
    }
    {
        uint32_t rhsRank = 2;
        uint64_t rhsSize[2] = {n, k};
        uint64_t rhsStride[1] = {n * sizeof(__half)};
        uint32_t rhsTileSize[2] = {TILE_SIZE, TILE_SIZE};
        uint32_t rhsElemStride[2] = {1, 1};
        checkDriverError(cuTensorMapEncodeTiled(&rhsTensorMap, CU_TENSOR_MAP_DATA_TYPE_FLOAT16, rhsRank, (void*)rhs,
            &rhsSize[0], &rhsStride[0], &rhsTileSize[0], &rhsElemStride[0], CU_TENSOR_MAP_INTERLEAVE_NONE,
            CU_TENSOR_MAP_SWIZZLE_NONE, CU_TENSOR_MAP_L2_PROMOTION_NONE, CU_TENSOR_MAP_FLOAT_OOB_FILL_NONE));
    }

    dim3 numBlocks(1 + (m - 1) / TILE_SIZE, 1 + (n - 1) / TILE_SIZE);
    mma_simt_tensor_cores_ws<TILE_SIZE><<<numBlocks, 32 * 5, 0, stream>>>(&lhs, &rhs, &lhsTensorMap, &rhsTensorMap,
        m, n, k, out);
}
```

Matrix Multiply in Practice

```
template<size_t TILE_SIZE>
__global__ void mma_simt_tensor_cores_ws(__half const* __restrict__ lhsPtr, __half const* __restrict__ rhsPtr,
    __grid_constant__ const CUTensorMap lhsTensorMap, __grid_constant__ const CUTensorMap rhsTensorMap,
    size_t m, size_t n, size_t k, __half* __restrict__ outPtr) {
    namespace ctd = cuda::std;
    auto lhs = ctd::mdspan(lhsPtr, ctd::extents{m, k});
    auto rhs = ctd::mdspan(rhsPtr, ctd::extents{k, n});
    auto out = ctd::mdspan(outPtr, ctd::extents{m, n});

    constexpr size_t QUEUE_SIZE = 2;
    alignas(128) __shared__ uint64_t fullBarrier[QUEUE_SIZE];
    alignas(128) __shared__ uint64_t emptyBarrier[QUEUE_SIZE];
    if (threadIdx.x == 0) {
        for (size_t idx = 0; idx != QUEUE_SIZE; ++idx) {
            cuda::ptx::mbarrier_init(&fullBarrier[idx], 5 * 32);
            cuda::ptx::mbarrier_init(&emptyBarrier[idx], 5 * 32);
        }
    }
    __syncthreads();

    alignas(128) __shared__ __half lhsShared[QUEUE_SIZE][TILE_SIZE][TILE_SIZE];
    alignas(128) __shared__ __half rhsShared[QUEUE_SIZE][TILE_SIZE][TILE_SIZE];
    size_t numTiles = k / TILE_SIZE;
    int32_t warpId = threadIdx.x / 32;

    if (warpId == 4) {
        size_t queueIdx = 0;
        for (size_t tileIdx = 0; tileIdx != numTiles; ++tileIdx) {
            uint64_t phase = cuda::ptx::mbarrier_arrive(&emptyBarrier[queueIdx]);
            while (!cuda::ptx::mbarrier_try_wait(&emptyBarrier[queueIdx], phase)) { /* spin */ };
            if (threadIdx.x % 32 == 0) {
                int32_t lhsCoord[2] = {(int32_t)TILE_SIZE * (int32_t)tileIdx, (int32_t)TILE_SIZE * (int32_t)blockIdx.x};
                cuda::ptx::cp_async_bulk_tensor(cuda::ptx::space_shared_t{}, cuda::ptx::space_global_t{},
                    &lhsShared[queueIdx][0][0], &lhsTensorMap, lhsCoord, &fullBarrier[queueIdx]);
                cuda::ptx::mbarrier_expect_tx(cuda::ptx::sem_relaxed_t{}, cuda::ptx::scope_cta_t{},
                    cuda::ptx::space_shared_t{}, &fullBarrier[queueIdx], TILE_SIZE * TILE_SIZE * sizeof(__half));
                int32_t rhsCoord[2] = {(int32_t)TILE_SIZE * (int32_t)blockIdx.y, (int32_t)TILE_SIZE * (int32_t)tileIdx};
                cuda::ptx::cp_async_bulk_tensor(cuda::ptx::space_shared_t{}, cuda::ptx::space_global_t{},
                    &rhsShared[queueIdx][0][0], &rhsTensorMap, rhsCoord, &fullBarrier[queueIdx]);
                cuda::ptx::mbarrier_expect_tx(cuda::ptx::sem_relaxed_t{}, cuda::ptx::scope_cta_t{},
                    cuda::ptx::space_shared_t{}, &fullBarrier[queueIdx], TILE_SIZE * TILE_SIZE * sizeof(__half));
            }
            cuda::ptx::mbarrier_arrive(&fullBarrier[queueIdx]);
            queueIdx = (queueIdx + 1) % QUEUE_SIZE;
        }
    } else {
        for (size_t idx = 0; idx != QUEUE_SIZE; ++idx) { cuda::ptx::mbarrier_arrive(&emptyBarrier[idx]); }

        uint32_t accFragment[4] = {0, 0, 0, 0};
        int32_t warpIdx = warpId / 2;
        int32_t warpIdY = warpId % 2;
        auto cornerX = blockIdx.x * TILE_SIZE + warpIdx * TILE_SIZE / 2;
        auto cornerY = blockIdx.y * TILE_SIZE + warpIdY * TILE_SIZE / 2;
        size_t queueIdx = 0;

        for (size_t tileIdx = 0; tileIdx != numTiles; ++tileIdx) {
            uint64_t phase = cuda::ptx::mbarrier_arrive(&fullBarrier[queueIdx]);
            while (!cuda::ptx::mbarrier_try_wait(&fullBarrier[queueIdx], phase)) { /* spin */ };
            uint32_t lhsFragment[8];
```

```
uint32_t rhsFragment[8];
for (size_t subTileIdx = 0; subTileIdx != 2; ++subTileIdx) {
    __half* lhsCorner = &lhsShared[queueIdx][0][0]
        + (TILE_SIZE * TILE_SIZE / 2) * warpIdx + subTileIdx * TILE_SIZE / 2;
    __half* rhsCorner = &rhsShared[queueIdx][0][0]
        + (TILE_SIZE * TILE_SIZE / 2) * subTileIdx + warpIdY * TILE_SIZE / 2;
    asm volatile (
        "mma.load.a.sync.aligned.row.m16n16k16.shared::cta.f16 {%0, %1, %2, %3, %4, %5, %6, %7}, [%20], %22;"
        "mma.load.b.sync.aligned.row.m16n16k16.shared::cta.f16 {%8, %9, %10, %11, %12, %13, %14, %15}, [%21], %22;"
        "mma.mma.sync.aligned.row.m16n16k16.f16.f16 {%16, %17, %18, %19}, "
        "{%0, %1, %2, %3, %4, %5, %6, %7}, {%8, %9, %10, %11, %12, %13, %14, %15}, {%16, %17, %18, %19};"
        : "=r"(lhsFragment[0]), "=r"(lhsFragment[1]), "=r"(lhsFragment[2]), "=r"(lhsFragment[3]),
          "=r"(lhsFragment[4]), "=r"(lhsFragment[5]), "=r"(lhsFragment[6]), "=r"(lhsFragment[7]),
          "=r"(rhsFragment[0]), "=r"(rhsFragment[1]), "=r"(rhsFragment[2]), "=r"(rhsFragment[3]),
          "=r"(rhsFragment[4]), "=r"(rhsFragment[5]), "=r"(rhsFragment[6]), "=r"(rhsFragment[7]),
          "+r"(accFragment[0]), "+r"(accFragment[1]), "+r"(accFragment[2]), "+r"(accFragment[3])
        : "1"(lhsCorner), "1"(rhsCorner), "r"(int32_t)(TILE_SIZE)) : "memory");
    }
    cuda::ptx::mbarrier_arrive(&emptyBarrier[queueIdx]);
    queueIdx = (queueIdx + 1) % QUEUE_SIZE;
}

__half* gOut = &out(cornerX, cornerY);
asm volatile (
    "mma.store.d.sync.aligned.row.m16n16k16.global.f16 [%4], {%0, %1, %2, %3}, %5;"
    :: "r"(accFragment[0]), "r"(accFragment[1]), "r"(accFragment[2]), "r"(accFragment[3]),
      "1"(gOut), "r"(int32_t)(n)) : "memory");
}

void launch_mma_simt_tensor_cores_ws(cudaStream_t stream, __half const* lhs, __half const* rhs,
    size_t m, size_t n, size_t k, __half* out) {
    constexpr size_t TILE_SIZE = 2 * 16;
    CUTensorMap lhsTensorMap;
    {
        uint32_t lhsRank = 2;
        uint64_t lhsSize[2] = {n, k};
        uint64_t lhsStride[1] = {n * sizeof(__half)};
        uint32_t lhsTileSize[2] = {TILE_SIZE, TILE_SIZE};
        uint32_t lhsElemStride[2] = {1, 1};
        checkDriverError(cuTensorMapEncodeTiled(&lhsTensorMap, CU_TENSOR_MAP_DATA_TYPE_FLOAT16, lhsRank, (void*)lhs,
            &lhsSize[0], &lhsStride[0], &lhsTileSize[0], &lhsElemStride[0], CU_TENSOR_MAP_INTERLEAVE_NONE,
            CU_TENSOR_MAP_SWIZZLE_NONE, CU_TENSOR_MAP_L2_PROMOTION_NONE, CU_TENSOR_MAP_FLOAT_OOB_FILL_NONE));
    }
    {
        uint32_t rhsRank = 2;
        uint64_t rhsSize[2] = {n, k};
        uint64_t rhsStride[1] = {n * sizeof(__half)};
        uint32_t rhsTileSize[2] = {TILE_SIZE, TILE_SIZE};
        uint32_t rhsElemStride[2] = {1, 1};
        checkDriverError(cuTensorMapEncodeTiled(&rhsTensorMap, CU_TENSOR_MAP_DATA_TYPE_FLOAT16, rhsRank, (void*)rhs,
            &rhsSize[0], &rhsStride[0], &rhsTileSize[0], &rhsElemStride[0], CU_TENSOR_MAP_INTERLEAVE_NONE,
            CU_TENSOR_MAP_SWIZZLE_NONE, CU_TENSOR_MAP_L2_PROMOTION_NONE, CU_TENSOR_MAP_FLOAT_OOB_FILL_NONE));
    }

    dim3 numBlocks(1 + (m - 1) / TILE_SIZE, 1 + (n - 1) / TILE_SIZE);
    mma_simt_tensor_cores_ws<TILE_SIZE><<<numBlocks, 32 * 5, 0, stream>>>(lhs, rhs, lhsTensorMap, rhsTensorMap,
        m, n, k, out);
}
```

M Barriers

Matrix Multiply in Practice

```
template<size_t TILE_SIZE>
__global__ void mma_simt_tensor_cores_ws(__half const* __restrict__ lhsPtr, __half const* __restrict__ rhsPtr,
    __grid_constant__ const CUtensorMap lhsTensorMap, __grid_constant__ const CUtensorMap rhsTensorMap,
    size_t m, size_t n, size_t k, __half* __restrict__ outPtr) {
    namespace ctd = cuda::std;
    auto lhs = ctd::mdspan(lhsPtr, ctd::extents{m, k});
    auto rhs = ctd::mdspan(rhsPtr, ctd::extents{k, n});
    auto out = ctd::mdspan(outPtr, ctd::extents{m, n});

    constexpr size_t QUEUE_SIZE = 2;
    alignas(128) __shared__ uint64_t fullBarrier[QUEUE_SIZE];
    alignas(128) __shared__ uint64_t emptyBarrier[QUEUE_SIZE];
    if (threadIdx.x == 0) {
        for (size_t idx = 0; idx != QUEUE_SIZE; ++idx) {
            cuda::ptx::mbarrier_init(&fullBarrier[idx], 5 * 32);
            cuda::ptx::mbarrier_init(&emptyBarrier[idx], 5 * 32);
        }
    }
    __syncthreads();

    alignas(128) __shared__ __half lhsShared[QUEUE_SIZE][TILE_SIZE][TILE_SIZE];
    alignas(128) __shared__ __half rhsShared[QUEUE_SIZE][TILE_SIZE][TILE_SIZE];
    size_t numTiles = k / TILE_SIZE;
    int32_t warpId = threadIdx.x / 32;

    if (warpId == 4) {
        size_t queueIdx = 0;
        for (size_t tileIdx = 0; tileIdx != numTiles; ++tileIdx) {
            uint64_t phase = cuda::ptx::mbarrier_arrive(&emptyBarrier[queueIdx]);
            while (!cuda::ptx::mbarrier_try_wait(&emptyBarrier[queueIdx], phase)) { /* spin */ };
            if (threadIdx.x % 32 == 0) {
                int32_t lhsCoord[2] = {(int32_t)TILE_SIZE * (int32_t)tileIdx, (int32_t)TILE_SIZE * (int32_t)blockIdx.x};
                cuda::ptx::cp_async_bulk_tensor(cuda::ptx::space_shared_t{}, cuda::ptx::space_global_t{},
                    &lhsShared[queueIdx][0][0], &lhsTensorMap, lhsCoord, &fullBarrier[queueIdx]);
                cuda::ptx::mbarrier_expect_tx(cuda::ptx::sem_relaxed_t{}, cuda::ptx::scope_cta_t{},
                    cuda::ptx::space_shared_t{}, &fullBarrier[queueIdx], TILE_SIZE * TILE_SIZE * sizeof(__half));
                int32_t rhsCoord[2] = {(int32_t)TILE_SIZE * (int32_t)blockIdx.y, (int32_t)TILE_SIZE * (int32_t)tileIdx};
                cuda::ptx::cp_async_bulk_tensor(cuda::ptx::space_shared_t{}, cuda::ptx::space_global_t{},
                    &rhsShared[queueIdx][0][0], &rhsTensorMap, rhsCoord, &fullBarrier[queueIdx]);
                cuda::ptx::mbarrier_expect_tx(cuda::ptx::sem_relaxed_t{}, cuda::ptx::scope_cta_t{},
                    cuda::ptx::space_shared_t{}, &fullBarrier[queueIdx], TILE_SIZE * TILE_SIZE * sizeof(__half));
            }
            cuda::ptx::mbarrier_arrive(&fullBarrier[queueIdx]);
            queueIdx = (queueIdx + 1) % QUEUE_SIZE;
        }
    } else {
        for (size_t idx = 0; idx != QUEUE_SIZE; ++idx) { cuda::ptx::mbarrier_arrive(&emptyBarrier[idx]); }

        uint32_t accFragment[4] = {0, 0, 0, 0};
        int32_t warpIdx = warpId / 2;
        int32_t warpIdY = warpId % 2;
        auto cornerX = blockIdx.x * TILE_SIZE + warpIdx * TILE_SIZE / 2;
        auto cornerY = blockIdx.y * TILE_SIZE + warpIdY * TILE_SIZE / 2;
        size_t queueIdx = 0;

        for (size_t tileIdx = 0; tileIdx != numTiles; ++tileIdx) {
            uint64_t phase = cuda::ptx::mbarrier_arrive(&fullBarrier[queueIdx]);
            while (!cuda::ptx::mbarrier_try_wait(&fullBarrier[queueIdx], phase)) { /* spin */ };
            uint32_t lhsFragment[8];
```

```
            uint32_t rhsFragment[8];
            for (size_t subTileIdx = 0; subTileIdx != 2; ++subTileIdx) {
                __half* lhsCorner = &lhsShared[queueIdx][0][0]
                    + (TILE_SIZE * TILE_SIZE / 2) * warpIdx + subTileIdx * TILE_SIZE / 2;
                __half* rhsCorner = &rhsShared[queueIdx][0][0]
                    + (TILE_SIZE * TILE_SIZE / 2) * subTileIdx + warpIdY * TILE_SIZE / 2;
                asm volatile (
                    "mma.load.a.sync.aligned.row.m16n16k16.shared::cta.f16 {%0, %1, %2, %3, %4, %5, %6, %7}, [%20], %22;"
                    "mma.load.b.sync.aligned.row.m16n16k16.shared::cta.f16 {%8, %9, %10, %11, %12, %13, %14, %15}, [%21], %22;"
                    "mma.mma.sync.aligned.row.row.m16n16k16.f16.f16 {%16, %17, %18, %19}, "
                    "{%0, %1, %2, %3, %4, %5, %6, %7}, {%8, %9, %10, %11, %12, %13, %14, %15}, {%16, %17, %18, %19};"
                    : "=r"(lhsFragment[0]), "=r"(lhsFragment[1]), "=r"(lhsFragment[2]), "=r"(lhsFragment[3]),
                      "=r"(lhsFragment[4]), "=r"(lhsFragment[5]), "=r"(lhsFragment[6]), "=r"(lhsFragment[7]),
                      "=r"(rhsFragment[0]), "=r"(rhsFragment[1]), "=r"(rhsFragment[2]), "=r"(rhsFragment[3]),
                      "=r"(rhsFragment[4]), "=r"(rhsFragment[5]), "=r"(rhsFragment[6]), "=r"(rhsFragment[7]),
                      "+r"(accFragment[0]), "+r"(accFragment[1]), "+r"(accFragment[2]), "+r"(accFragment[3])
                    : "1"(lhsCorner), "1"(rhsCorner), "r"(int32_t(TILE_SIZE)) : "memory");
            }
            cuda::ptx::mbarrier_arrive(&emptyBarrier[queueIdx]);
            queueIdx = (queueIdx + 1) % QUEUE_SIZE;
        }

        __half* gOut = &out(cornerX, cornerY);
        asm volatile (
            "mma.store.d.sync.aligned.row.m16n16k16.global.f16 [%4], {%0, %1, %2, %3}, %5;"
            :: "r"(accFragment[0]), "r"(accFragment[1]), "r"(accFragment[2]), "r"(accFragment[3]),
              "1"(gOut), "r"(int32_t(n)) : "memory");
    }
}

void launch_mma_simt_tensor_cores_ws(cudaStream_t stream, __half const* lhs, __half const* rhs,
    size_t m, size_t n, size_t k, __half* out) {
    constexpr size_t TILE_SIZE = 2 * 16;
    CUtensorMap lhsTensorMap, rhsTensorMap;
    {
        uint32_t lhsRank = 2;
        uint64_t lhsSize[2] = {k, m};
        uint64_t lhsStride[1] = {k * sizeof(__half)};
        uint32_t lhsTileSize[2] = {TILE_SIZE, TILE_SIZE};
        uint32_t lhsElemStride[2] = {1, 1};
        checkDriverError(cuTensorMapEncodeTiled(&lhsTensorMap, CU_TENSOR_MAP_DATA_TYPE_FLOAT16, lhsRank, (void*)lhs,
            &lhsSize[0], &lhsStride[0], &lhsTileSize[0], &lhsElemStride[0], CU_TENSOR_MAP_INTERLEAVE_NONE,
            CU_TENSOR_MAP_SWIZZLE_NONE, CU_TENSOR_MAP_L2_PROMOTION_NONE, CU_TENSOR_MAP_FLOAT_OOB_FILL_NONE));
    }
    {
        uint32_t rhsRank = 2;
        uint64_t rhsSize[2] = {n, k};
        uint64_t rhsStride[1] = {n * sizeof(__half)};
        uint32_t rhsTileSize[2] = {TILE_SIZE, TILE_SIZE};
        uint32_t rhsElemStride[2] = {1, 1};
        checkDriverError(cuTensorMapEncodeTiled(&rhsTensorMap, CU_TENSOR_MAP_DATA_TYPE_FLOAT16, rhsRank, (void*)rhs,
            &rhsSize[0], &rhsStride[0], &rhsTileSize[0], &rhsElemStride[0], CU_TENSOR_MAP_INTERLEAVE_NONE,
            CU_TENSOR_MAP_SWIZZLE_NONE, CU_TENSOR_MAP_L2_PROMOTION_NONE, CU_TENSOR_MAP_FLOAT_OOB_FILL_NONE));
    }

    dim3 numBlocks(1 + (m - 1) / TILE_SIZE, 1 + (n - 1) / TILE_SIZE);
    mma_simt_tensor_cores_ws<TILE_SIZE><<<numBlocks, 32 * 5, 0, stream>>>(&lhs, &rhs, &lhsTensorMap, &rhsTensorMap,
        m, n, k, out);
}
```

Challenges with “Single Instruction Multiple Thread (SIMT)” Code

- **Complexity**

- The programmer must manually manage the hardware resources
- Performance requires attention to many details
- GPU code is difficult to learn, debug, and author

- **Portability**

- Each architectural generation introduces new hardware units
- Some hardware features are architecture specific
- Your algorithm must be rewritten each year to get optimal performance

Matrix Multiply in Tile C++

```
__tile_global__ void mma_tile_kernel(  
    const __half* __restrict__ aPtr,  
    const __half* __restrict__ bPtr,  
    int M, int N, int K,
```

```
    auto A = ct::tensor_span{aPtr, ct::extents{M, K}};  
    auto B = ct::tensor_span{bPtr, ct::extents{K, N}};  
    auto C = ct::tensor_span{cPtr, ct::extents{M, N}};
```

Good performance in hours instead of days; across all GPU architectures

```
    bPtr = ct::assume_aligned(bPtr, 16_ic);  
    cPtr = ct::assume_aligned(cPtr, 16_ic);
```

```
    M = ct::assume_divisible(M, 16_ic);  
    N = ct::assume_divisible(N, 16_ic);  
    K = ct::assume_divisible(K, 16_ic);
```

```
    using MxN = ct::shape<TILE_M, TILE_N>;  
    using MxK = ct::shape<TILE_M, TILE_K>;  
    using KxN = ct::shape<TILE_K, TILE_N>;
```

```
    auto acc = ct::zeros<ct::tile<__half, MxN>>();  
    int numTiles = 1 + (K - 1) / TILE_K;
```

```
    for (auto idx : ct::irange(0, numTiles)) {  
        auto aTile = aView.load_masked(bidx, idx);  
        auto bTile = bView.load_masked(idx, bidx);  
        acc = ct::mma(aTile, bTile, acc);  
    }
```

```
    cView.store_masked(acc, bidx, bidx);  
}
```

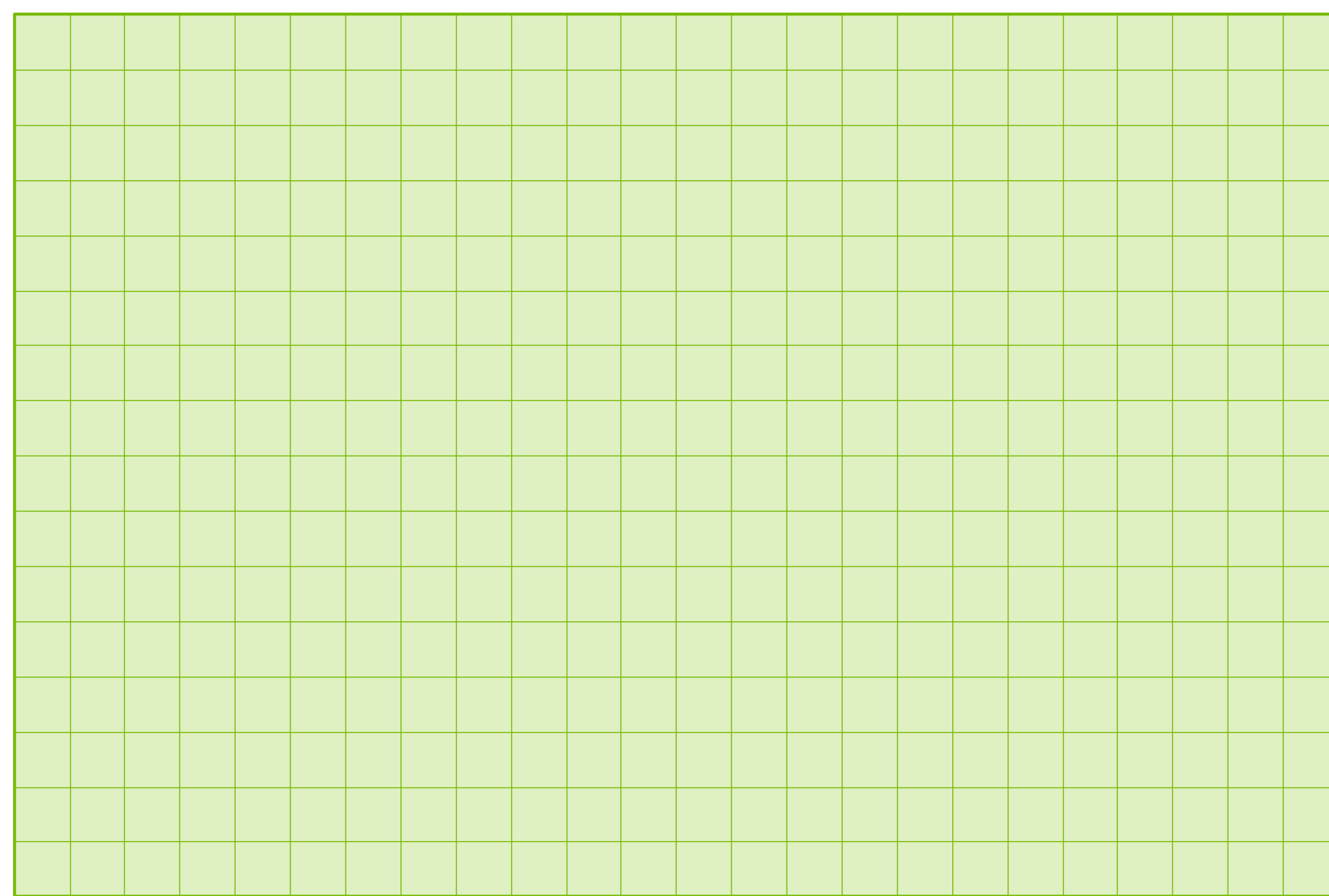
Single Instruction Multiple Thread (SIMT)

Designing an algorithm for the GPU using the SIMT paradigm

- Partition the data set into **blocks** which can be processed independently
- Within a **block**, use **threads** to perform the computation

$$C = A * B$$

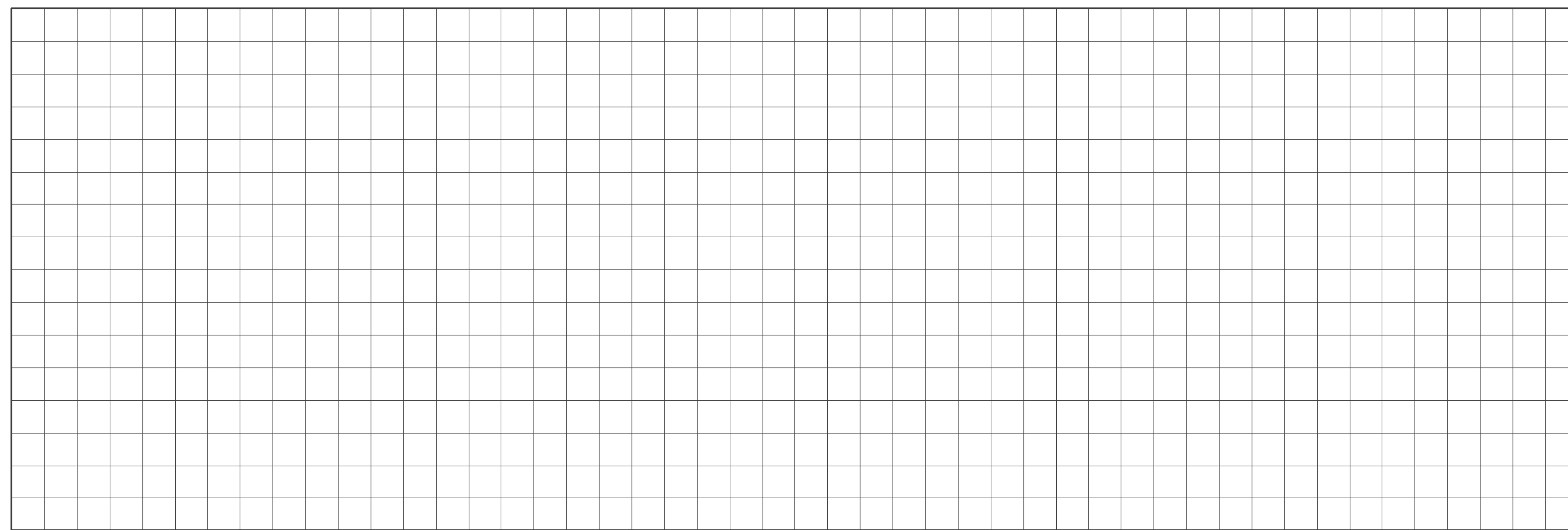
M



A

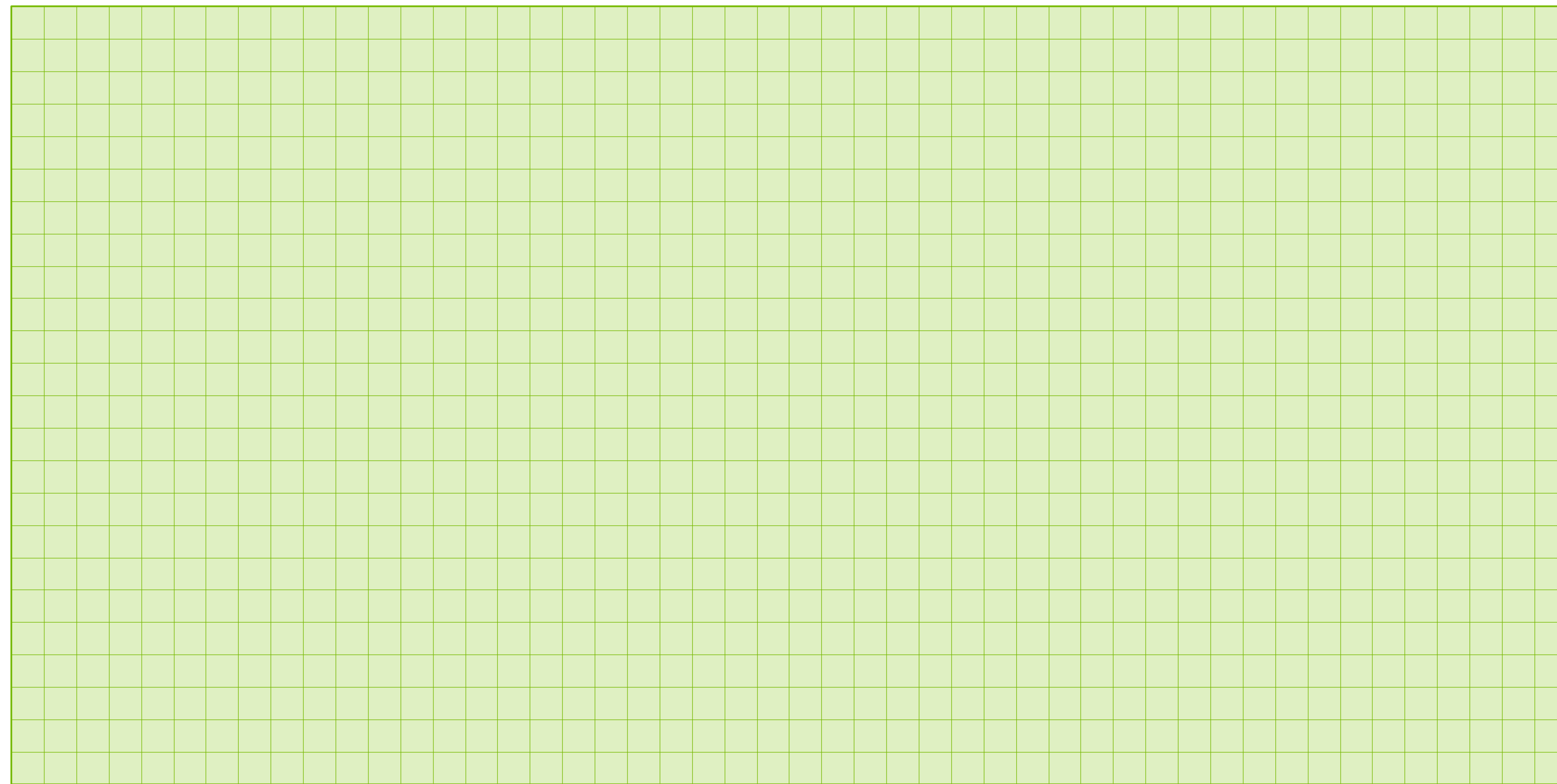
K

K



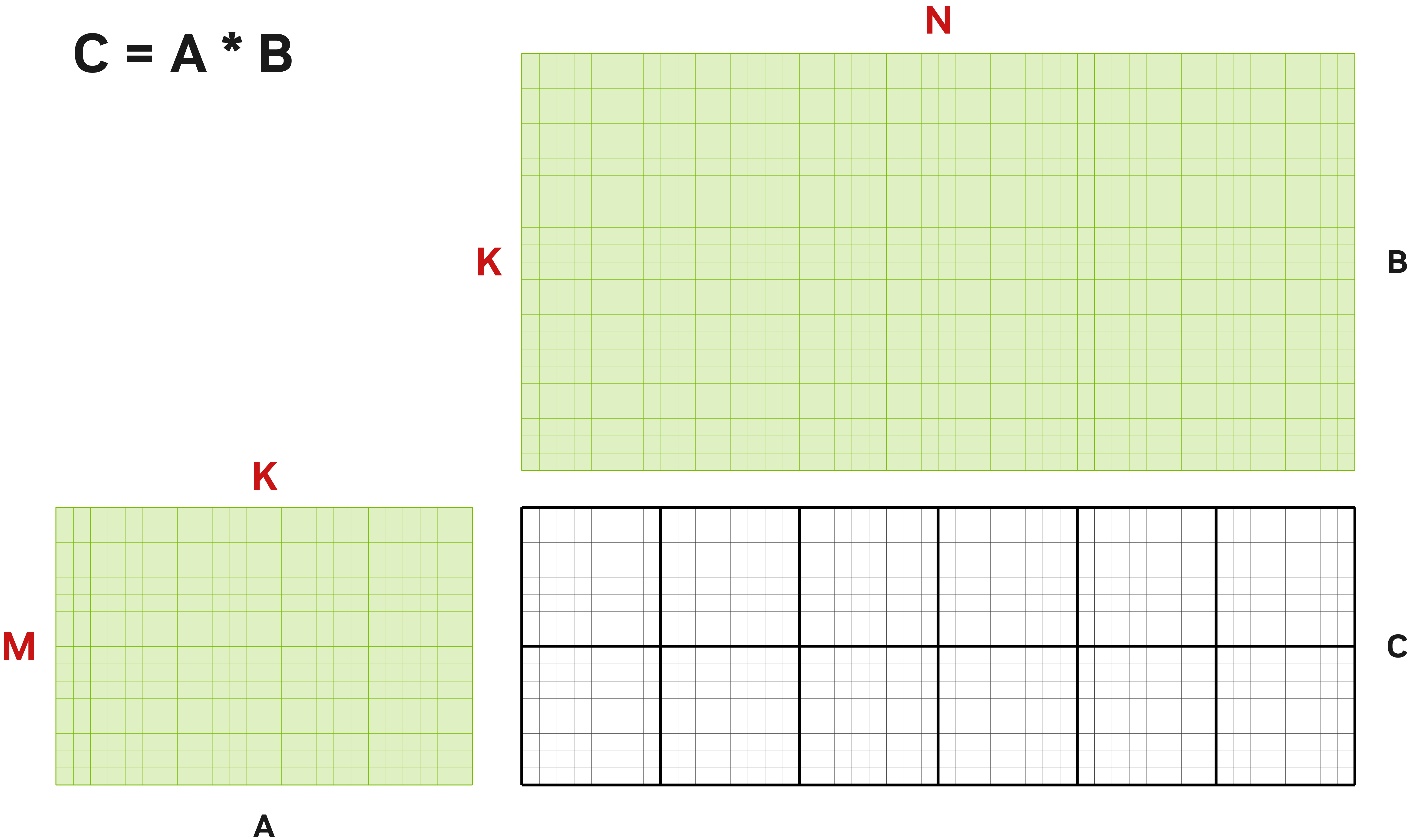
C

N

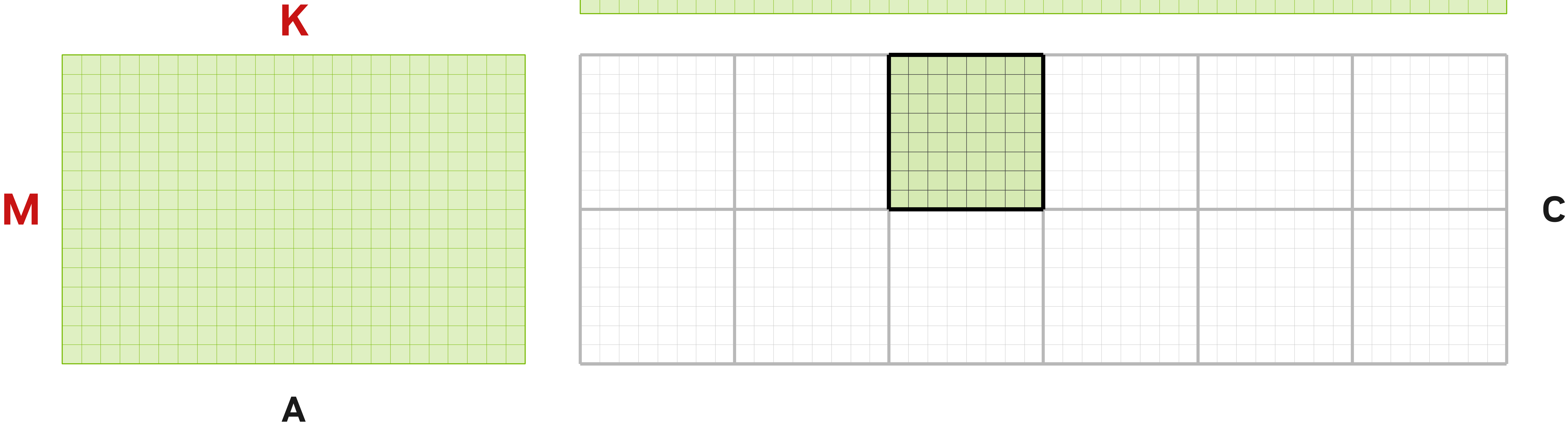


B

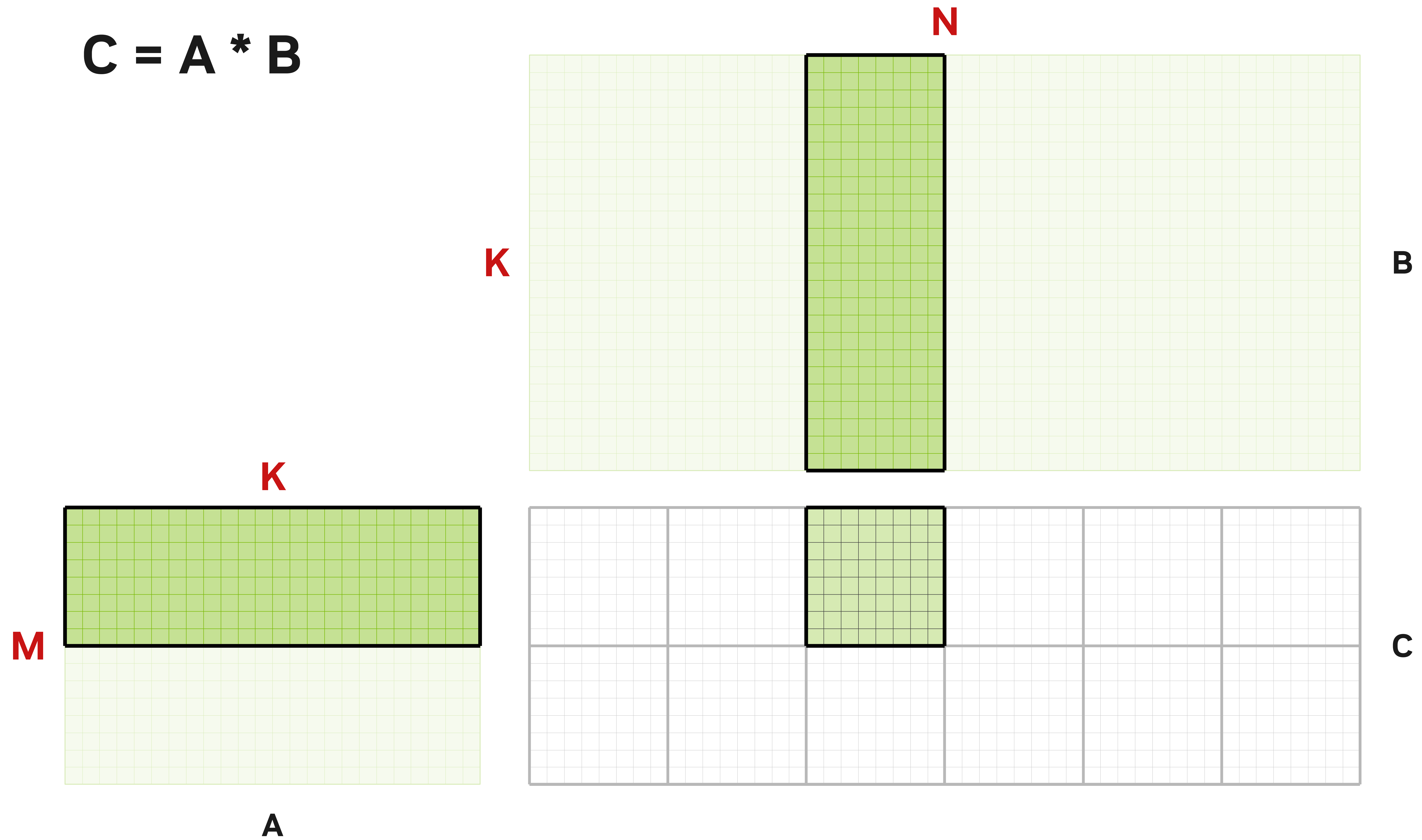
$C = A * B$



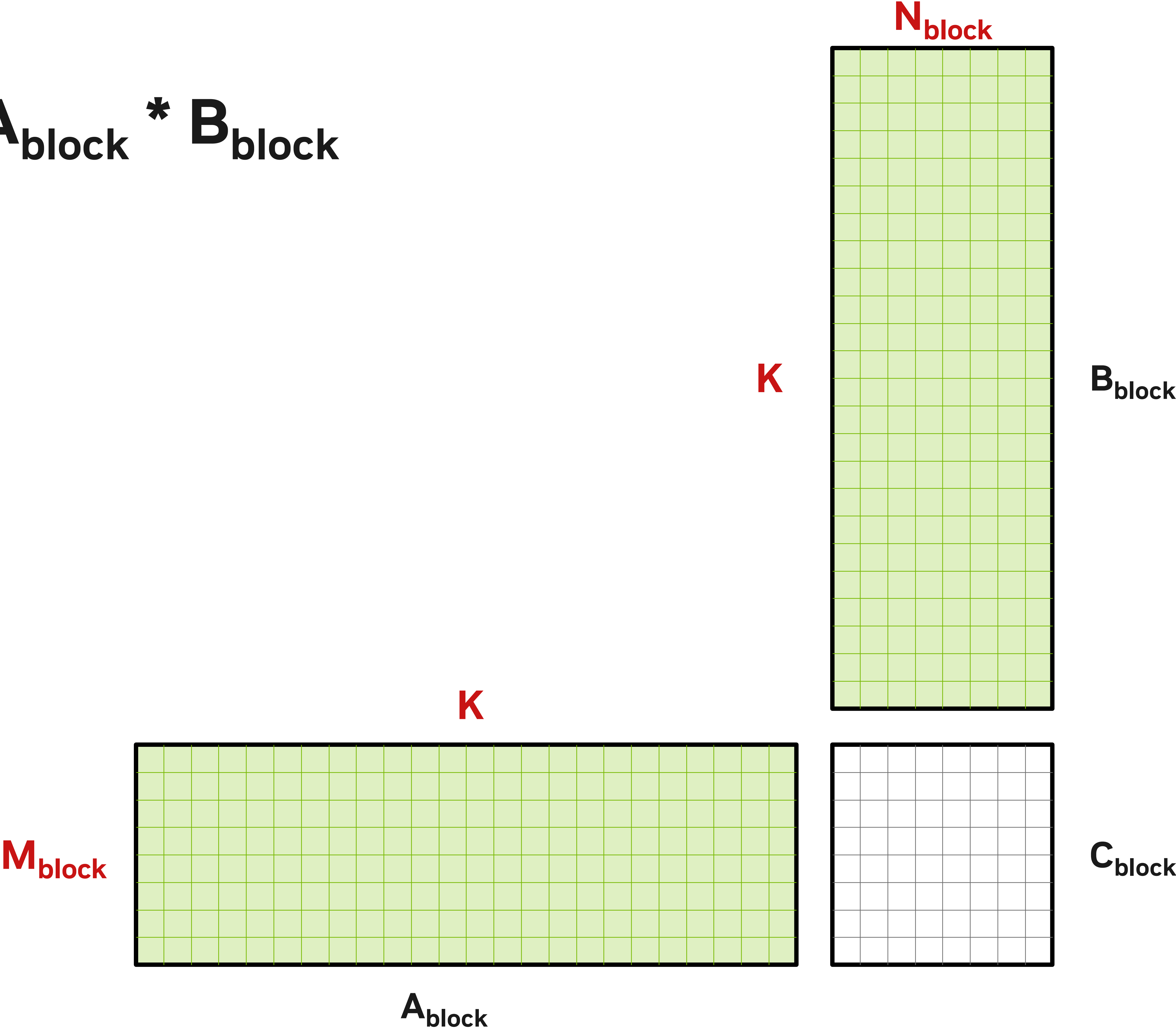
$C = A * B$



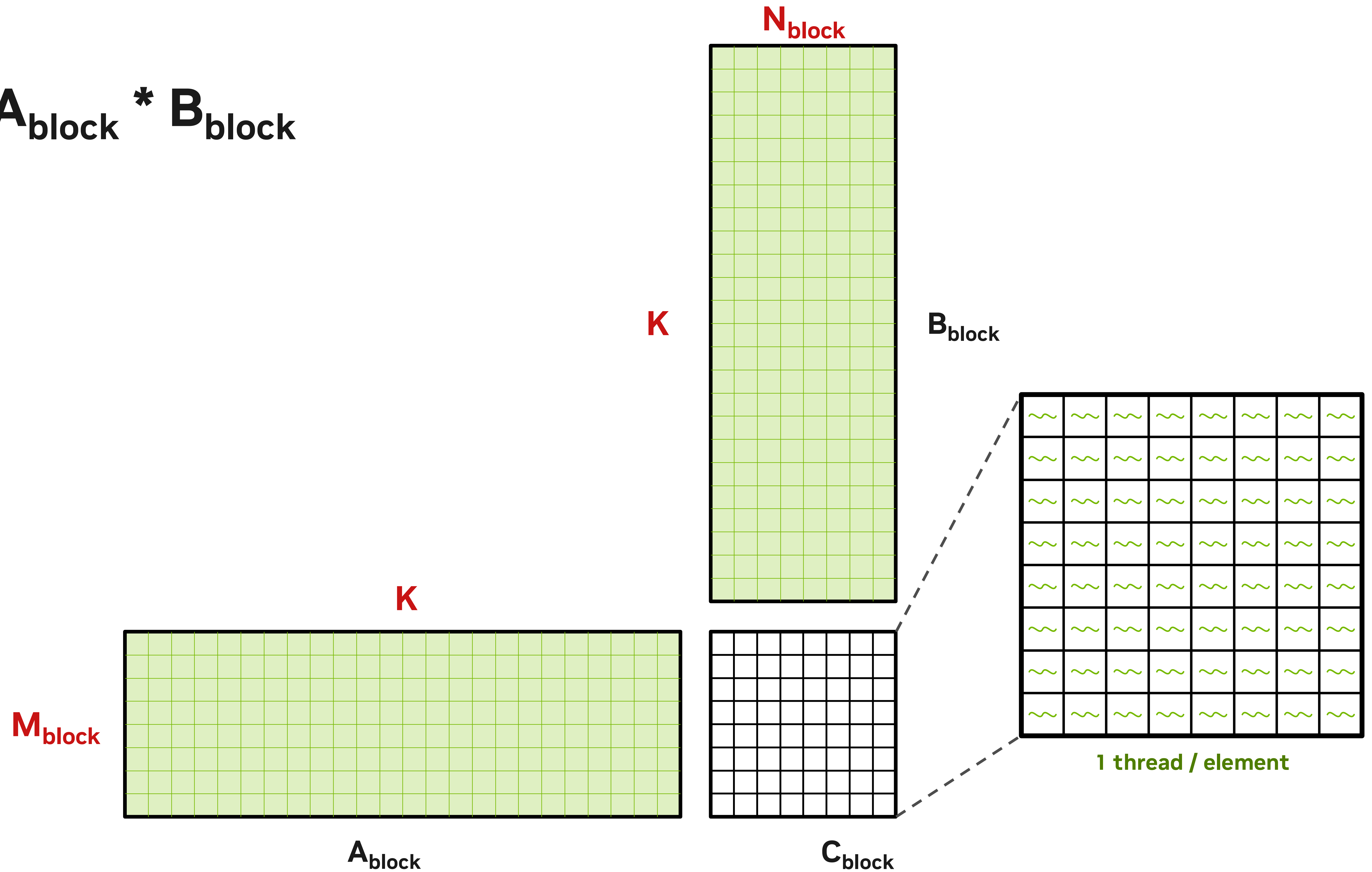
$$C = A * B$$



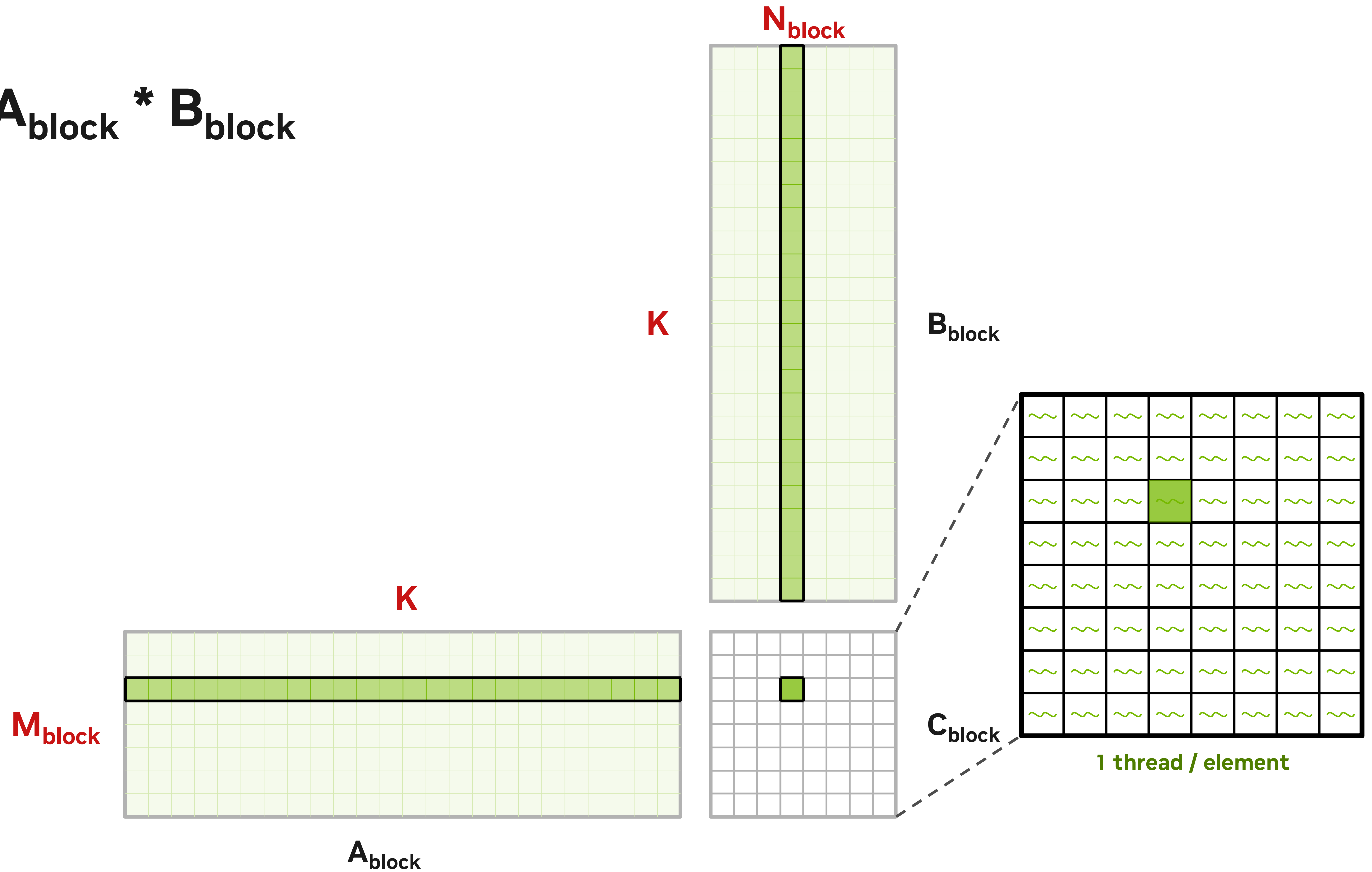
$C_{block} = A_{block} * B_{block}$



$$C_{\text{block}} = A_{\text{block}} * B_{\text{block}}$$



$$C_{\text{block}} = A_{\text{block}} * B_{\text{block}}$$



SIMT: Naive Matrix Multiply

```
__global__ void mma_simt_naive(__half const* aPtr, __half const* bPtr,
                               size_t M, size_t N, size_t K,
                               __half* cPtr) {
    auto A = cuda::std::mdspan(aPtr, cuda::std::extents{M, K});
    auto B = cuda::std::mdspan(bPtr, cuda::std::extents{K, N});
    auto C = cuda::std::mdspan(cPtr, cuda::std::extents{M, N});

    auto x = threadIdx.y + blockIdx.y * blockDim.y;
    auto y = threadIdx.x + blockIdx.x * blockDim.x;

    __half acc{0};

    for (size_t idx = 0; idx != K; ++idx) {
        acc += A(x, idx) * B(idx, y);
    }

    C(x, y) = acc;
}
```

SIMT: Naive Matrix Multiply

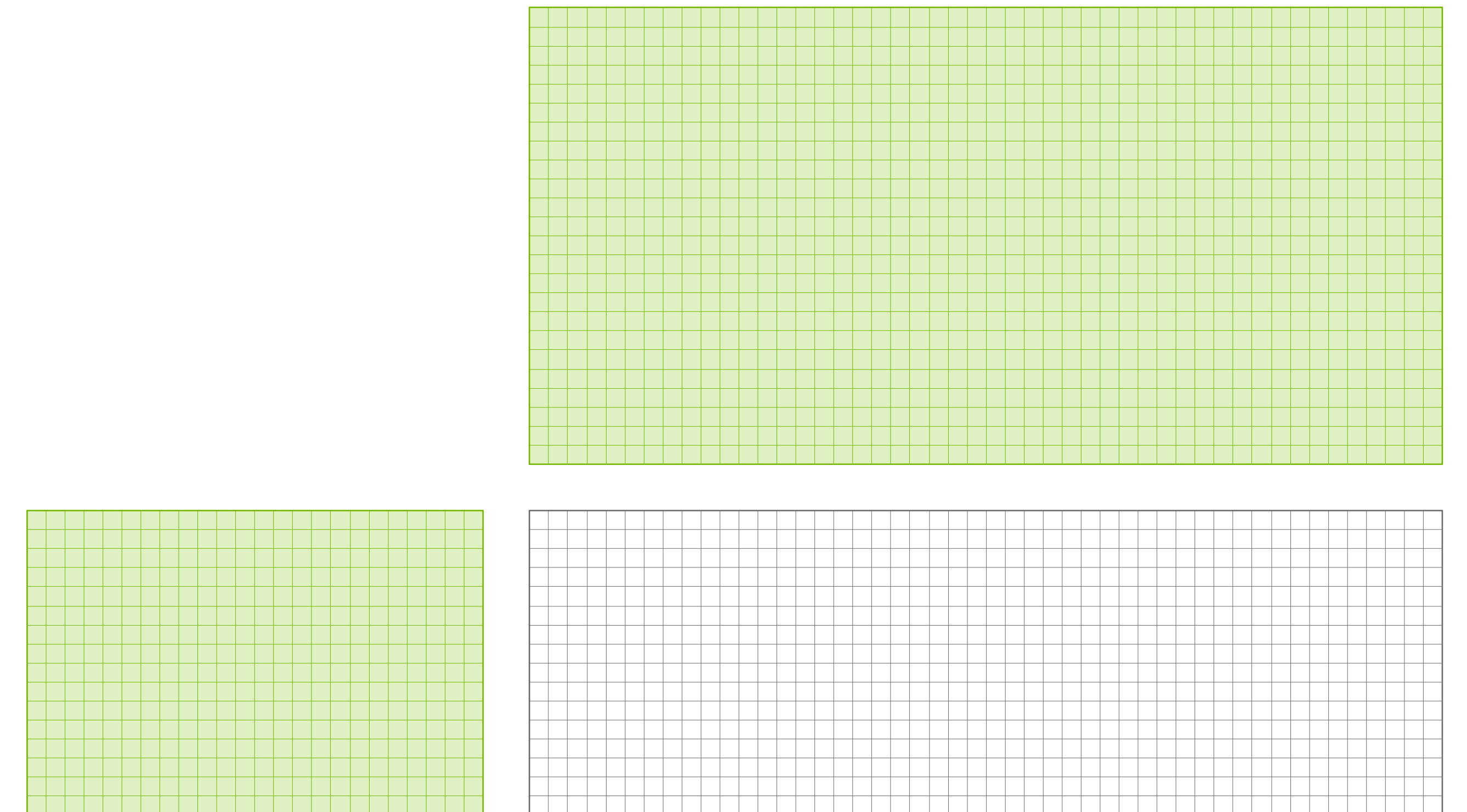
```
__global__ void mma_simt_naive(__half const* aPtr, __half const* bPtr,
                               size_t M, size_t N, size_t K,
                               __half* cPtr) {
    auto A = cuda::std::mdspan(aPtr, cuda::std::extents{M, K});
    auto B = cuda::std::mdspan(bPtr, cuda::std::extents{K, N});
    auto C = cuda::std::mdspan(cPtr, cuda::std::extents{M, N});

    auto x = threadIdx.y + blockIdx.y * blockDim.y;
    auto y = threadIdx.x + blockIdx.x * blockDim.x;

    __half acc{0};

    for (size_t idx = 0; idx != K; ++idx) {
        acc += A(x, idx) * B(idx, y);
    }

    C(x, y) = acc;
}
```



SIMT: Naive Matrix Multiply

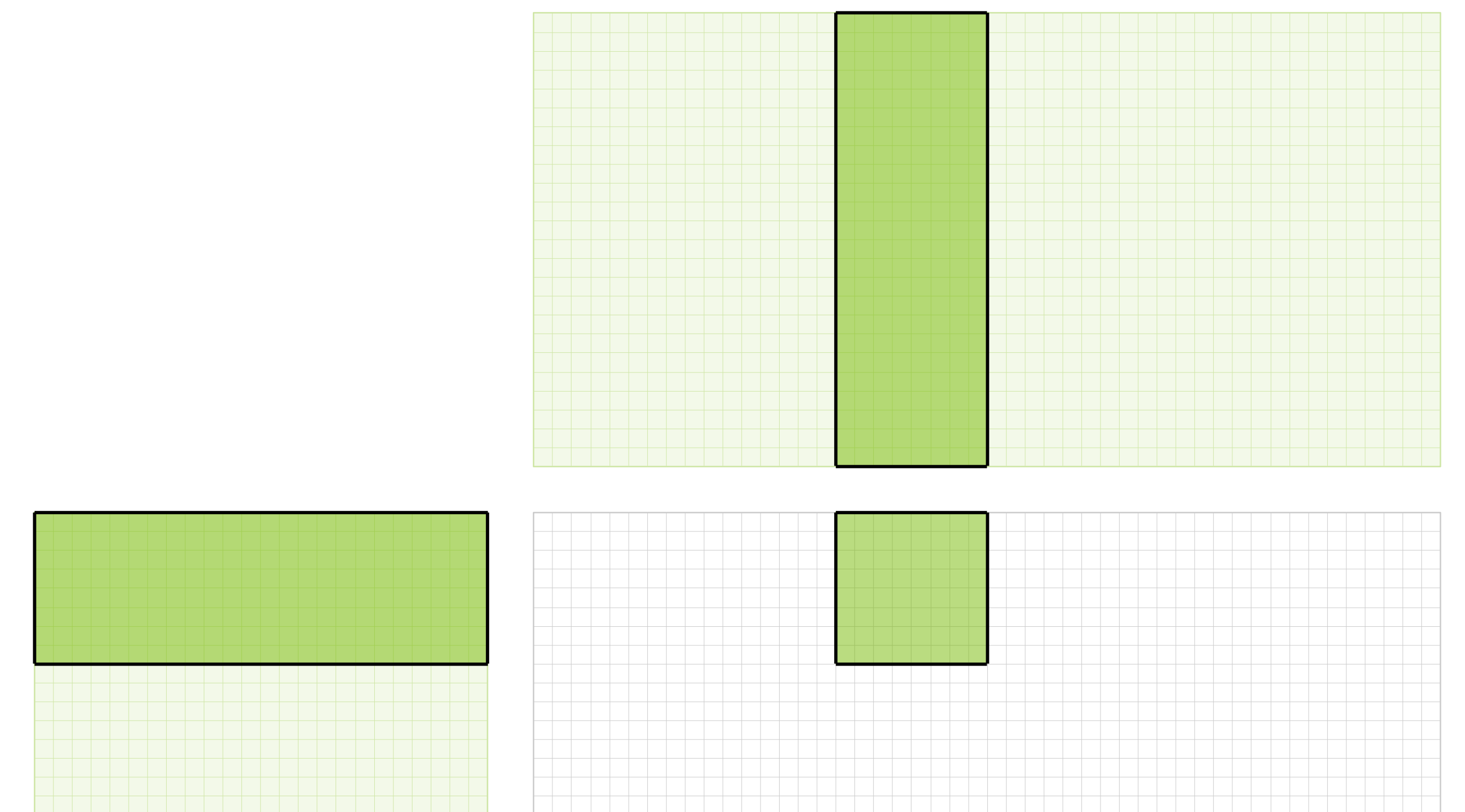
```
__global__ void mma_simt_naive(__half const* aPtr, __half const* bPtr,
                               size_t M, size_t N, size_t K,
                               __half* cPtr) {
    auto A = cuda::std::mdspan(aPtr, cuda::std::extents{M, K});
    auto B = cuda::std::mdspan(bPtr, cuda::std::extents{K, N});
    auto C = cuda::std::mdspan(cPtr, cuda::std::extents{M, N});

    auto x = threadIdx.y + blockIdx.y * blockDim.y;
    auto y = threadIdx.x + blockIdx.x * blockDim.x;

    __half acc{0};

    for (size_t idx = 0; idx != K; ++idx) {
        acc += A(x, idx) * B(idx, y);
    }

    C(x, y) = acc;
}
```



SIMT: Naive Matrix Multiply

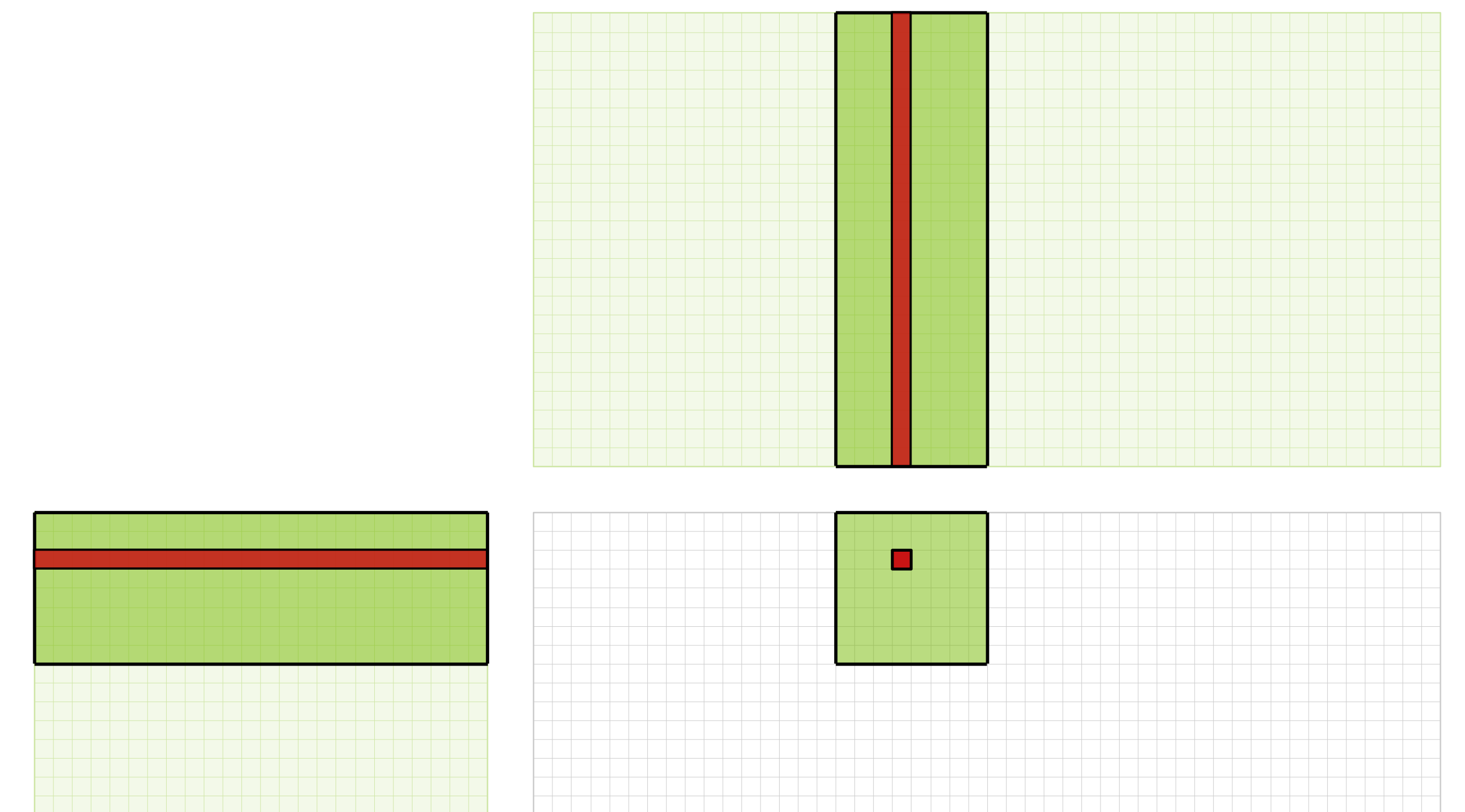
```
__global__ void mma_simt_naive(__half const* aPtr, __half const* bPtr,
                               size_t M, size_t N, size_t K,
                               __half* cPtr) {
    auto A = cuda::std::mdspan(aPtr, cuda::std::extents{M, K});
    auto B = cuda::std::mdspan(bPtr, cuda::std::extents{K, N});
    auto C = cuda::std::mdspan(cPtr, cuda::std::extents{M, N});

    auto x = threadIdx.y + blockIdx.y * blockDim.y;
    auto y = threadIdx.x + blockIdx.x * blockDim.x;

    __half acc{0};

    for (size_t idx = 0; idx != K; ++idx) {
        acc += A(x, idx) * B(idx, y);
    }

    C(x, y) = acc;
}
```



SIMT: Naive Matrix Multiply

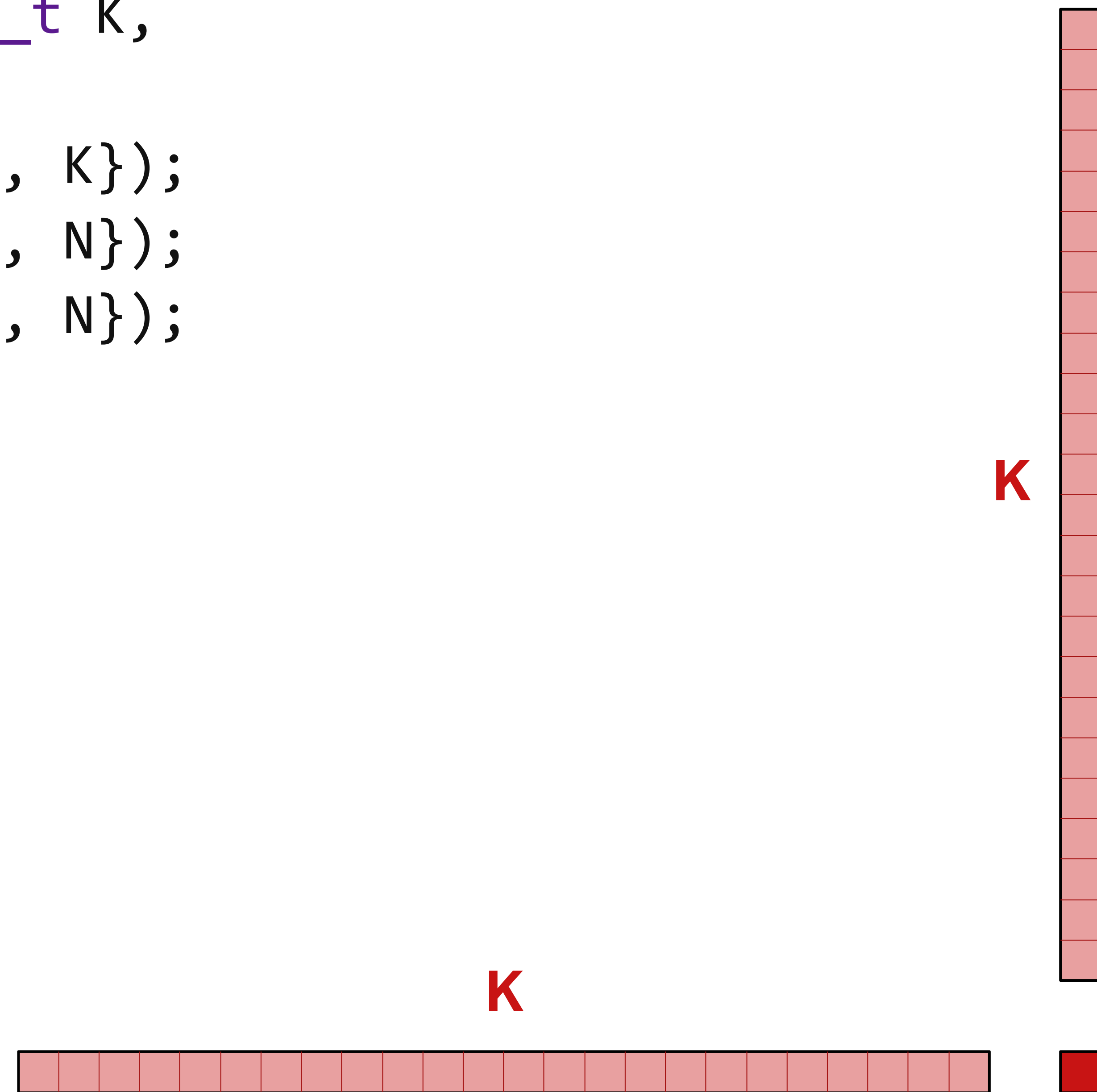
```
__global__ void mma_simt_naive(__half const* aPtr, __half const* bPtr,
                               size_t M, size_t N, size_t K,
                               __half* cPtr) {
    auto A = cuda::std::mdspan(aPtr, cuda::std::extents{M, K});
    auto B = cuda::std::mdspan(bPtr, cuda::std::extents{K, N});
    auto C = cuda::std::mdspan(cPtr, cuda::std::extents{M, N});

    auto x = threadIdx.y + blockIdx.y * blockDim.y;
    auto y = threadIdx.x + blockIdx.x * blockDim.x;

    __half acc{0};

    for (size_t idx = 0; idx != K; ++idx) {
        acc += A(x, idx) * B(idx, y);
    }

    C(x, y) = acc;
}
```



SIMT: Naive Matrix Multiply

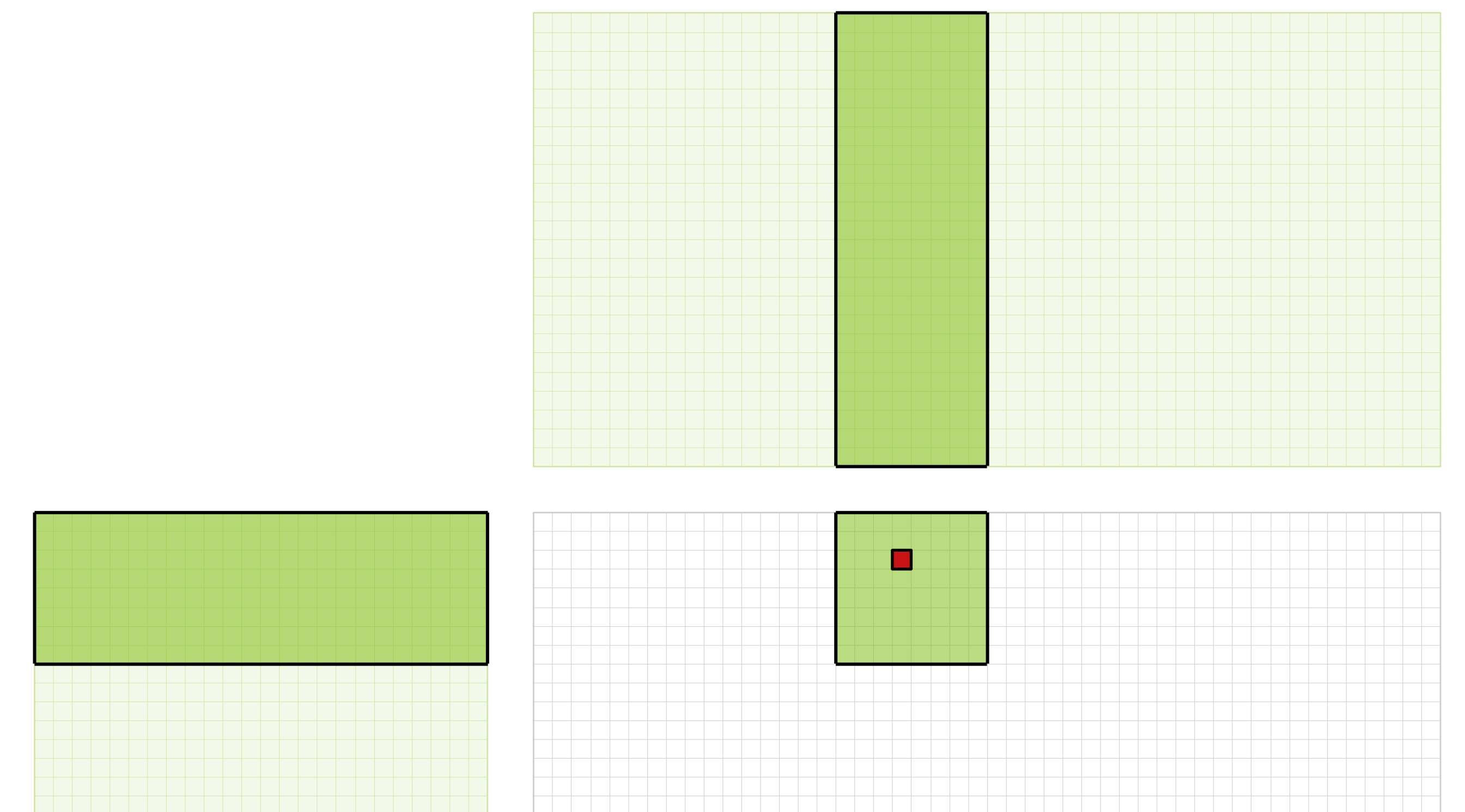
```
__global__ void mma_simt_naive(__half const* aPtr, __half const* bPtr,
                               size_t M, size_t N, size_t K,
                               __half* cPtr) {
    auto A = cuda::std::mdspan(aPtr, cuda::std::extents{M, K});
    auto B = cuda::std::mdspan(bPtr, cuda::std::extents{K, N});
    auto C = cuda::std::mdspan(cPtr, cuda::std::extents{M, N});

    auto x = threadIdx.y + blockIdx.y * blockDim.y;
    auto y = threadIdx.x + blockIdx.x * blockDim.x;

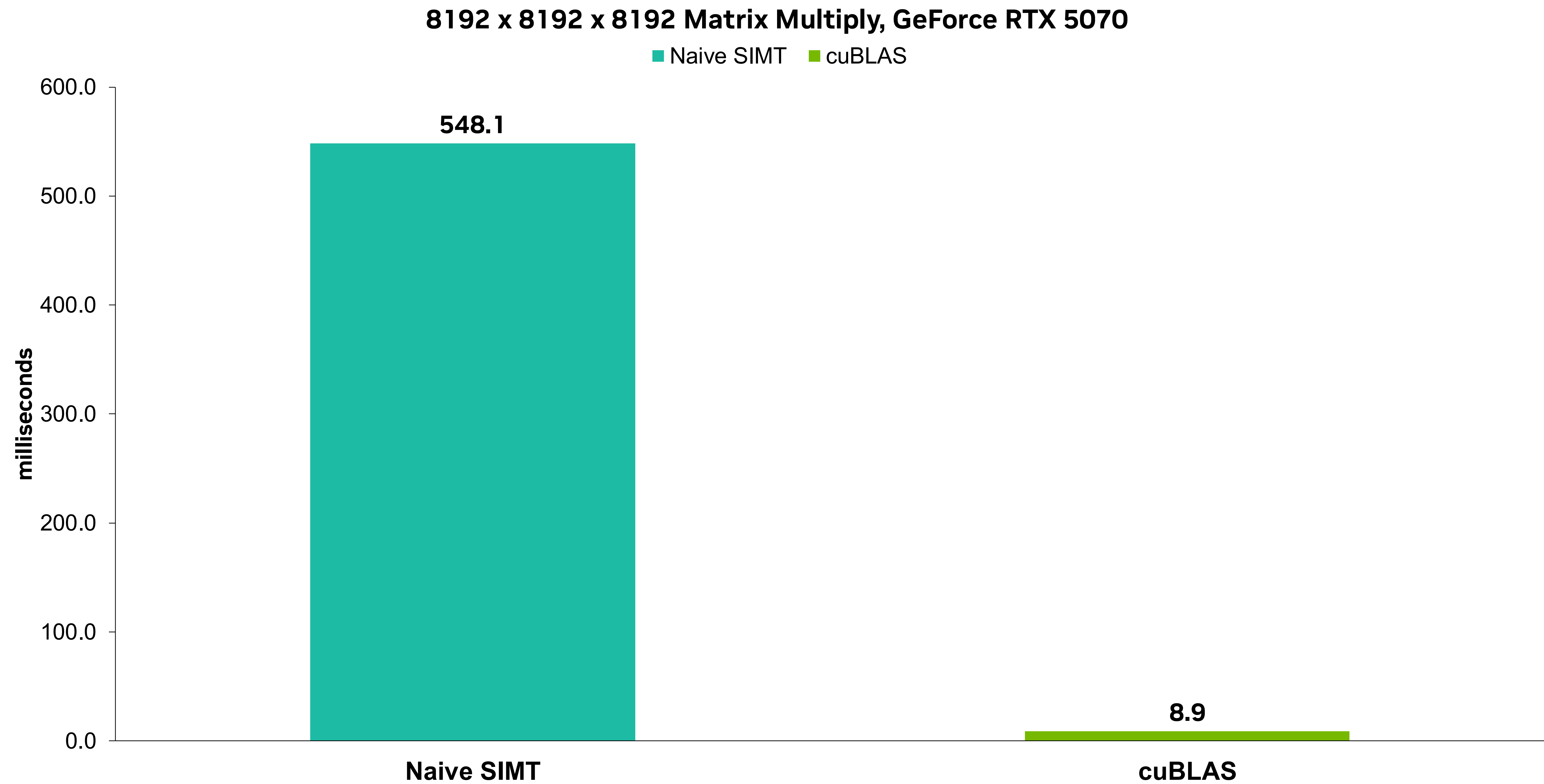
    __half acc{0};

    for (size_t idx = 0; idx != K; ++idx) {
        acc += A(x, idx) * B(idx, y);
    }

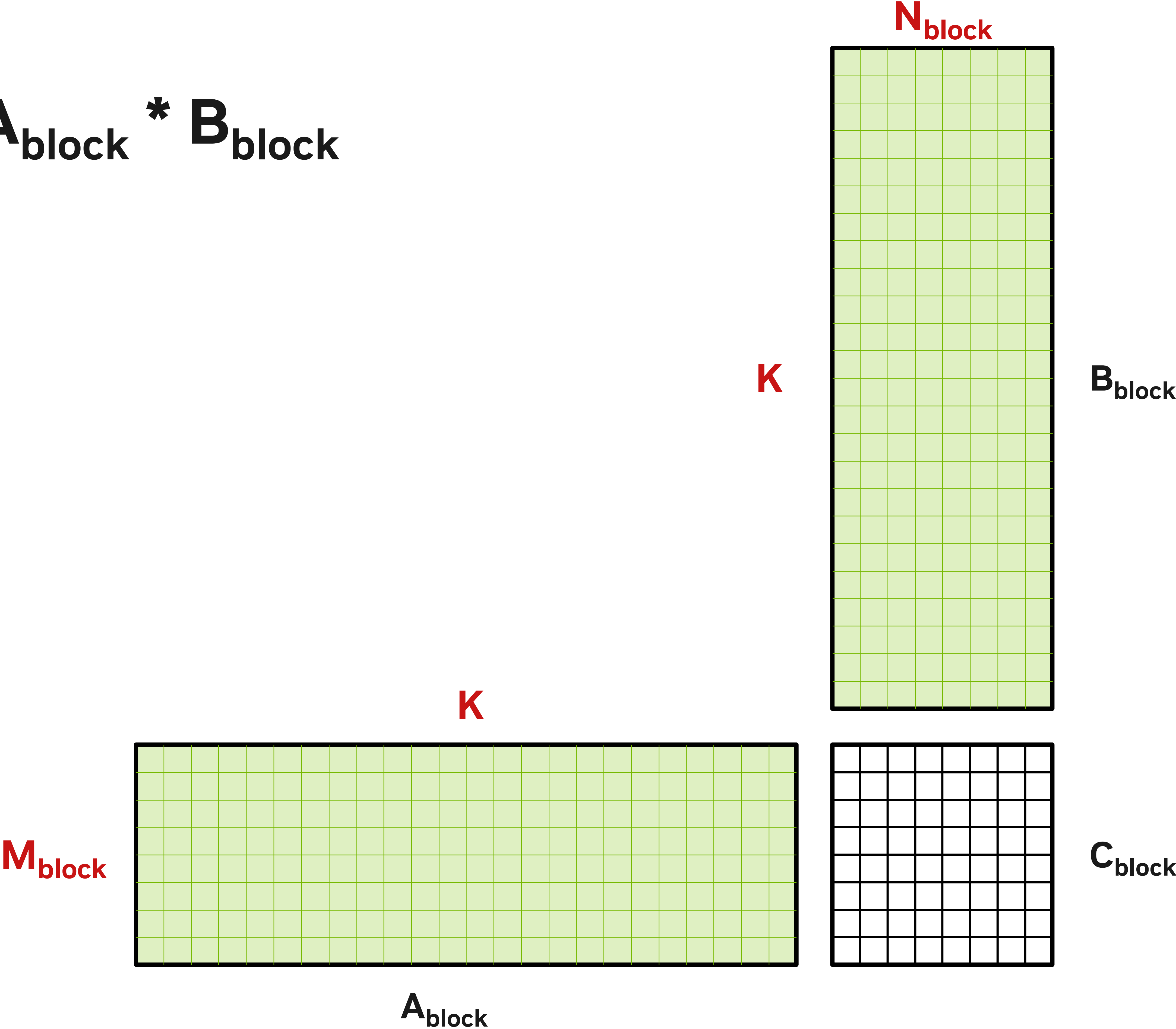
    C(x, y) = acc;
}
```



SIMT: Naive Matrix Multiply Performance

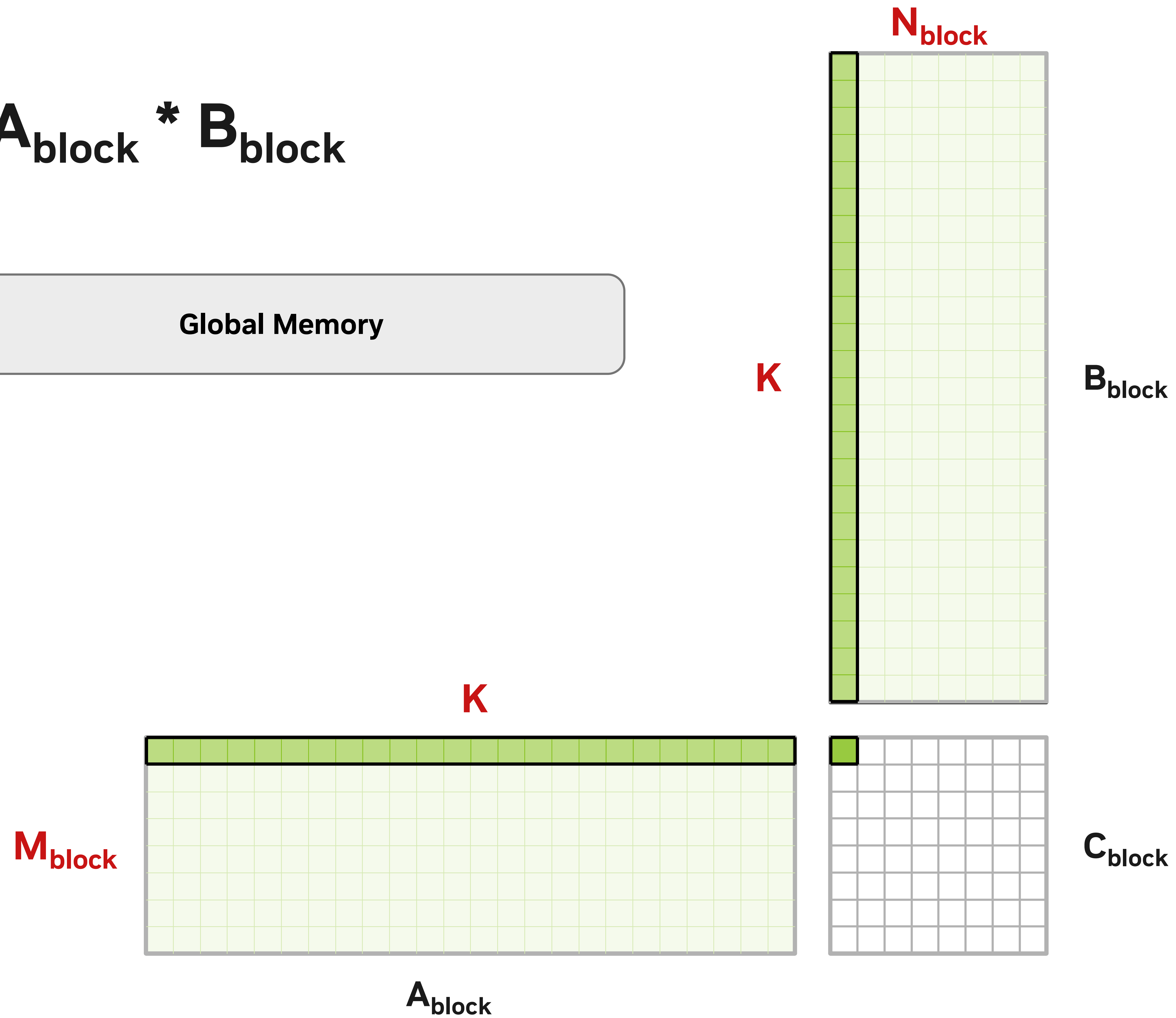


$C_{block} = A_{block} * B_{block}$



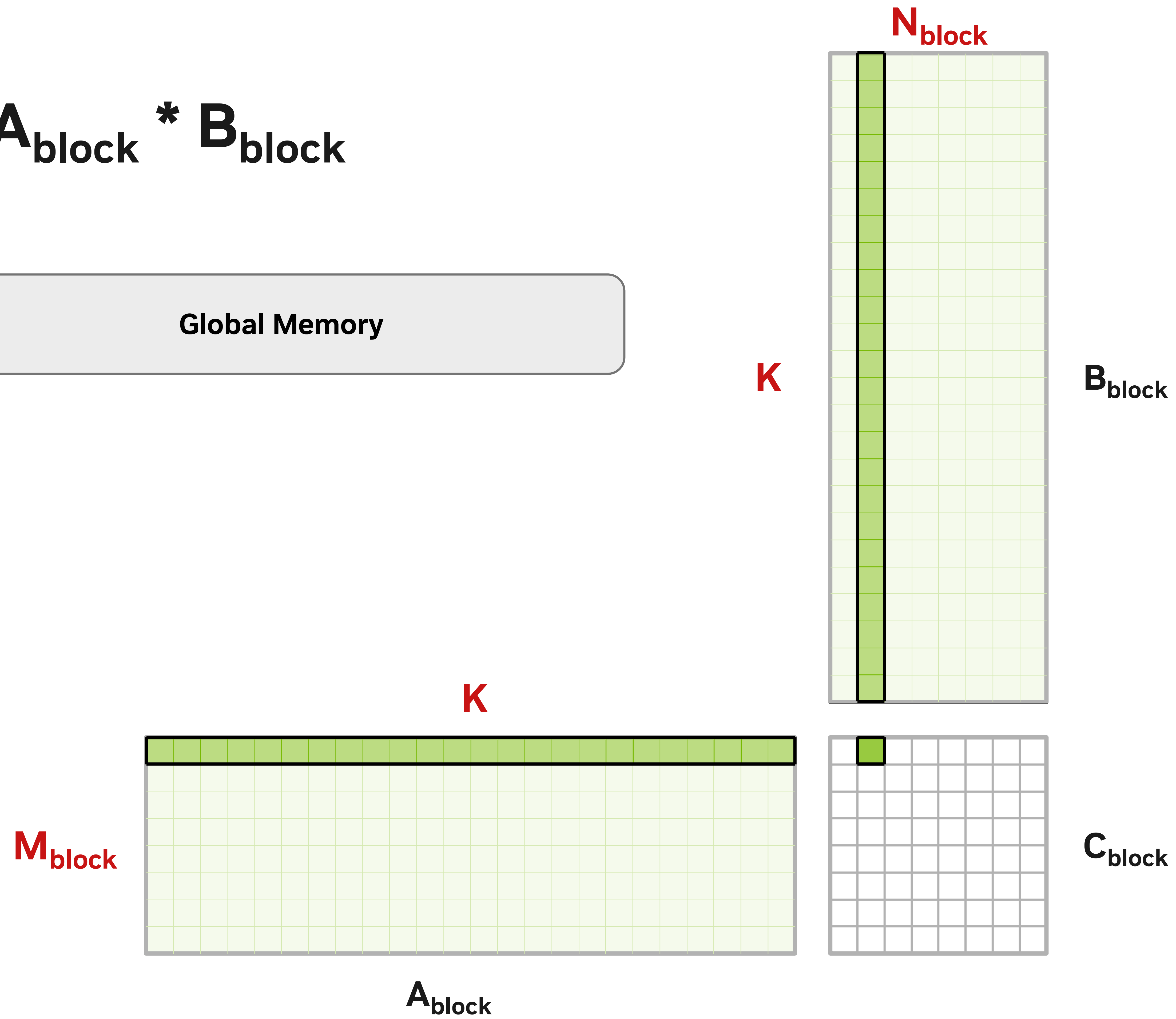
$$C_{\text{block}} = A_{\text{block}} * B_{\text{block}}$$

Global Memory



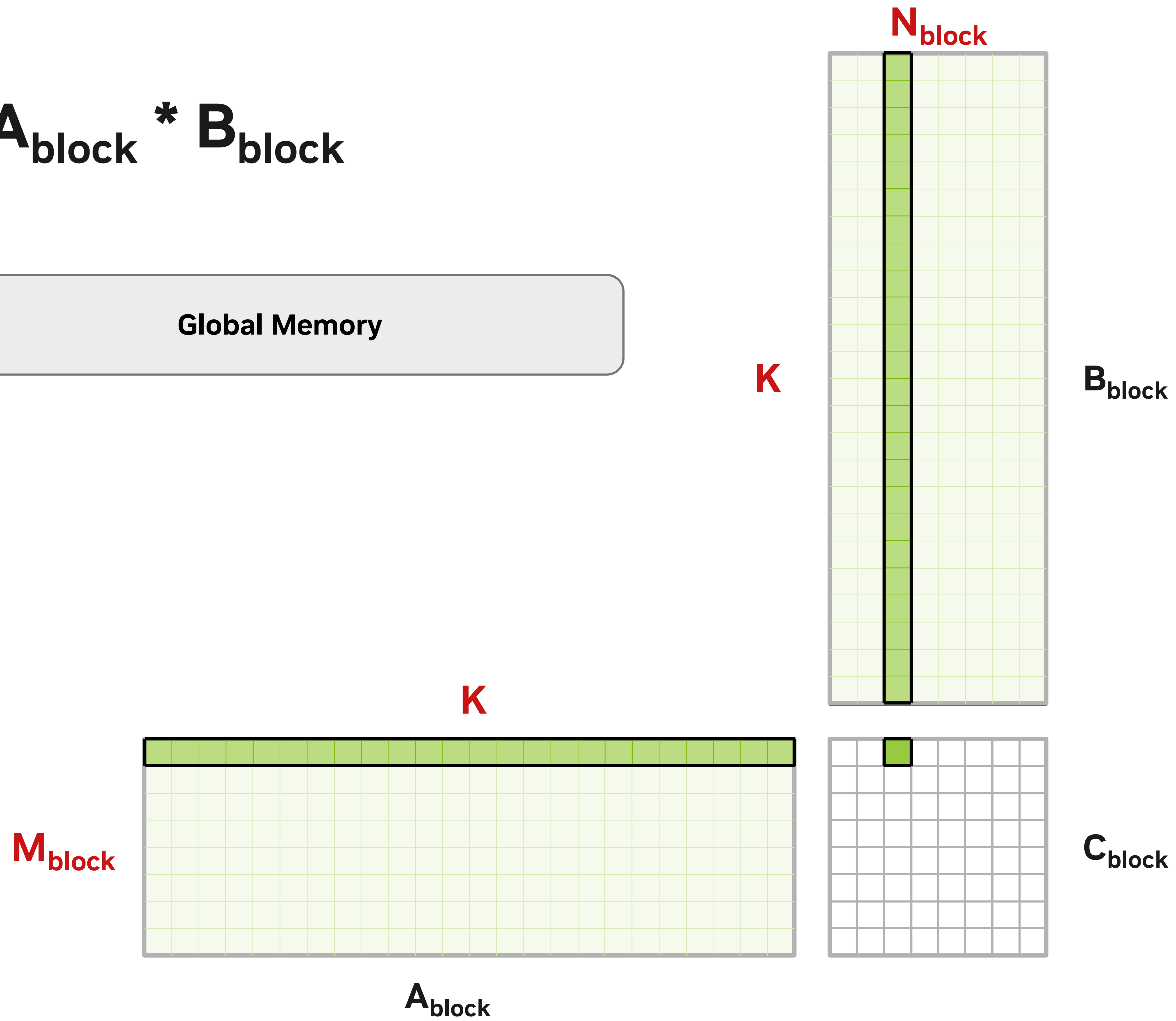
$$C_{\text{block}} = A_{\text{block}} * B_{\text{block}}$$

Global Memory



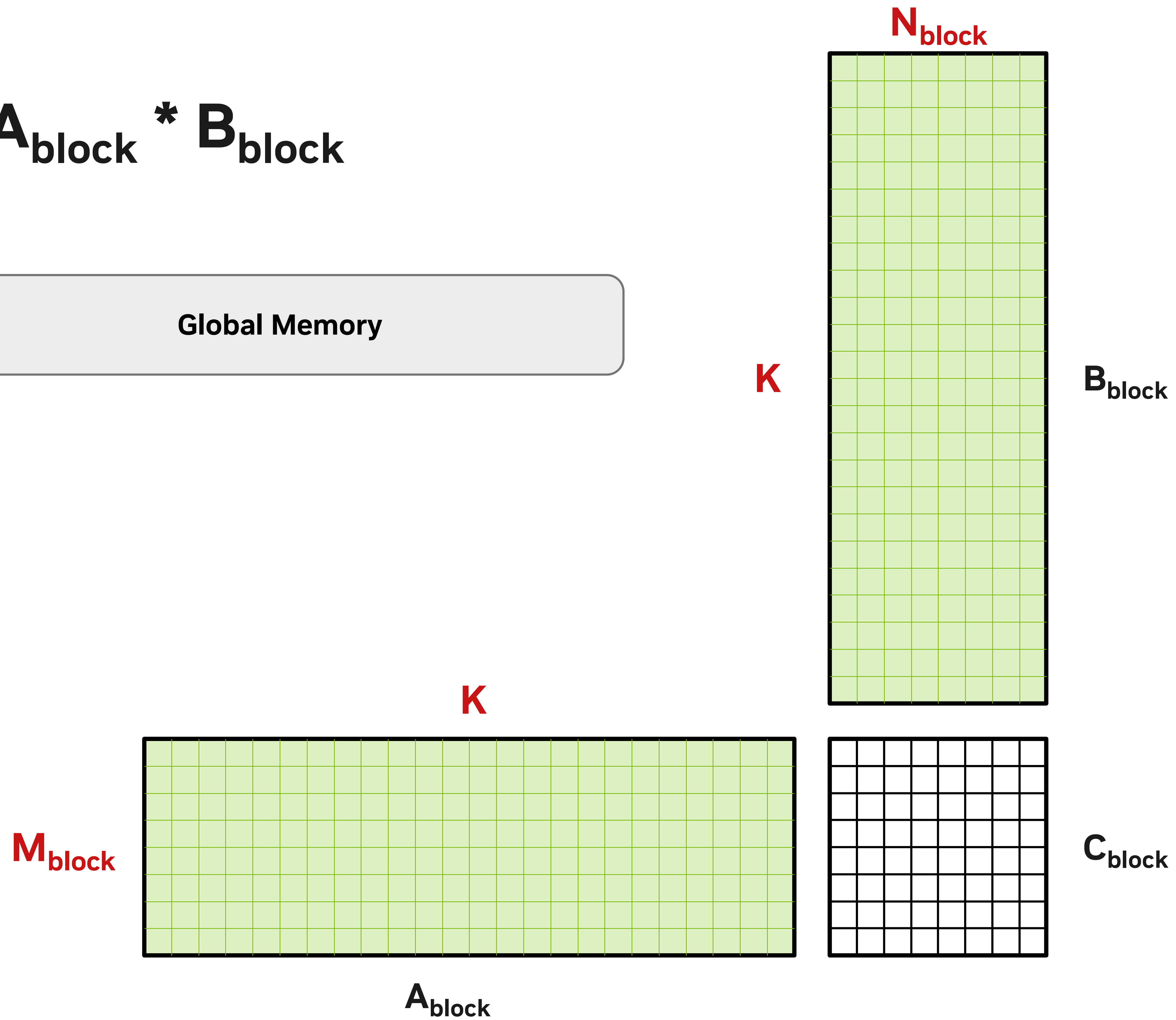
$$C_{\text{block}} = A_{\text{block}} * B_{\text{block}}$$

Global Memory

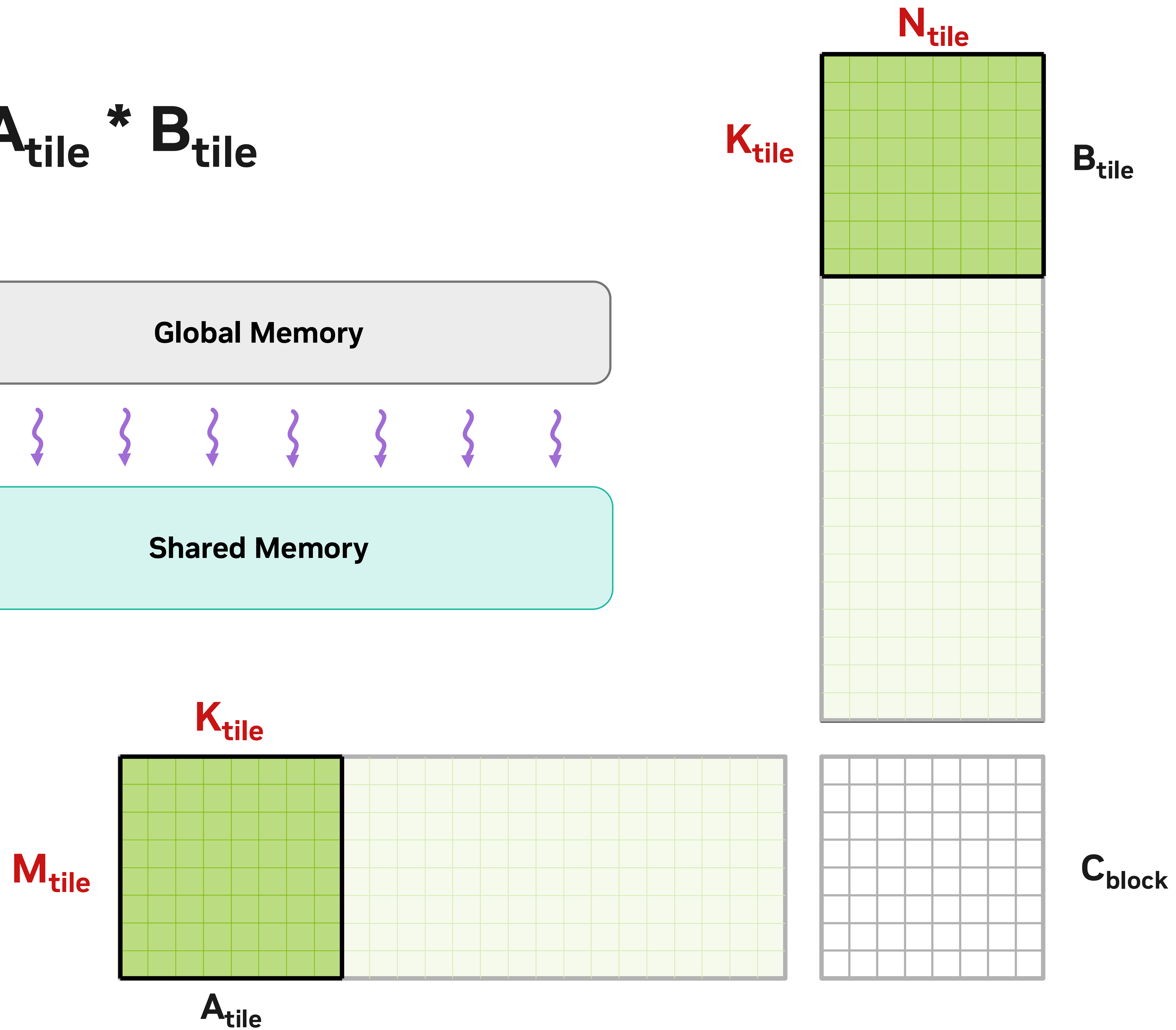
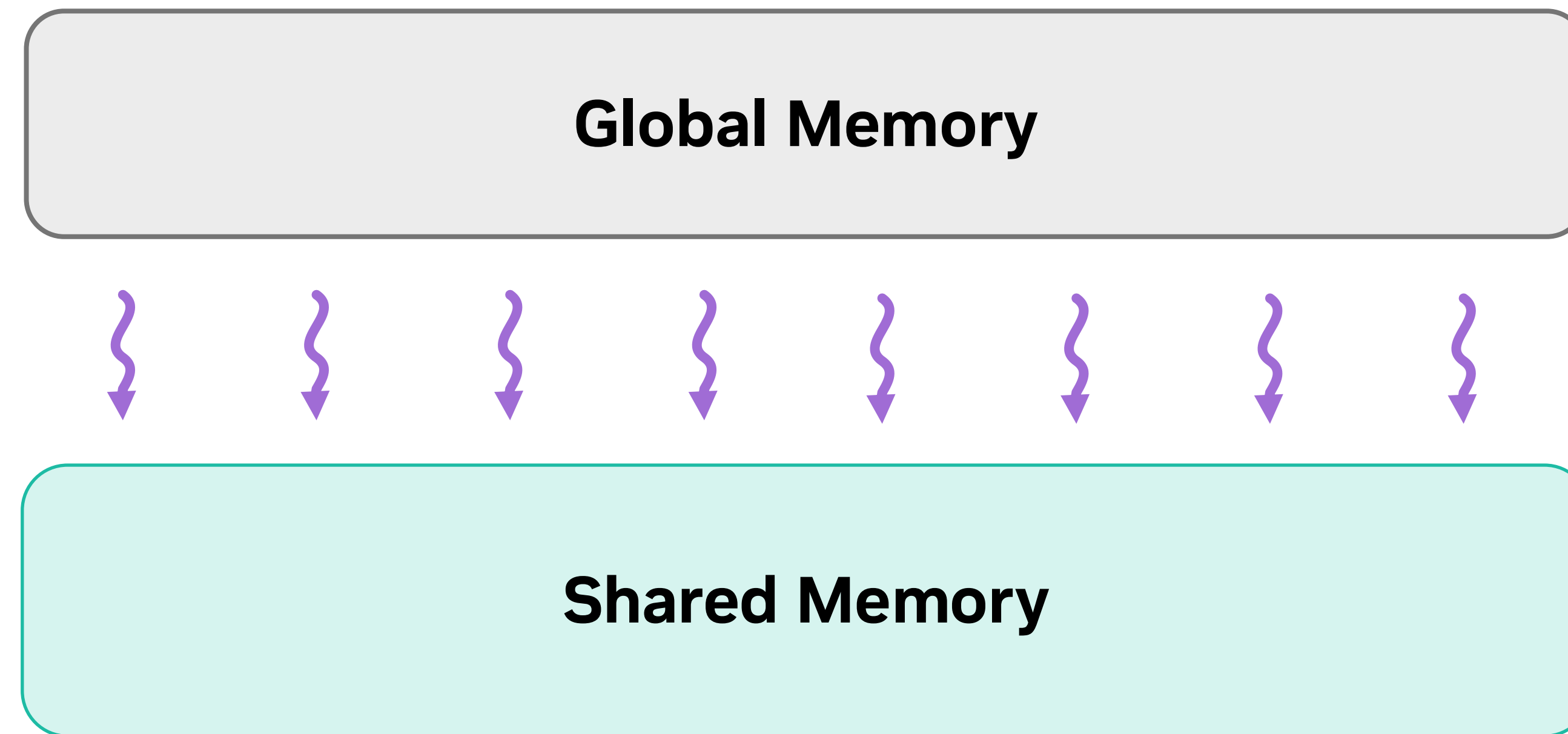


$$C_{\text{block}} = A_{\text{block}} * B_{\text{block}}$$

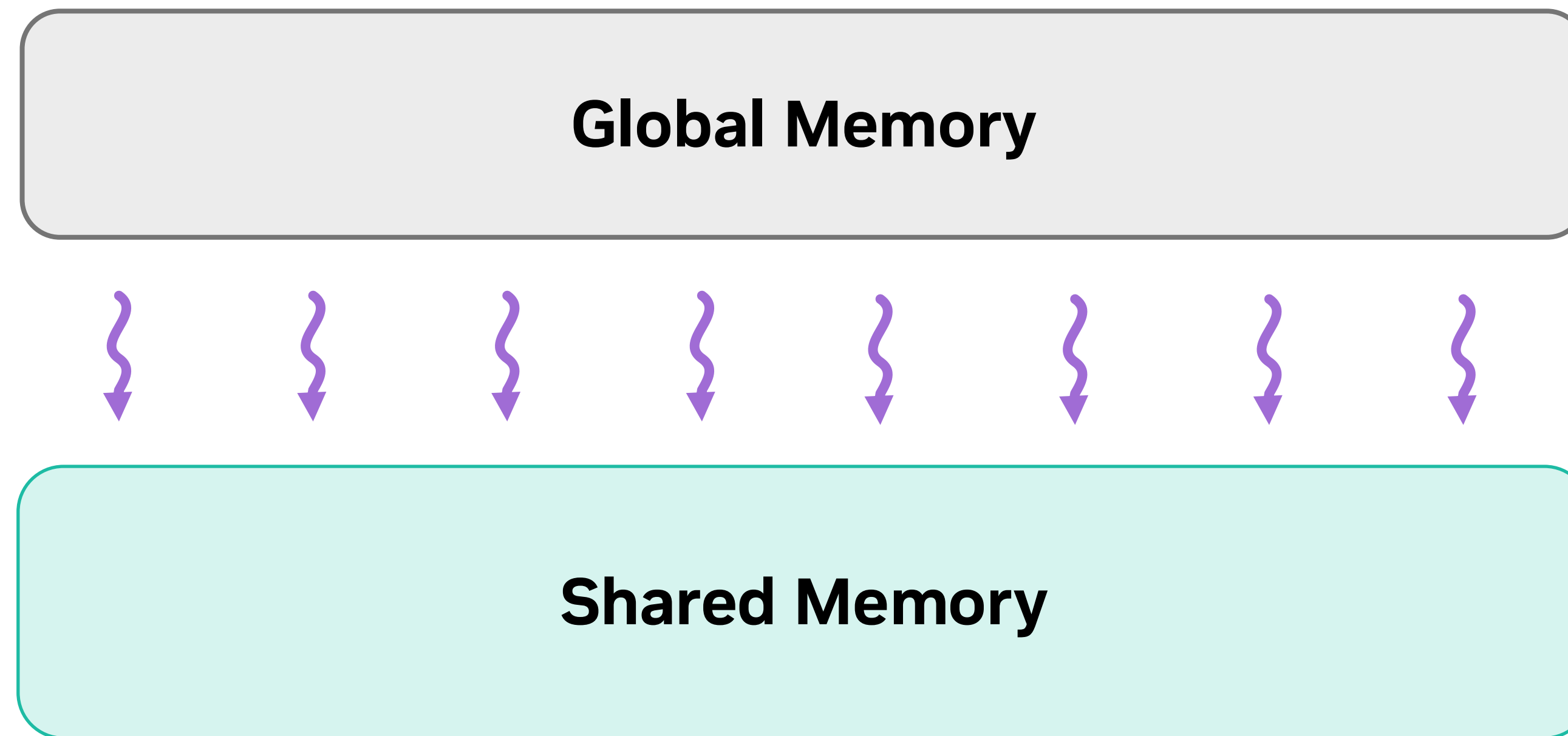
Global Memory



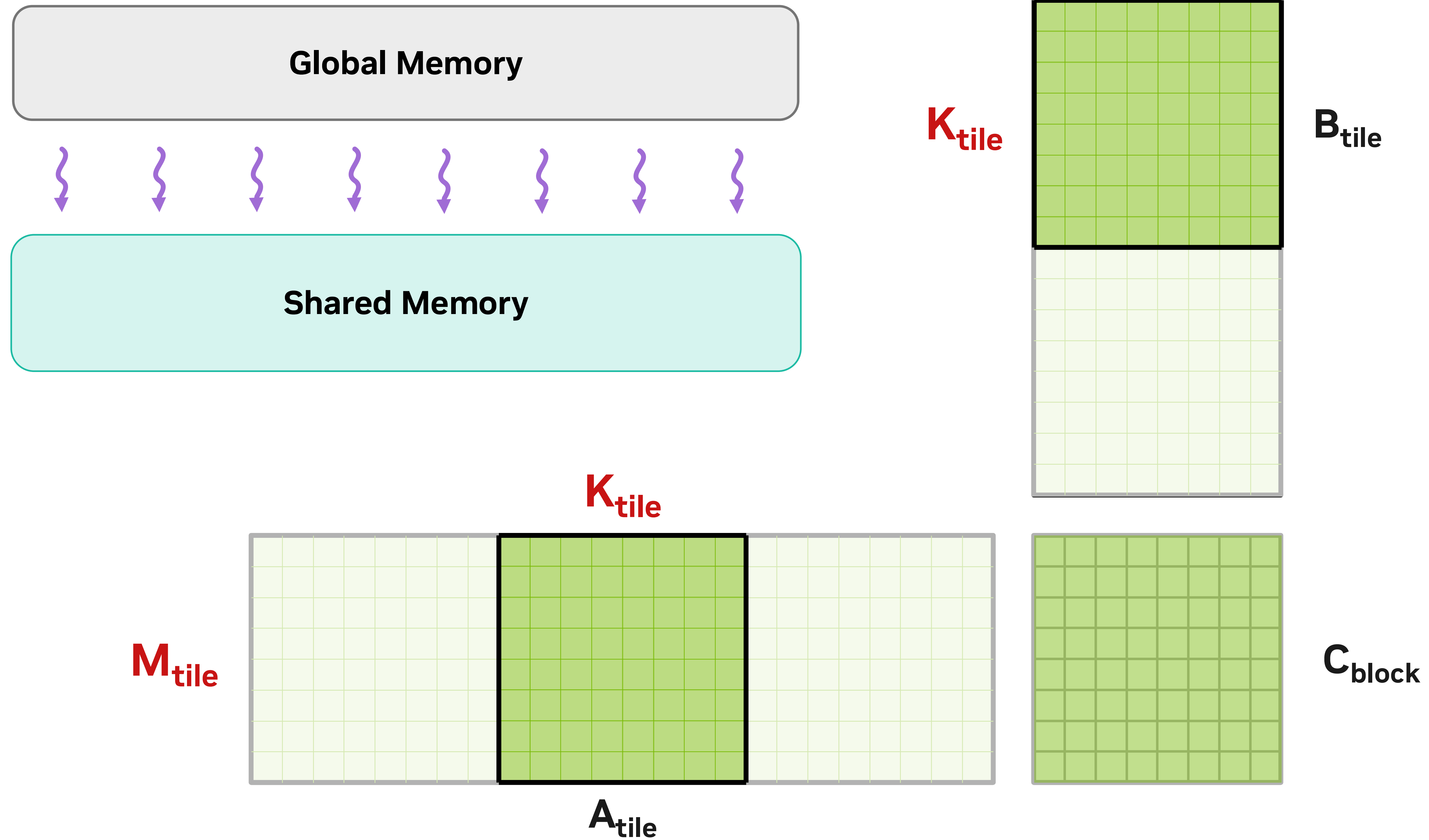
$$C_{\text{tile}} = A_{\text{tile}} * B_{\text{tile}}$$



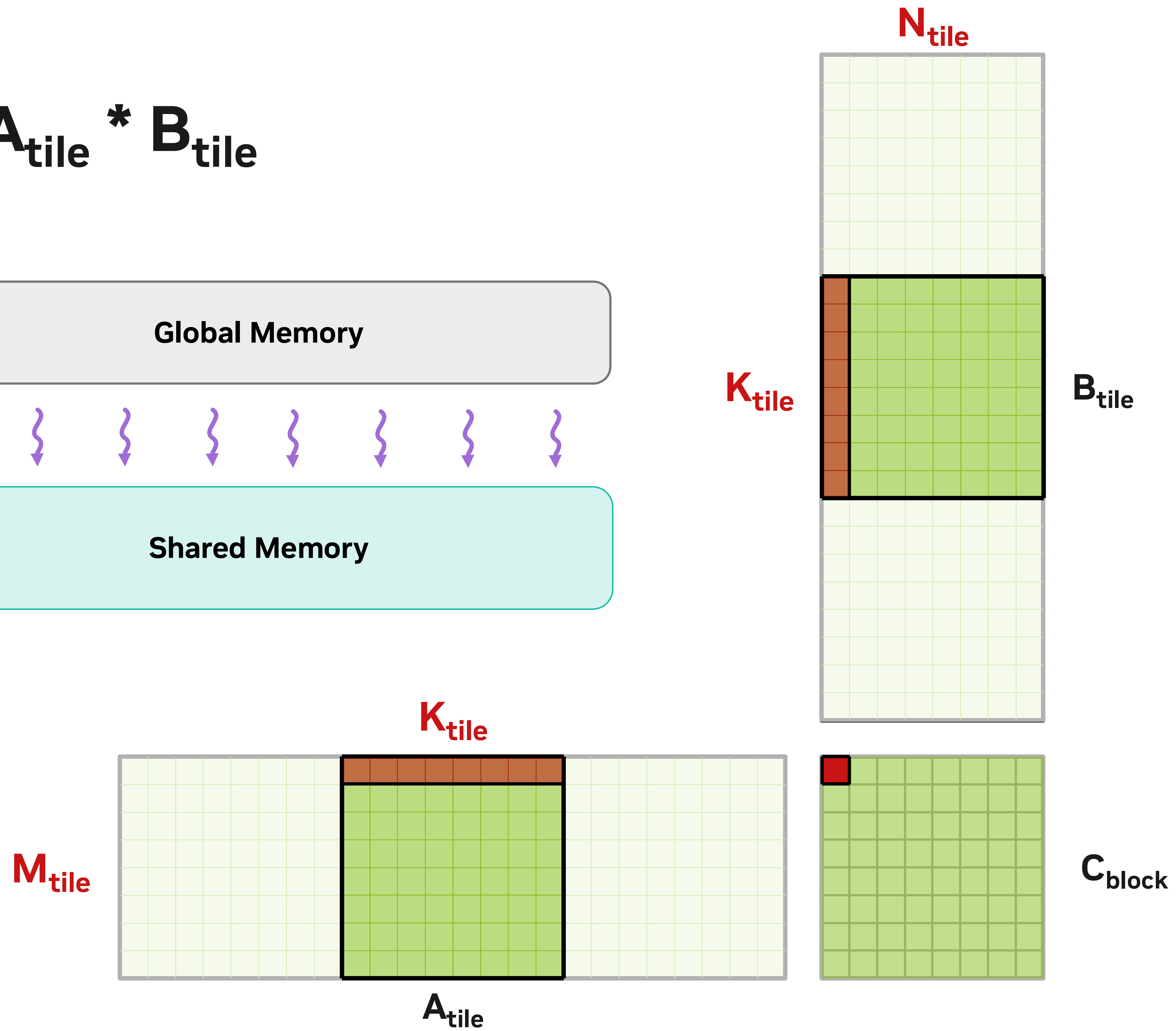
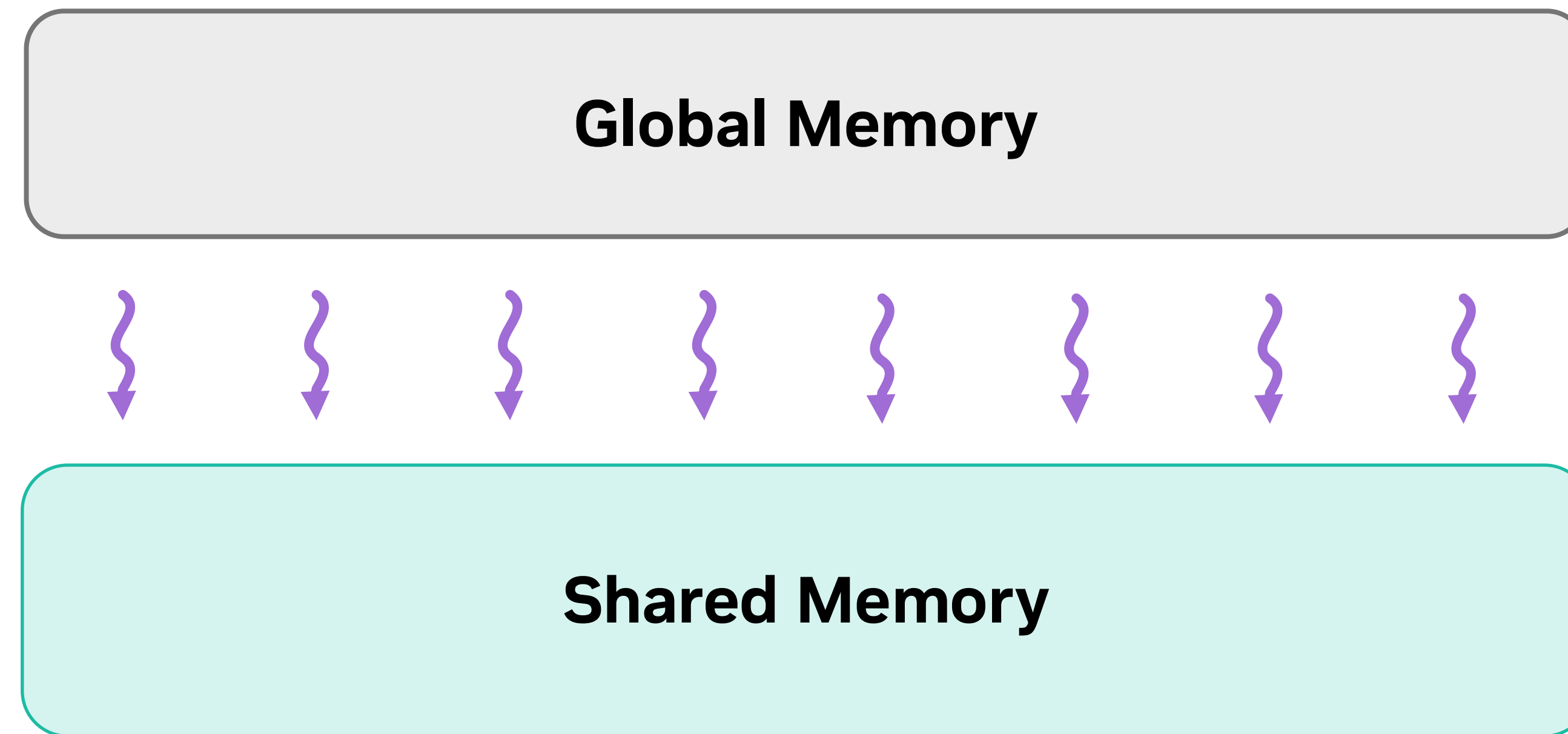
$$C_{\text{tile}} = A_{\text{tile}} * B_{\text{tile}}$$



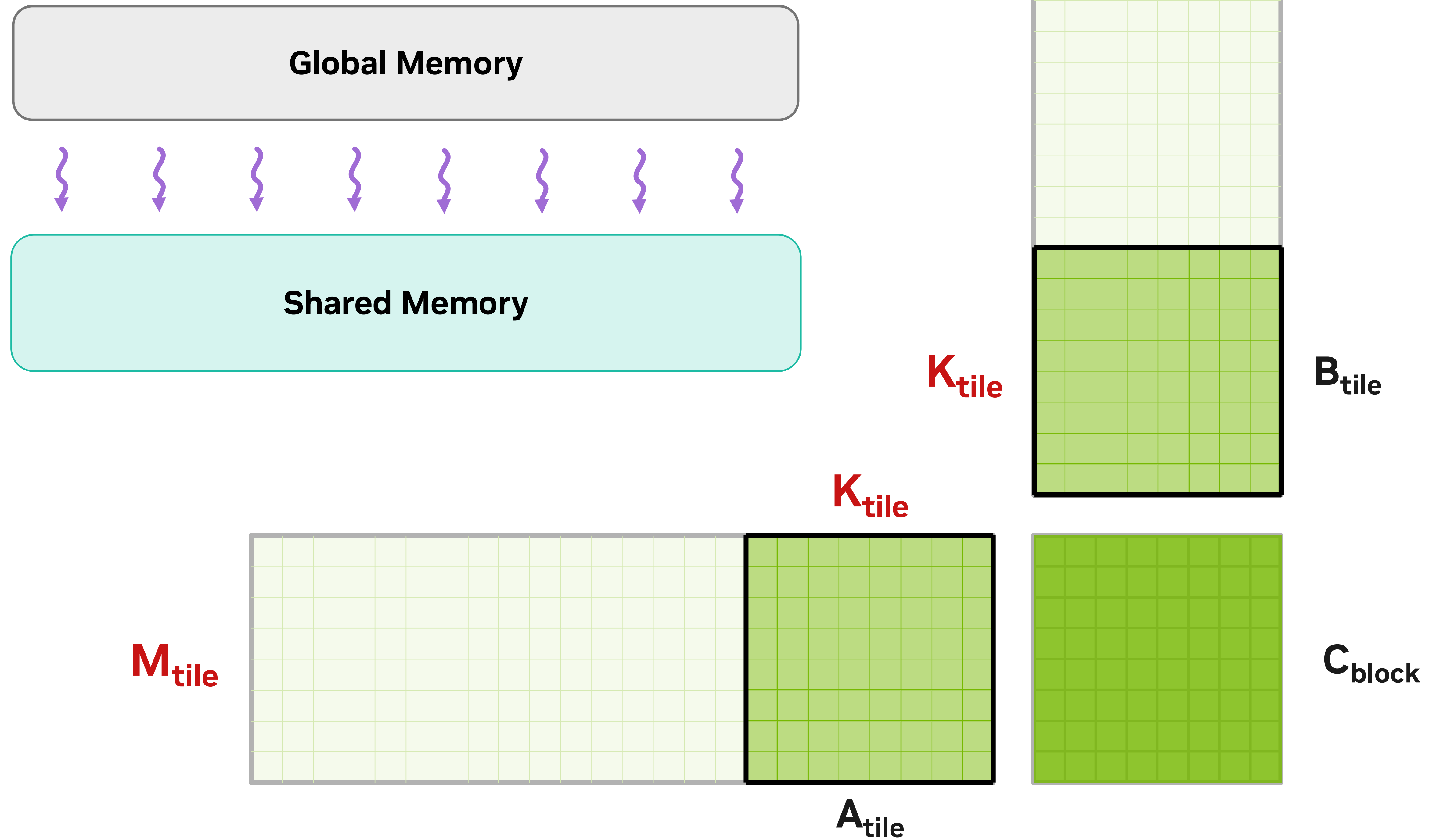
$$C_{\text{tile}} = A_{\text{tile}} * B_{\text{tile}}$$



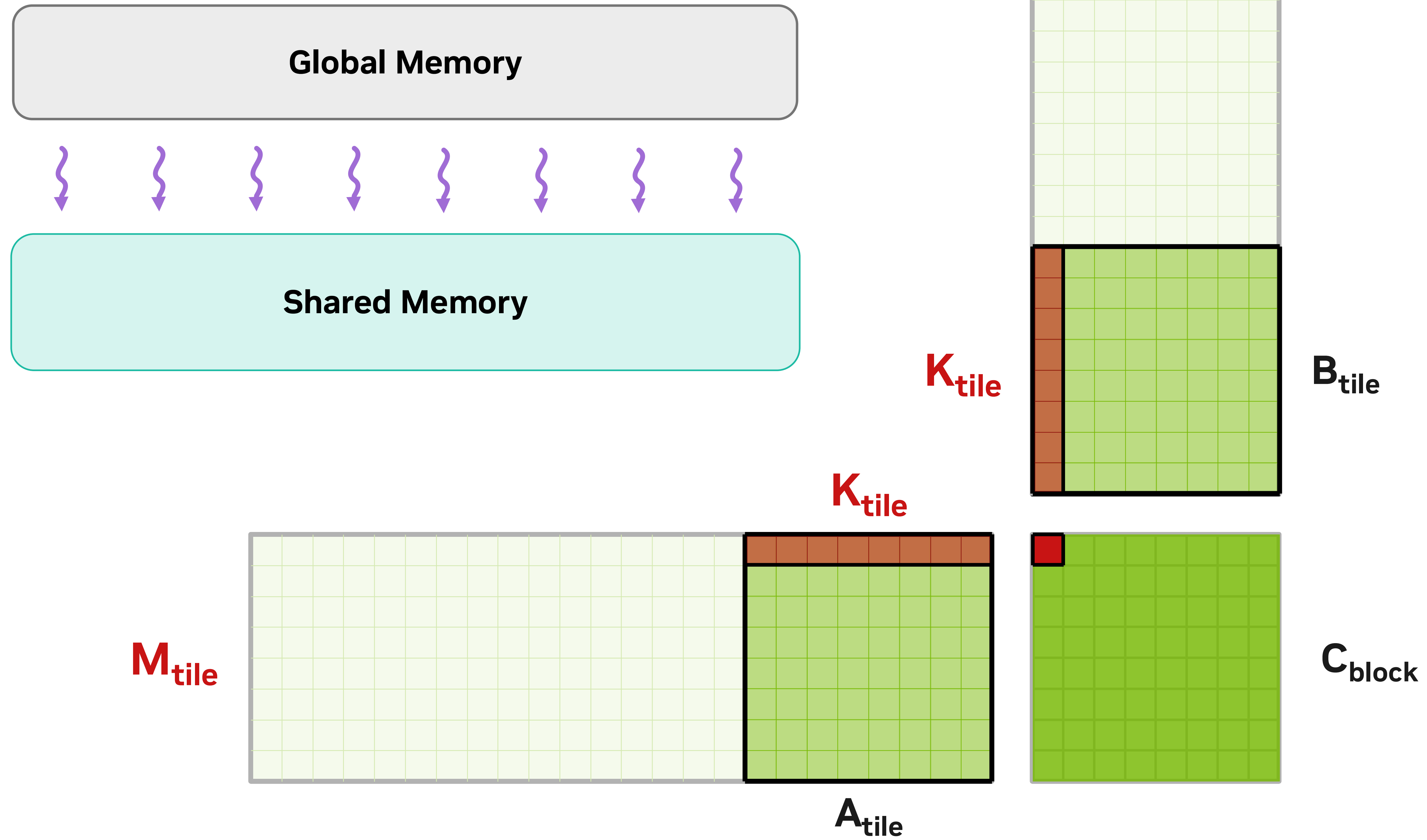
$$C_{\text{tile}} = A_{\text{tile}} * B_{\text{tile}}$$



$$C_{\text{tile}} = A_{\text{tile}} * B_{\text{tile}}$$



$$C_{\text{tile}} = A_{\text{tile}} * B_{\text{tile}}$$



SIMT: Textbook Matrix Multiply

```
__global__ void matmul(__half const* aPtr, __half const* bPtr,
                      __half* cPtr, size_t M, size_t N, size_t K) {
    auto A = cuda::std::mdspan(aPtr, cuda::std::extents{M, K});
    auto B = cuda::std::mdspan(bPtr, cuda::std::extents{K, N});
    auto C = cuda::std::mdspan(cPtr, cuda::std::extents{M, N});
    int lX = threadIdx.y, lY = threadIdx.x;
    int gX = 16 * blockIdx.x + lX, gY = 16 * blockIdx.y + lY;
    __half acc{0};

    __shared__ __half A_S[16][16], B_S[16][16];

    for (size_t tl = 0; tl != K; tl += 16) {
        A_S[lX][lY] = A(gX, tl + lY);
        B_S[lX][lY] = B(tl + lX, gY);
        __syncthreads();

        for (size_t idx = 0; idx != 16; ++idx) {
            acc += A_S[lX][idx] * B_S[idx][lY];
        }
        __syncthreads();
    }
    C(gX, gY) = acc;
}
```

SIMT: Textbook Matrix Multiply

```
__global__ void matmul(__half const* aPtr, __half const* bPtr,
                      __half* cPtr, size_t M, size_t N, size_t K) {
    auto A = cuda::std::mdspan(aPtr, cuda::std::extents{M, K});
    auto B = cuda::std::mdspan(bPtr, cuda::std::extents{K, N});
    auto C = cuda::std::mdspan(cPtr, cuda::std::extents{M, N});
    int lX = threadIdx.y, lY = threadIdx.x;
    int gX = 16 * blockIdx.x + lX, gY = 16 * blockIdx.y + lY;
    __half acc{0};

    __shared__ __half A_S[16][16], B_S[16][16];

    for (size_t tl = 0; tl != K; tl += 16) {
        A_S[lX][lY] = A(gX, tl + lY);
        B_S[lX][lY] = B(tl + lX, gY);
        __syncthreads();

        for (size_t idx = 0; idx != 16; ++idx) {
            acc += A_S[lX][idx] * B_S[idx][lY];
        }
        __syncthreads();
    }
    C(gX, gY) = acc;
}
```

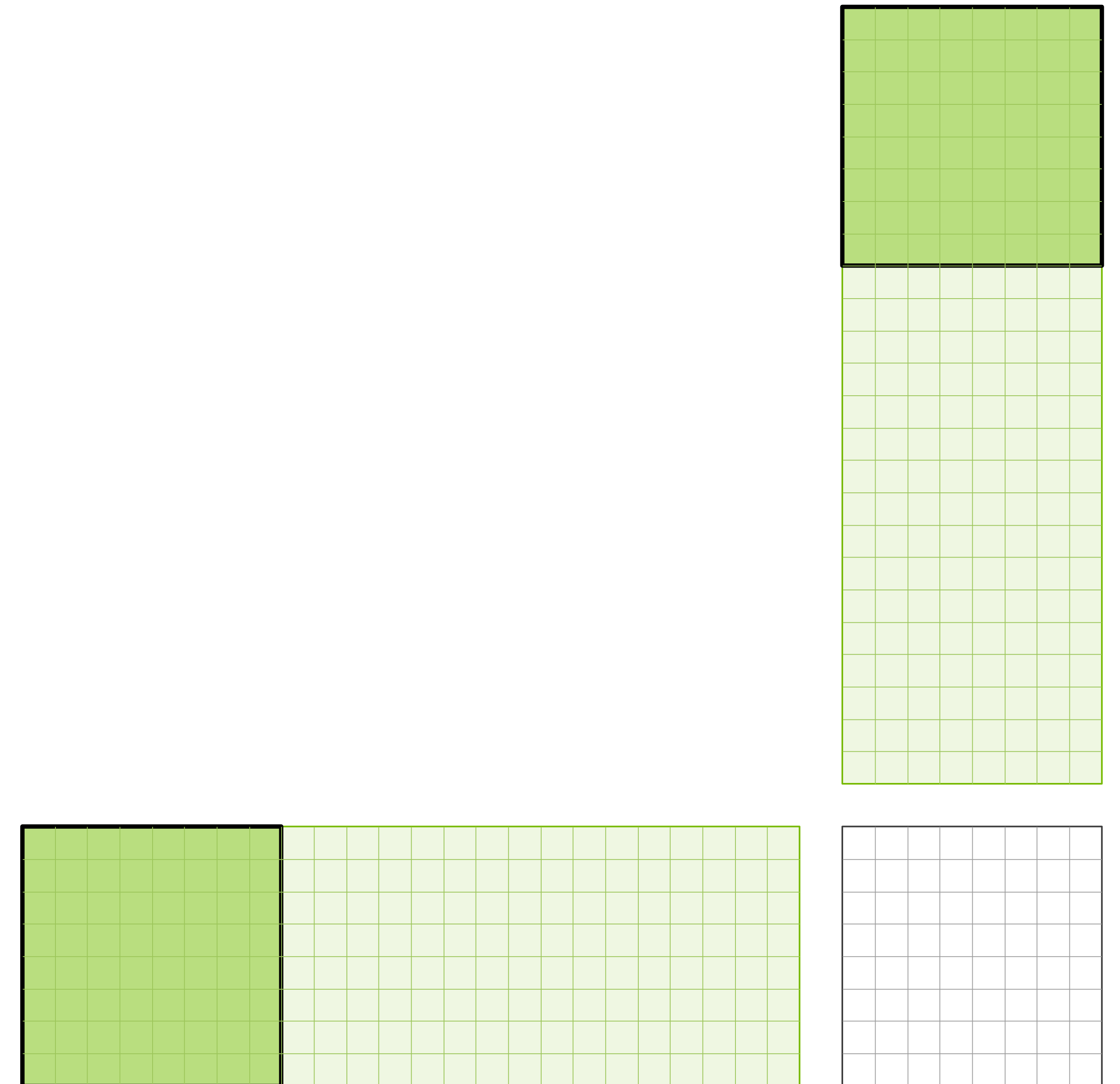
SIMT: Textbook Matrix Multiply

```
__global__ void matmul(__half const* aPtr, __half const* bPtr,
                      __half* cPtr, size_t M, size_t N, size_t K) {
    auto A = cuda::std::mdspan(aPtr, cuda::std::extents{M, K});
    auto B = cuda::std::mdspan(bPtr, cuda::std::extents{K, N});
    auto C = cuda::std::mdspan(cPtr, cuda::std::extents{M, N});
    int lX = threadIdx.y,      lY = threadIdx.x;
    int gX = 16 * blockIdx.x + lX, gY = 16 * blockIdx.y + lY;
    __half acc{0};

    __shared__ __half A_S[16][16], B_S[16][16];

    for (size_t tl = 0; tl != K; tl += 16) {
        A_S[lX][lY] = A(gX,      tl + lY);
        B_S[lX][lY] = B(tl + lX, gY);
        __syncthreads();

        for (size_t idx = 0; idx != 16; ++idx) {
            acc += A_S[lX][idx] * B_S[idx][lY];
        }
        __syncthreads();
    }
    C(gX, gY) = acc;
}
```



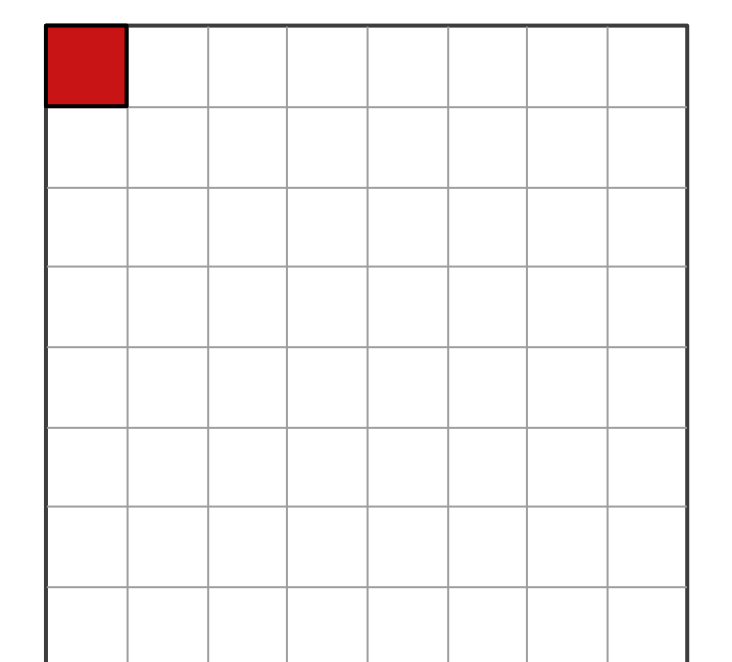
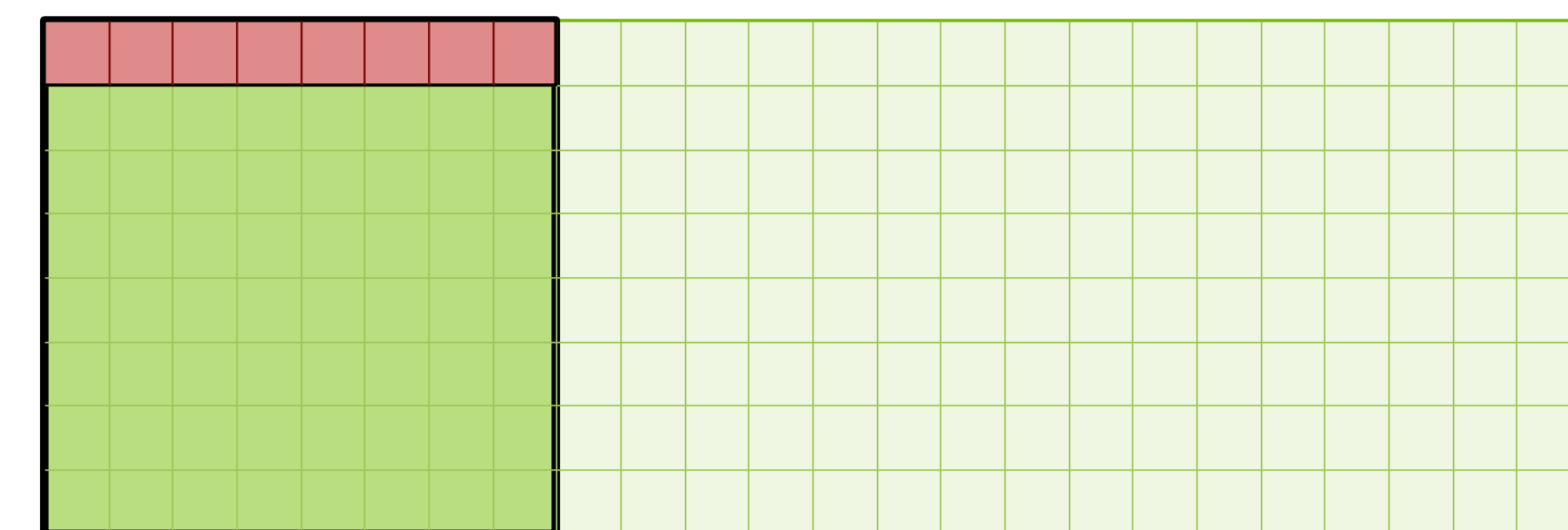
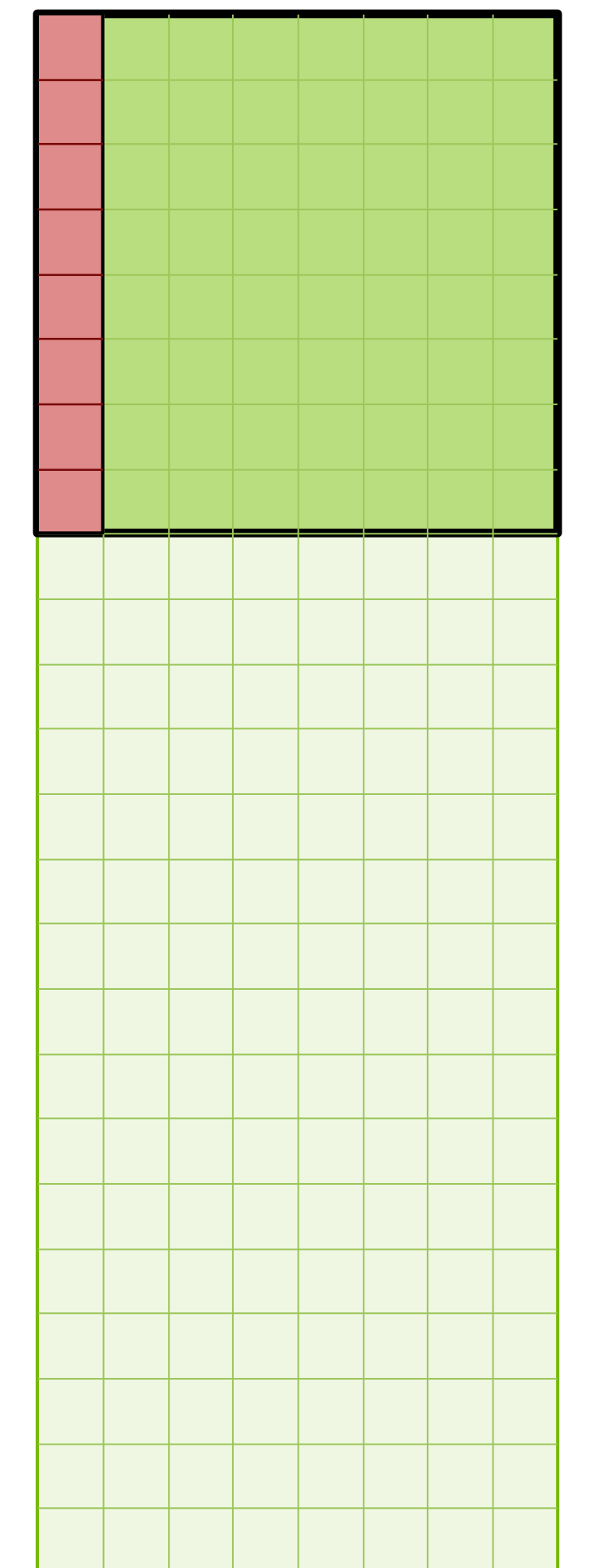
SIMT: Textbook Matrix Multiply

```
__global__ void matmul(__half const* aPtr, __half const* bPtr,
                      __half* cPtr, size_t M, size_t N, size_t K) {
    auto A = cuda::std::mdspan(aPtr, cuda::std::extents{M, K});
    auto B = cuda::std::mdspan(bPtr, cuda::std::extents{K, N});
    auto C = cuda::std::mdspan(cPtr, cuda::std::extents{M, N});
    int lX = threadIdx.y, lY = threadIdx.x;
    int gX = 16 * blockIdx.x + lX, gY = 16 * blockIdx.y + lY;
    __half acc{0};

    __shared__ __half A_S[16][16], B_S[16][16];

    for (size_t tl = 0; tl != K; tl += 16) {
        A_S[lX][lY] = A(gX, tl + lY);
        B_S[lX][lY] = B(tl + lX, gY);
        __syncthreads();

        for (size_t idx = 0; idx != 16; ++idx) {
            acc += A_S[lX][idx] * B_S[idx][lY];
        }
        __syncthreads();
    }
    C(gX, gY) = acc;
}
```



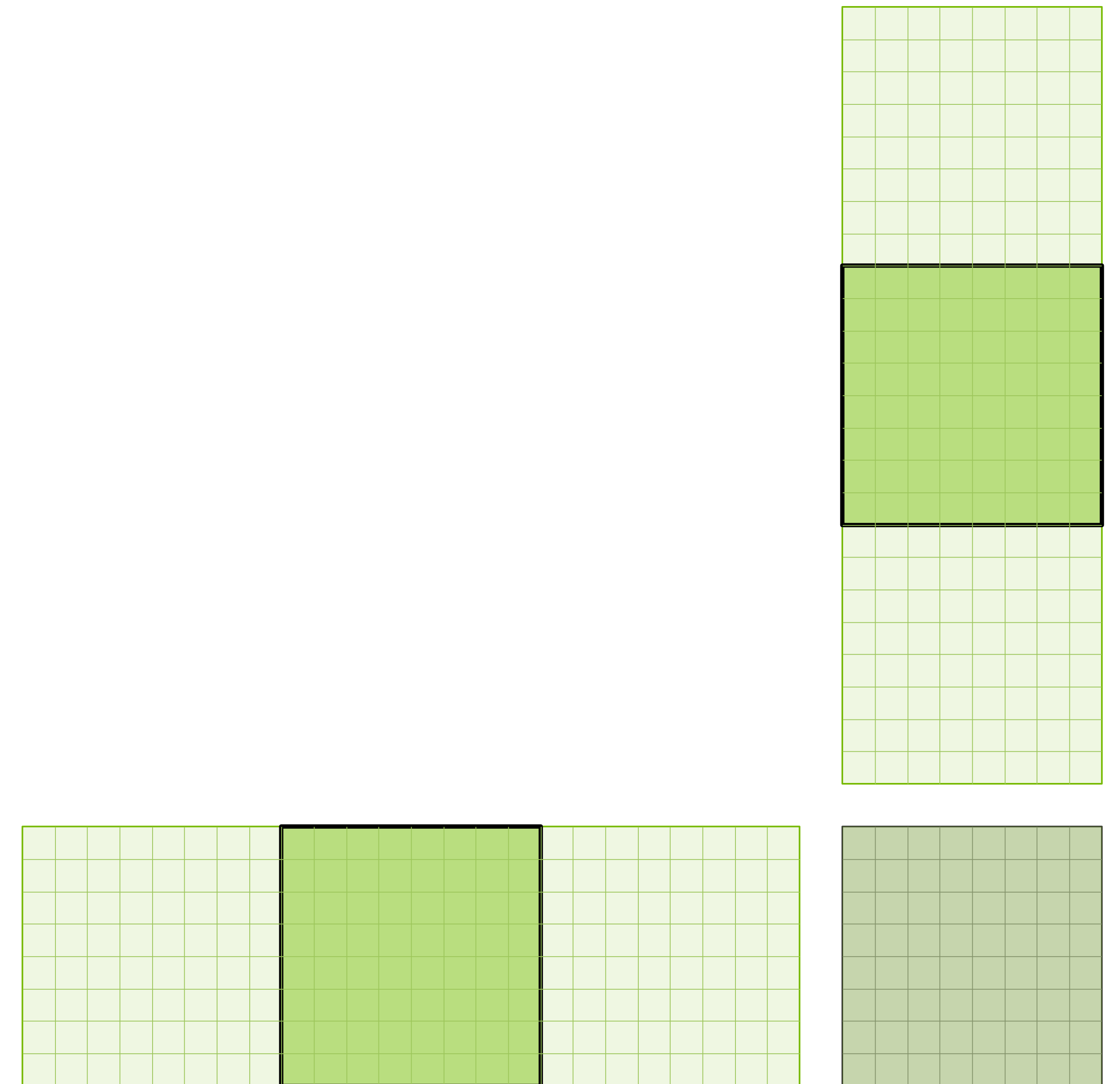
SIMT: Textbook Matrix Multiply

```
__global__ void matmul(__half const* aPtr, __half const* bPtr,
                      __half* cPtr, size_t M, size_t N, size_t K) {
    auto A = cuda::std::mdspan(aPtr, cuda::std::extents{M, K});
    auto B = cuda::std::mdspan(bPtr, cuda::std::extents{K, N});
    auto C = cuda::std::mdspan(cPtr, cuda::std::extents{M, N});
    int lX = threadIdx.y,      lY = threadIdx.x;
    int gX = 16 * blockIdx.x + lX, gY = 16 * blockIdx.y + lY;
    __half acc{0};

    __shared__ __half A_S[16][16], B_S[16][16];

    for (size_t tl = 0; tl != K; tl += 16) {
        A_S[lX][lY] = A(gX,      tl + lY);
        B_S[lX][lY] = B(tl + lX, gY);
        __syncthreads();

        for (size_t idx = 0; idx != 16; ++idx) {
            acc += A_S[lX][idx] * B_S[idx][lY];
        }
        __syncthreads();
    }
    C(gX, gY) = acc;
}
```



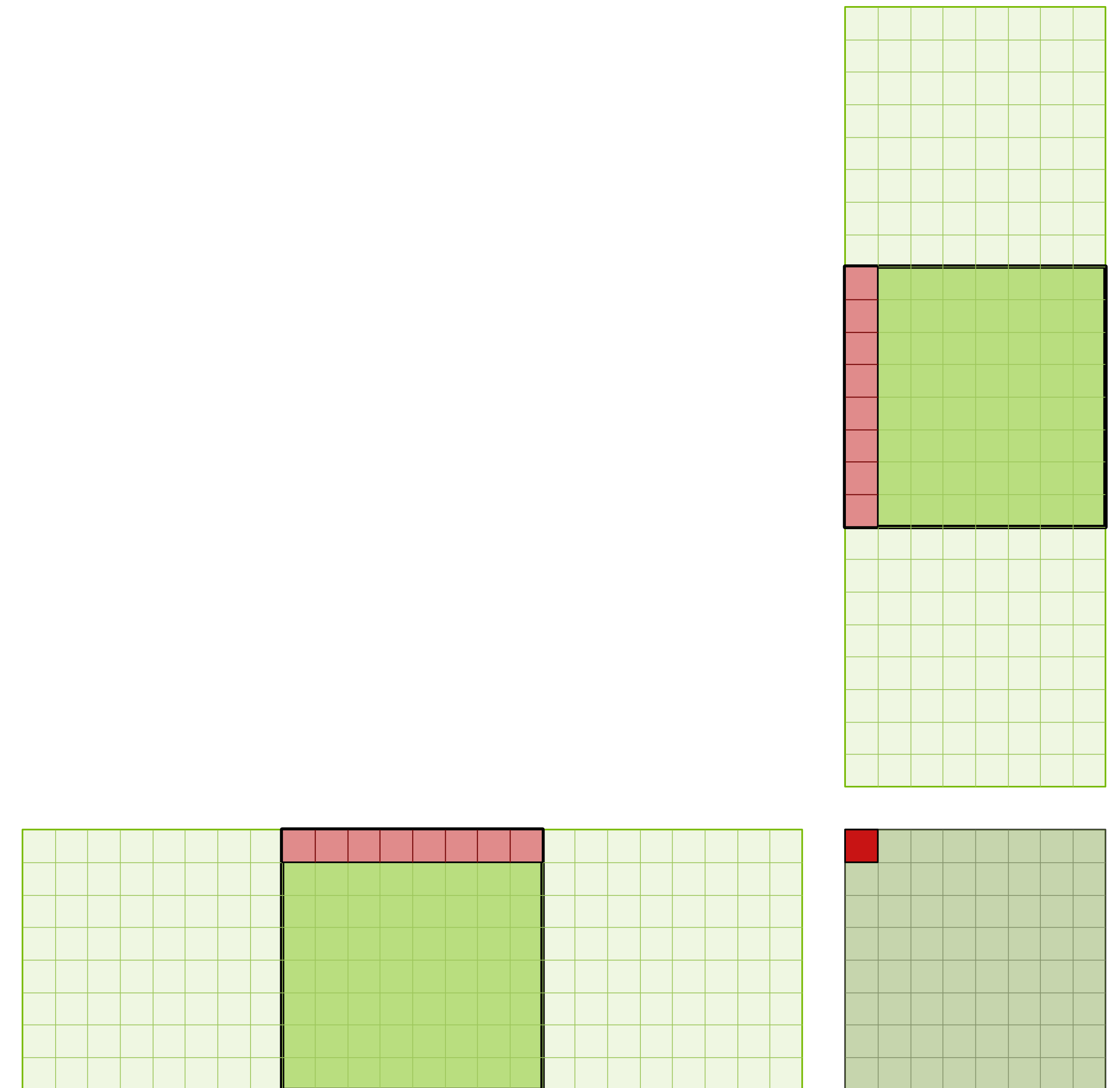
SIMT: Textbook Matrix Multiply

```
__global__ void matmul(__half const* aPtr, __half const* bPtr,
                      __half* cPtr, size_t M, size_t N, size_t K) {
    auto A = cuda::std::mdspan(aPtr, cuda::std::extents{M, K});
    auto B = cuda::std::mdspan(bPtr, cuda::std::extents{K, N});
    auto C = cuda::std::mdspan(cPtr, cuda::std::extents{M, N});
    int lX = threadIdx.y, lY = threadIdx.x;
    int gX = 16 * blockIdx.x + lX, gY = 16 * blockIdx.y + lY;
    __half acc{0};

    __shared__ __half A_S[16][16], B_S[16][16];

    for (size_t tl = 0; tl != K; tl += 16) {
        A_S[lX][lY] = A(gX, tl + lY);
        B_S[lX][lY] = B(tl + lX, gY);
        __syncthreads();

        for (size_t idx = 0; idx != 16; ++idx) {
            acc += A_S[lX][idx] * B_S[idx][lY];
        }
        __syncthreads();
    }
    C(gX, gY) = acc;
}
```



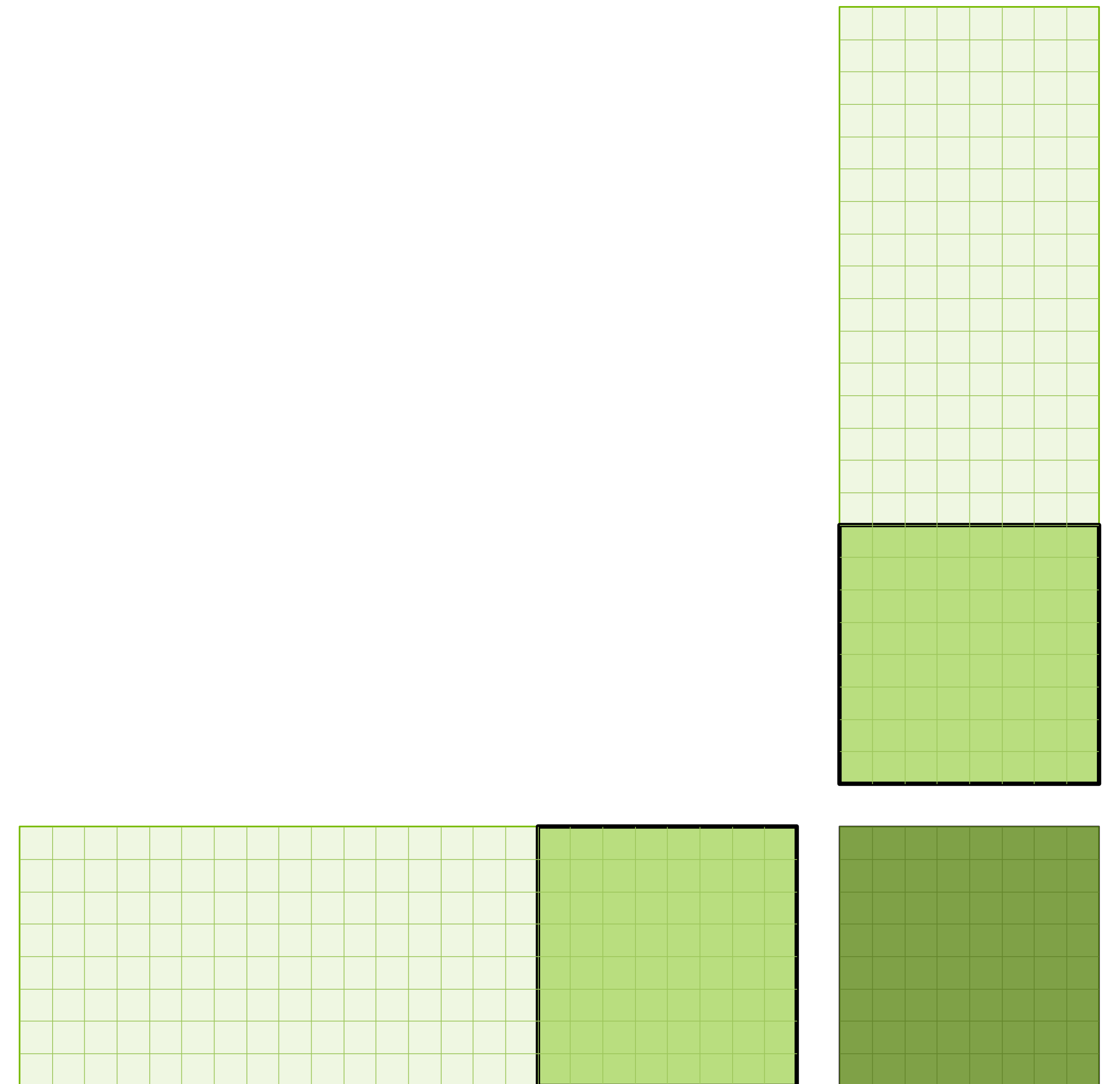
SIMT: Textbook Matrix Multiply

```
__global__ void matmul(__half const* aPtr, __half const* bPtr,
                      __half* cPtr, size_t M, size_t N, size_t K) {
    auto A = cuda::std::mdspan(aPtr, cuda::std::extents{M, K});
    auto B = cuda::std::mdspan(bPtr, cuda::std::extents{K, N});
    auto C = cuda::std::mdspan(cPtr, cuda::std::extents{M, N});
    int lX = threadIdx.y,      lY = threadIdx.x;
    int gX = 16 * blockIdx.x + lX, gY = 16 * blockIdx.y + lY;
    __half acc{0};

    __shared__ __half A_S[16][16], B_S[16][16];

    for (size_t tl = 0; tl != K; tl += 16) {
        A_S[lX][lY] = A(gX,      tl + lY);
        B_S[lX][lY] = B(tl + lX, gY);
        __syncthreads();

        for (size_t idx = 0; idx != 16; ++idx) {
            acc += A_S[lX][idx] * B_S[idx][lY];
        }
        __syncthreads();
    }
    C(gX, gY) = acc;
}
```



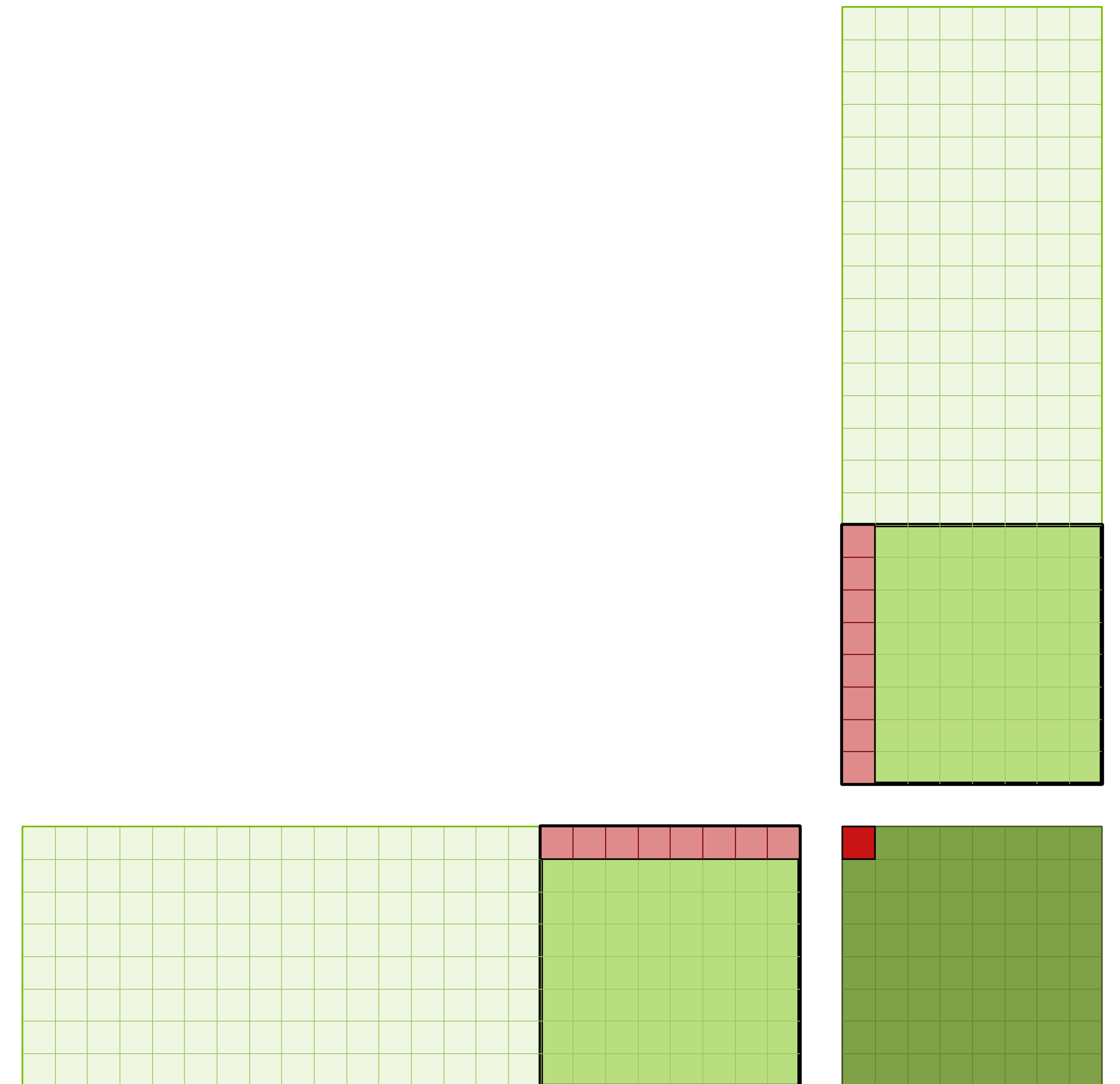
SIMT: Textbook Matrix Multiply

```
__global__ void matmul(__half const* aPtr, __half const* bPtr,
                      __half* cPtr, size_t M, size_t N, size_t K) {
    auto A = cuda::std::mdspan(aPtr, cuda::std::extents{M, K});
    auto B = cuda::std::mdspan(bPtr, cuda::std::extents{K, N});
    auto C = cuda::std::mdspan(cPtr, cuda::std::extents{M, N});
    int lX = threadIdx.y,      lY = threadIdx.x;
    int gX = 16 * blockIdx.x + lX, gY = 16 * blockIdx.y + lY;
    __half acc{0};

    __shared__ __half A_S[16][16], B_S[16][16];

    for (size_t tl = 0; tl != K; tl += 16) {
        A_S[lX][lY] = A(gX,      tl + lY);
        B_S[lX][lY] = B(tl + lX, gY);
        __syncthreads();

        for (size_t idx = 0; idx != 16; ++idx) {
            acc += A_S[lX][idx] * B_S[idx][lY];
        }
        __syncthreads();
    }
    C(gX, gY) = acc;
}
```



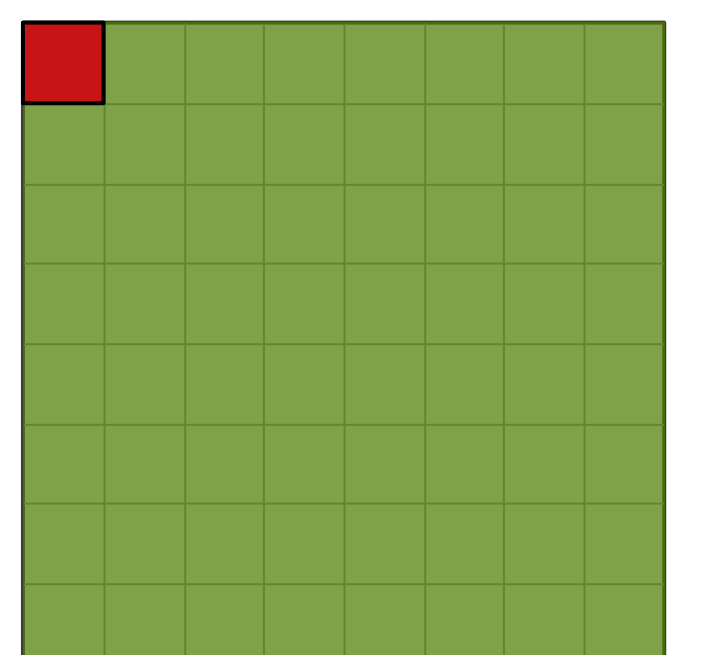
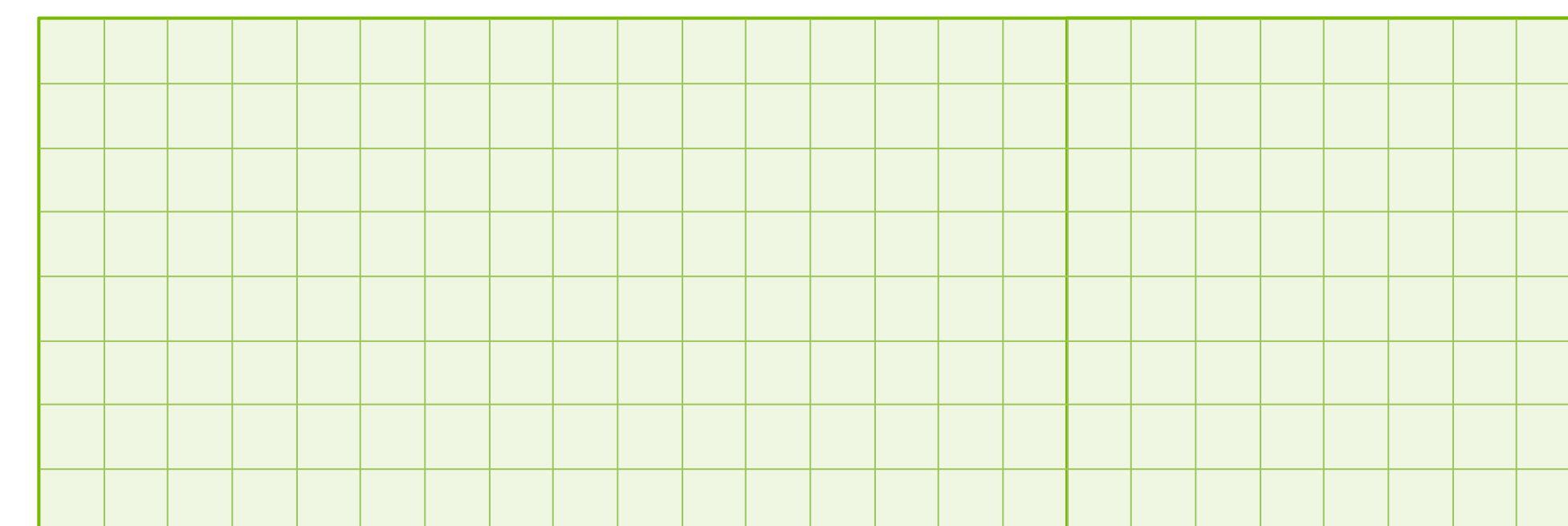
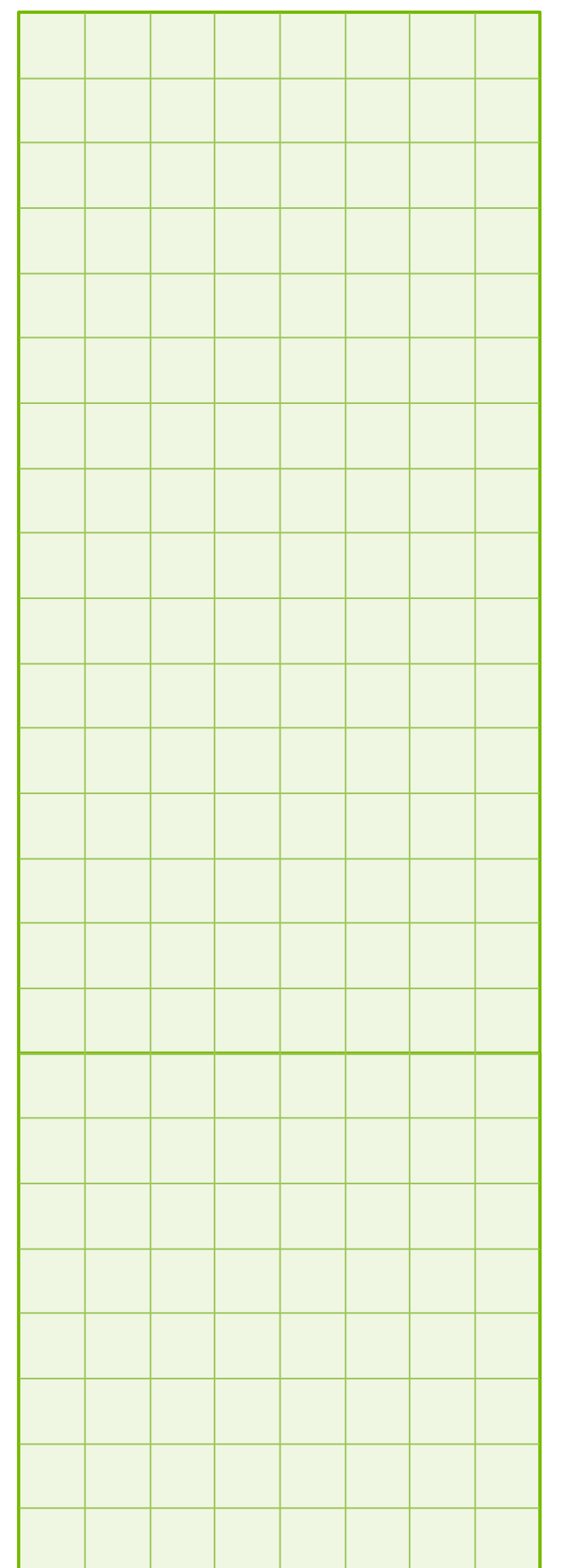
SIMT: Textbook Matrix Multiply

```
__global__ void matmul(__half const* aPtr, __half const* bPtr,
                      __half* cPtr, size_t M, size_t N, size_t K) {
    auto A = cuda::std::mdspan(aPtr, cuda::std::extents{M, K});
    auto B = cuda::std::mdspan(bPtr, cuda::std::extents{K, N});
    auto C = cuda::std::mdspan(cPtr, cuda::std::extents{M, N});
    int lX = threadIdx.y,      lY = threadIdx.x;
    int gX = 16 * blockIdx.x + lX, gY = 16 * blockIdx.y + lY;
    __half acc{0};

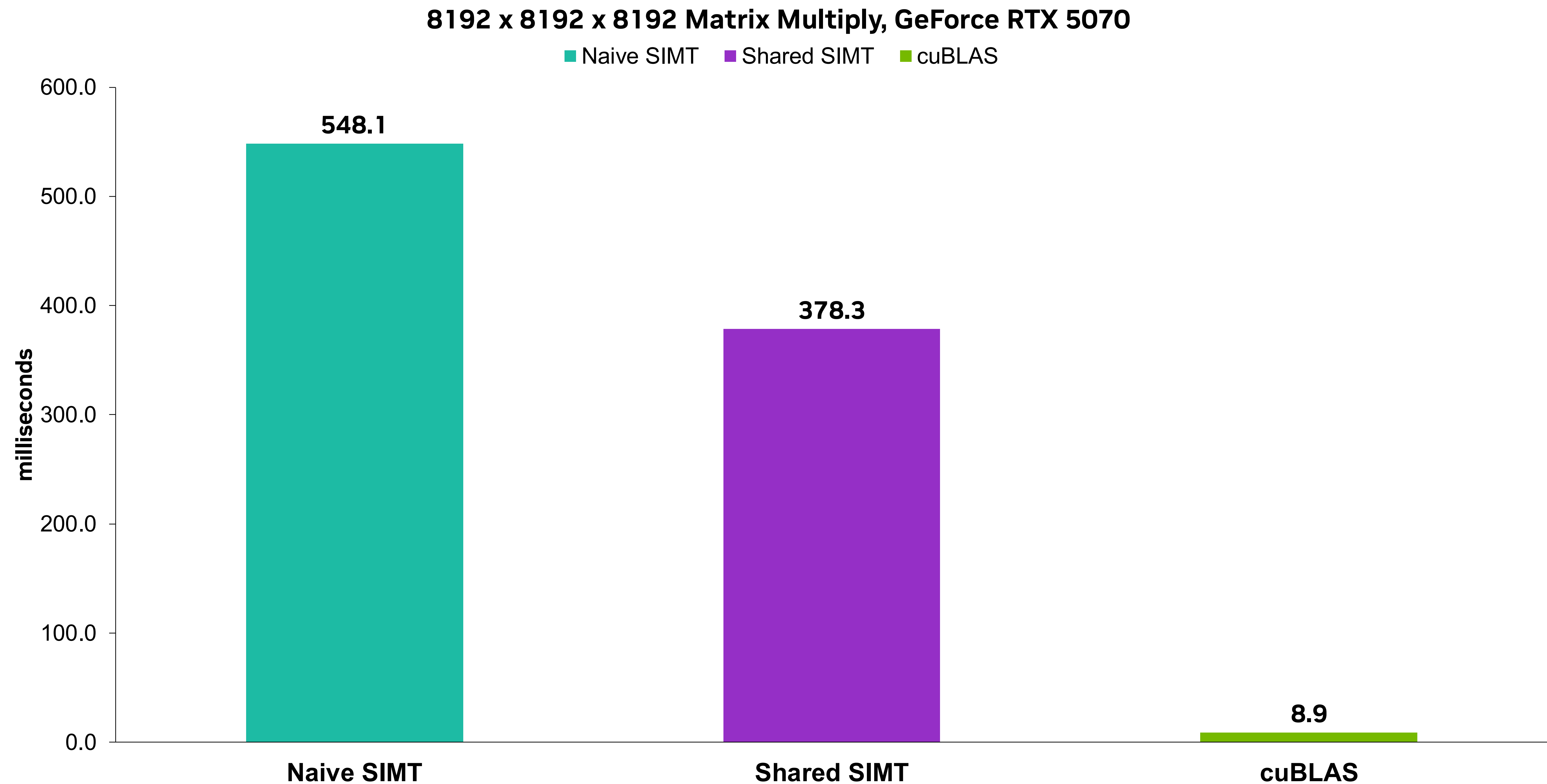
    __shared__ __half A_S[16][16], B_S[16][16];

    for (size_t tl = 0; tl != K; tl += 16) {
        A_S[lX][lY] = A(gX,      tl + lY);
        B_S[lX][lY] = B(tl + lX, gY);
        __syncthreads();

        for (size_t idx = 0; idx != 16; ++idx) {
            acc += A_S[lX][idx] * B_S[idx][lY];
        }
        __syncthreads();
    }
    C(gX, gY) = acc;
}
```



SIMT: Shared Memory Matrix Multiply Performance



SIMT: Textbook Matrix Multiply

```
__global__ void matmul(__half const* aPtr, __half const* bPtr,
                      __half* cPtr, size_t M, size_t N, size_t K) {
    auto A = cuda::std::mdspan(aPtr, cuda::std::extents{M, K});
    auto B = cuda::std::mdspan(bPtr, cuda::std::extents{K, N});
    auto C = cuda::std::mdspan(cPtr, cuda::std::extents{M, N});
    int lX = threadIdx.y, lY = threadIdx.x;
    int gX = 16 * blockIdx.x + lX, gY = 16 * blockIdx.y + lY;
    __half acc{0};

    __shared__ __half A_S[16][16], B_S[16][16];

    for (size_t tl = 0; tl != K; tl += 16) {
        A_S[lX][lY] = A(gX, tl + lY);
        B_S[lX][lY] = B(tl + lX, gY);
        __syncthreads();

        for (size_t idx = 0; idx != 16; ++idx) {
            acc += A_S[lX][idx] * B_S[idx][lY];
        }
        __syncthreads();
    }
    C(gX, gY) = acc;
}
```

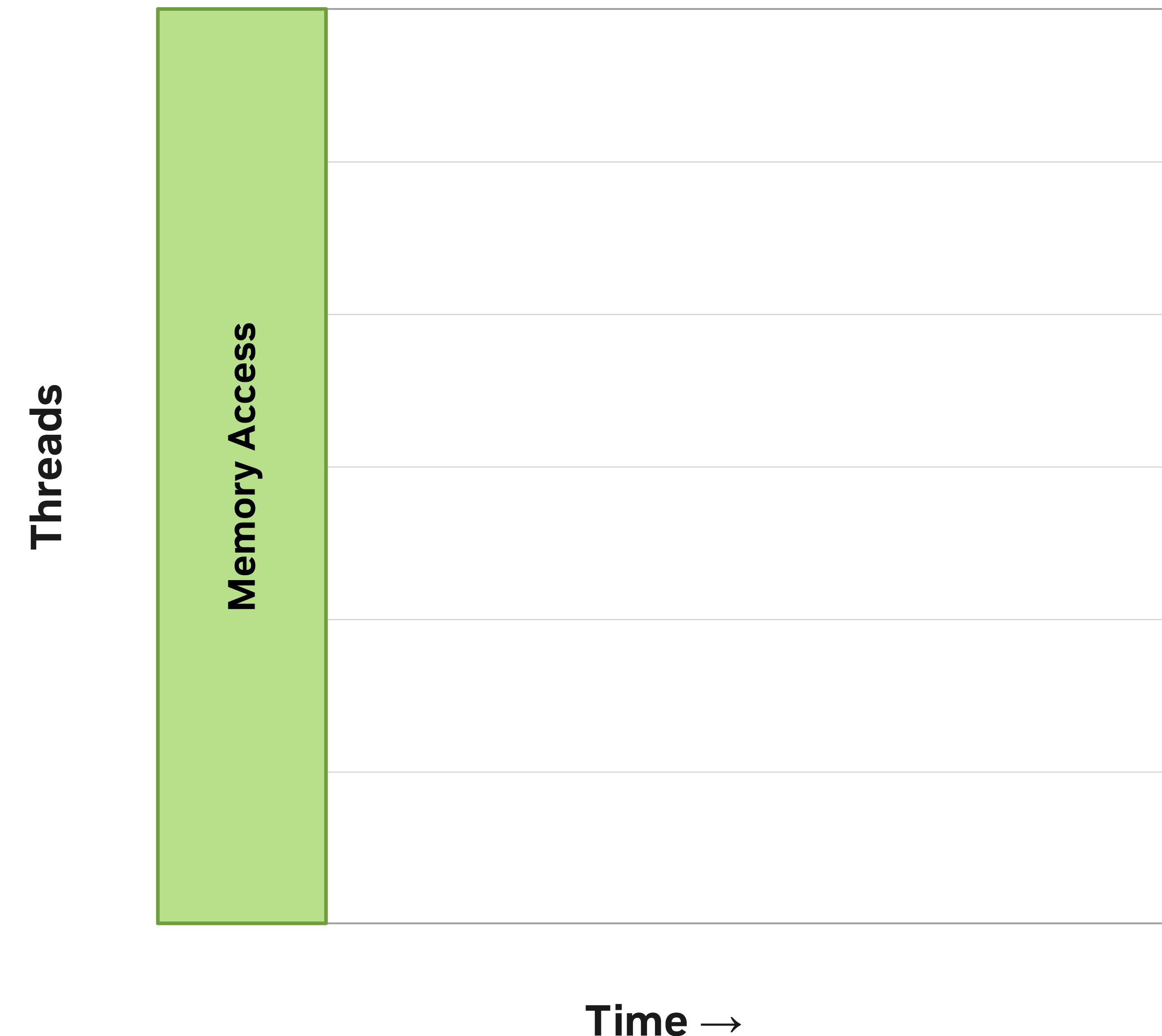
SIMT: Textbook Matrix Multiply

```
__global__ void matmul(__half const* aPtr, __half const* bPtr,
                      __half* cPtr, size_t M, size_t N, size_t K) {
    auto A = cuda::std::mdspan(aPtr, cuda::std::extents{M, K});
    auto B = cuda::std::mdspan(bPtr, cuda::std::extents{K, N});
    auto C = cuda::std::mdspan(cPtr, cuda::std::extents{M, N});
    int lX = threadIdx.y,      lY = threadIdx.x;
    int gX = 16 * blockIdx.x + lX, gY = 16 * blockIdx.y + lY;
    __half acc{0};

    __shared__ __half A_S[16][16], B_S[16][16];

    for (size_t tl = 0; tl != K; tl += 16) {
        A_S[lX][lY] = A(gX,      tl + lY);
        B_S[lX][lY] = B(tl + lX, gY);
        __syncthreads();

        for (size_t idx = 0; idx != 16; ++idx) {
            acc += A_S[lX][idx] * B_S[idx][lY];
        }
        __syncthreads();
    }
    C(gX, gY) = acc;
}
```



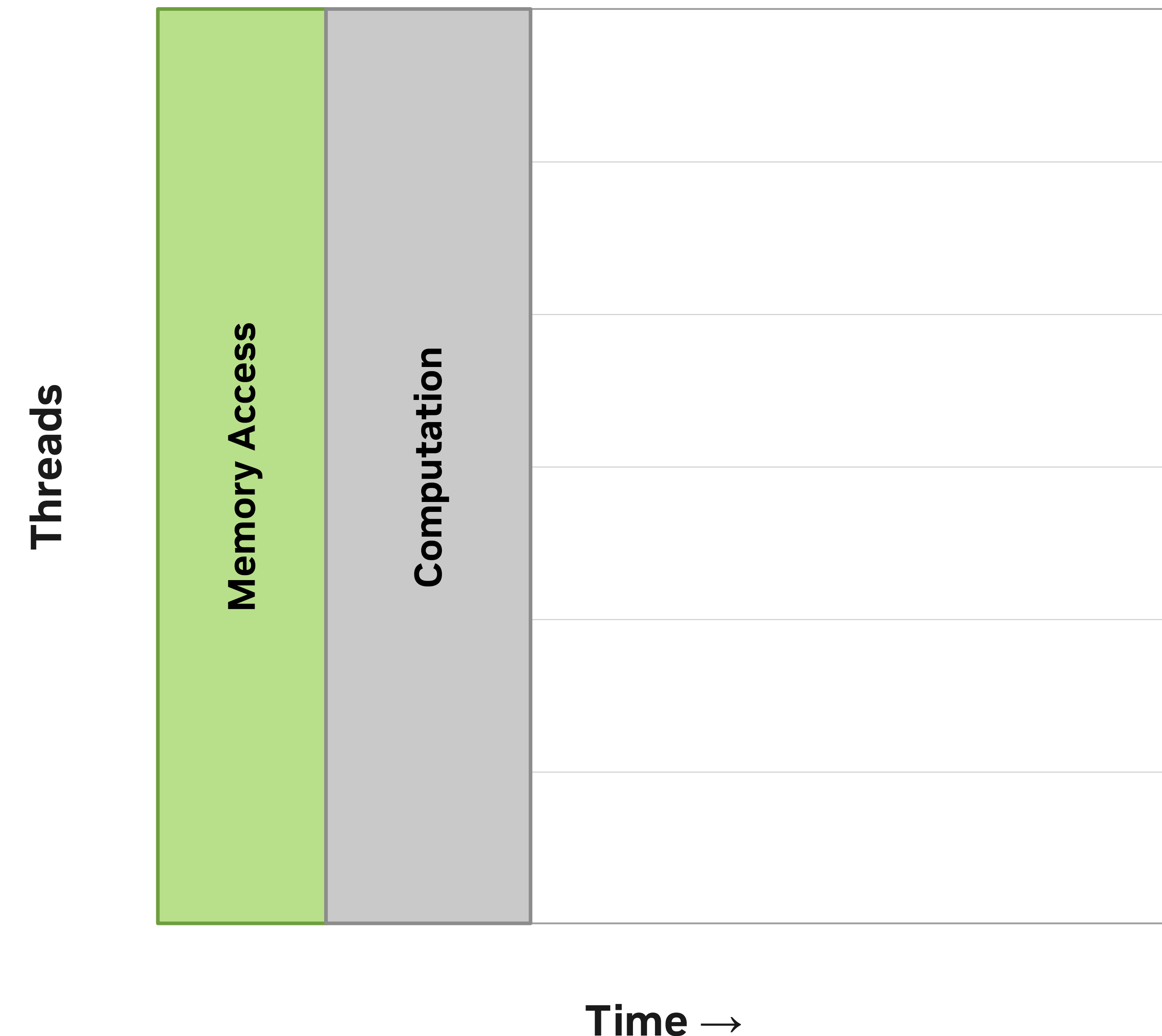
SIMT: Textbook Matrix Multiply

```
__global__ void matmul(__half const* aPtr, __half const* bPtr,
                      __half* cPtr, size_t M, size_t N, size_t K) {
    auto A = cuda::std::mdspan(aPtr, cuda::std::extents{M, K});
    auto B = cuda::std::mdspan(bPtr, cuda::std::extents{K, N});
    auto C = cuda::std::mdspan(cPtr, cuda::std::extents{M, N});
    int lX = threadIdx.y,      lY = threadIdx.x;
    int gX = 16 * blockIdx.x + lX, gY = 16 * blockIdx.y + lY;
    __half acc{0};

    __shared__ __half A_S[16][16], B_S[16][16];

    for (size_t tl = 0; tl != K; tl += 16) {
        A_S[lX][lY] = A(gX,      tl + lY);
        B_S[lX][lY] = B(tl + lX, gY);
        __syncthreads();

        for (size_t idx = 0; idx != 16; ++idx) {
            acc += A_S[lX][idx] * B_S[idx][lY];
        }
        __syncthreads();
    }
    C(gX, gY) = acc;
}
```



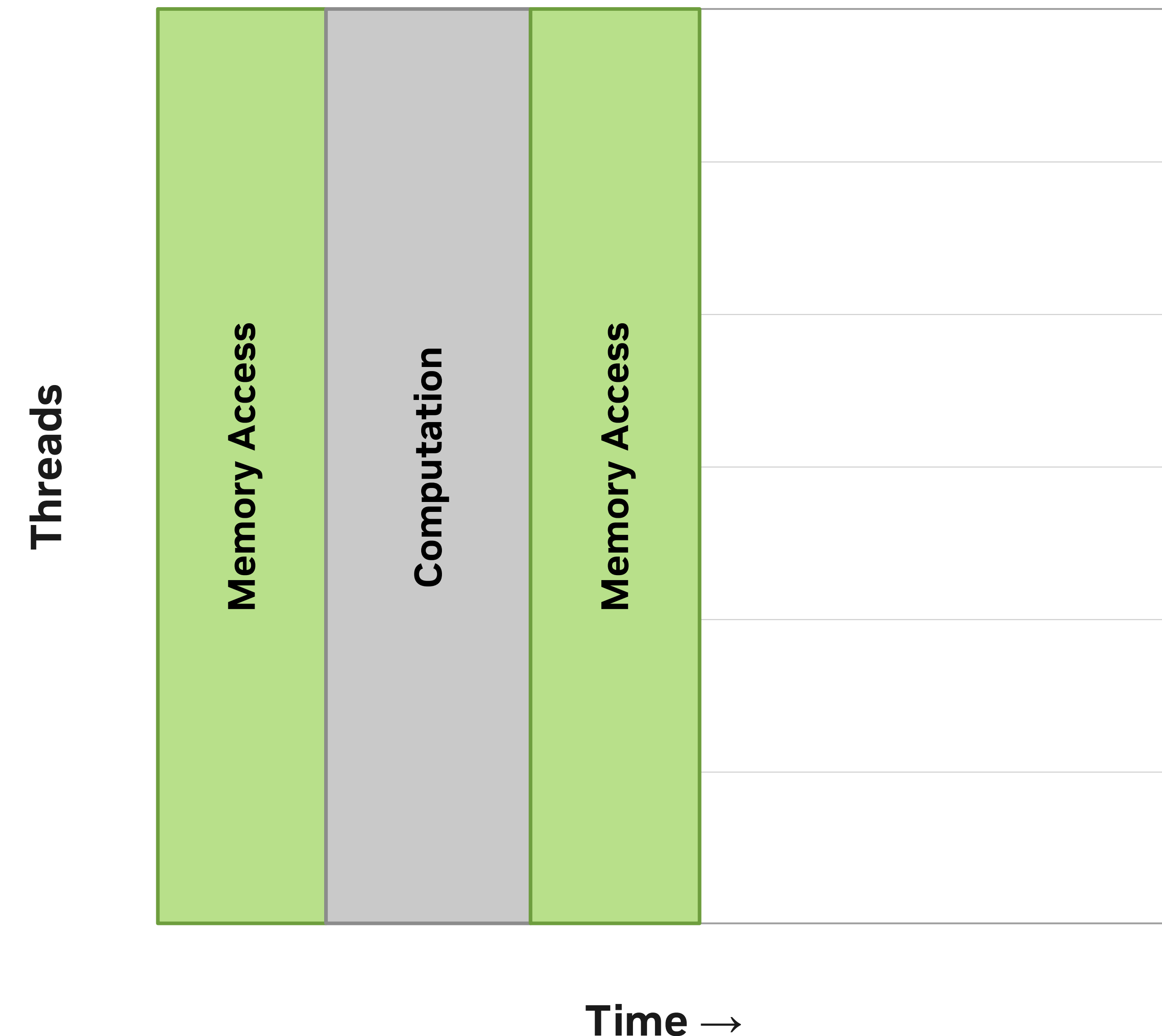
SIMT: Textbook Matrix Multiply

```
__global__ void matmul(__half const* aPtr, __half const* bPtr,
                      __half* cPtr, size_t M, size_t N, size_t K) {
    auto A = cuda::std::mdspan(aPtr, cuda::std::extents{M, K});
    auto B = cuda::std::mdspan(bPtr, cuda::std::extents{K, N});
    auto C = cuda::std::mdspan(cPtr, cuda::std::extents{M, N});
    int lX = threadIdx.y,      lY = threadIdx.x;
    int gX = 16 * blockIdx.x + lX, gY = 16 * blockIdx.y + lY;
    __half acc{0};

    __shared__ __half A_S[16][16], B_S[16][16];

    for (size_t tl = 0; tl != K; tl += 16) {
        A_S[lX][lY] = A(gX,      tl + lY);
        B_S[lX][lY] = B(tl + lX, gY);
        __syncthreads();

        for (size_t idx = 0; idx != 16; ++idx) {
            acc += A_S[lX][idx] * B_S[idx][lY];
        }
        __syncthreads();
    }
    C(gX, gY) = acc;
}
```



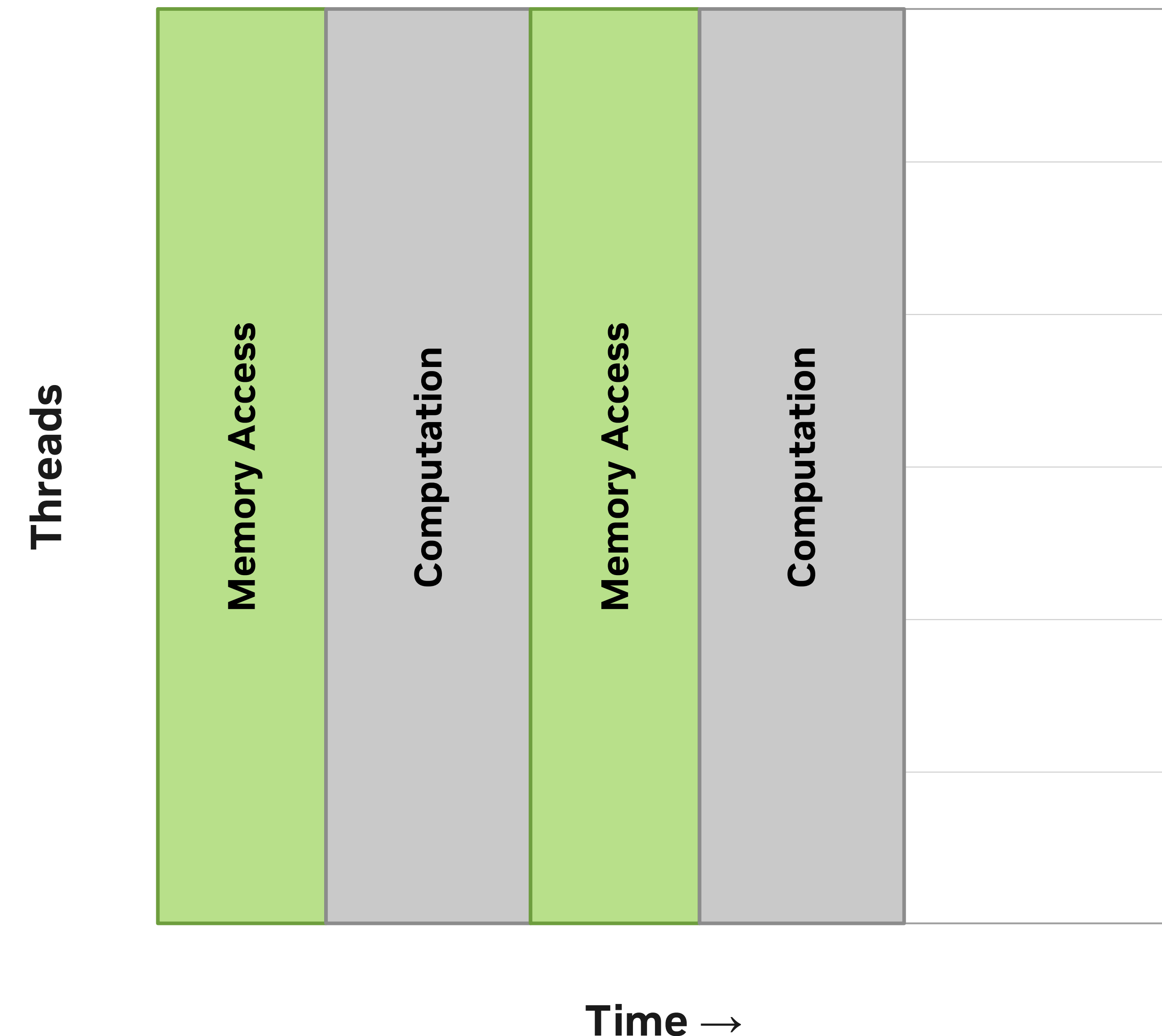
SIMT: Textbook Matrix Multiply

```
__global__ void matmul(__half const* aPtr, __half const* bPtr,
                      __half* cPtr, size_t M, size_t N, size_t K) {
    auto A = cuda::std::mdspan(aPtr, cuda::std::extents{M, K});
    auto B = cuda::std::mdspan(bPtr, cuda::std::extents{K, N});
    auto C = cuda::std::mdspan(cPtr, cuda::std::extents{M, N});
    int lX = threadIdx.y,      lY = threadIdx.x;
    int gX = 16 * blockIdx.x + lX, gY = 16 * blockIdx.y + lY;
    __half acc{0};

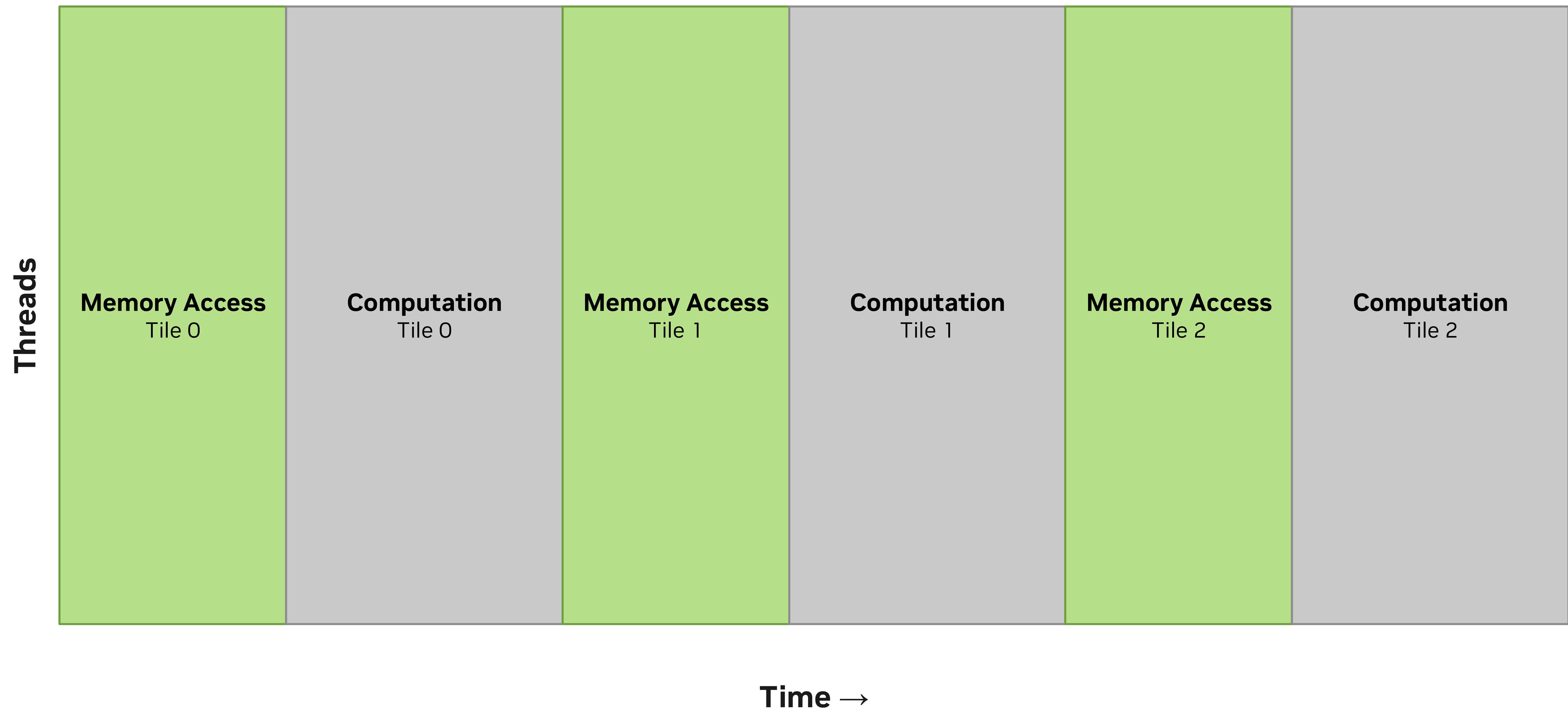
    __shared__ __half A_S[16][16], B_S[16][16];

    for (size_t tl = 0; tl != K; tl += 16) {
        A_S[lX][lY] = A(gX,      tl + lY);
        B_S[lX][lY] = B(tl + lX, gY);
        __syncthreads();

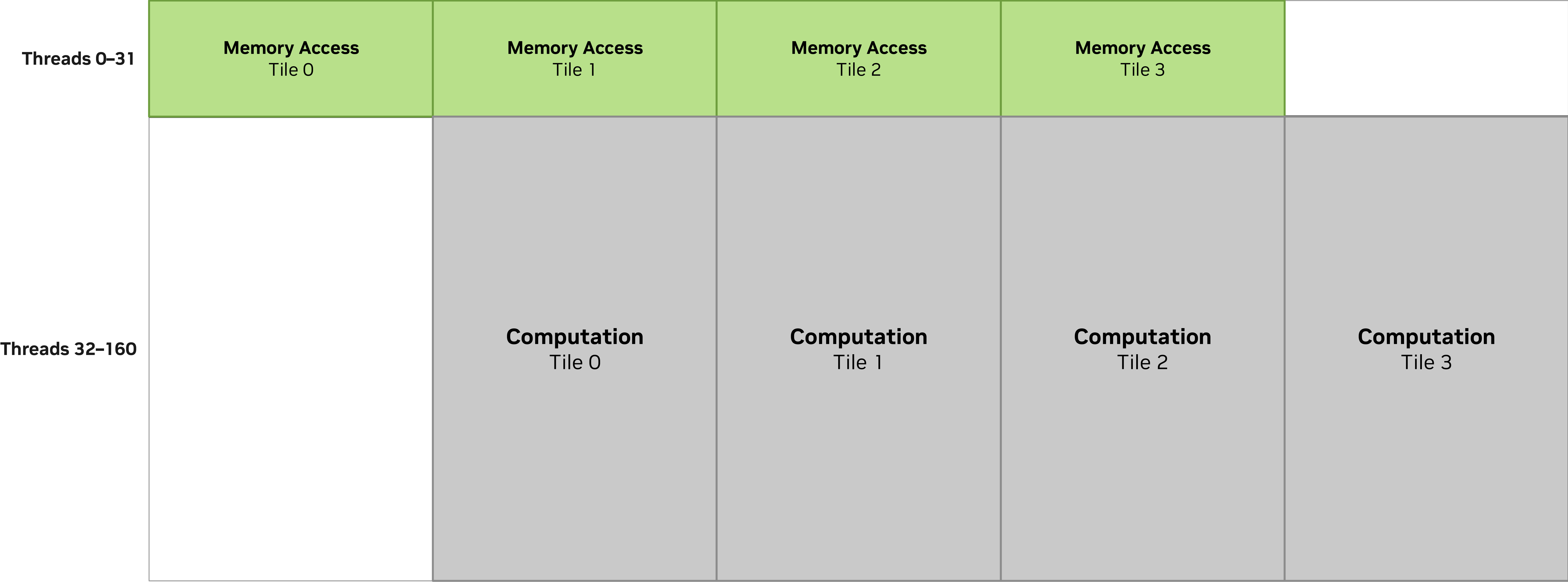
        for (size_t idx = 0; idx != 16; ++idx) {
            acc += A_S[lX][idx] * B_S[idx][lY];
        }
        __syncthreads();
    }
    C(gX, gY) = acc;
}
```



SIMT: Textbook Matrix Multiply

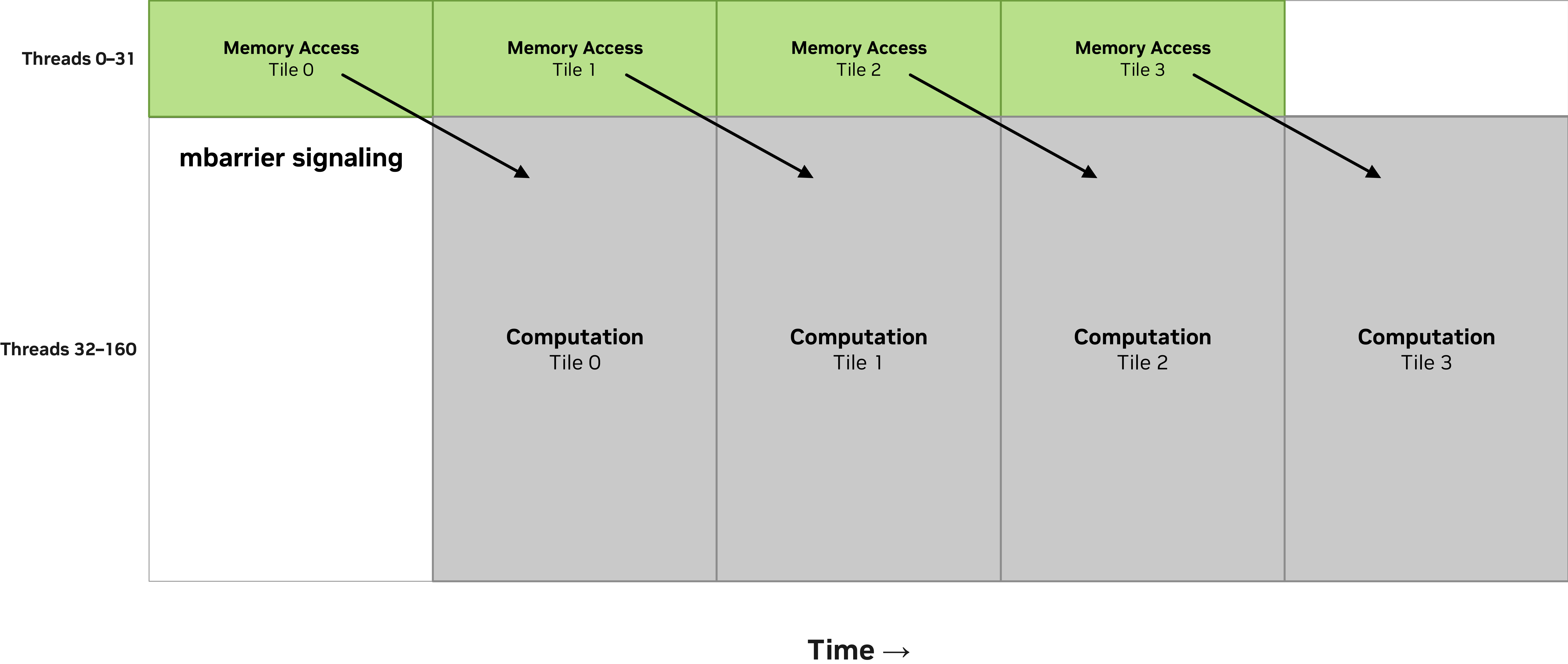


SIMT: Pipelining (Warp Specialization)



Time →

SIMT: Pipelining (Warp Specialization)



SIMT: Pipelining (Warp Specialization)

```
template<size_t TILE_SIZE>
__global__ void mma_simt_tensor_cores_ws(__half const* __restrict__ lhsPtr, __half const* __restrict__ rhsPtr,
    __grid_constant__ const CUtensorMap lhsTensorMap, __grid_constant__ const CUtensorMap rhsTensorMap,
    size_t m, size_t n, size_t k, __half* __restrict__ outPtr) {
    namespace ctd = cuda::std;
    auto lhs = ctd::mdspan(lhsPtr, ctd::extents{m, k});
    auto rhs = ctd::mdspan(rhsPtr, ctd::extents{k, n});
    auto out = ctd::mdspan(outPtr, ctd::extents{m, n});

    constexpr size_t QUEUE_SIZE = 2;
    alignas(128) __shared__ uint64_t fullBarrier[QUEUE_SIZE];
    alignas(128) __shared__ uint64_t emptyBarrier[QUEUE_SIZE];
    if (threadIdx.x == 0) {
        for (size_t idx = 0; idx != QUEUE_SIZE; ++idx) {
            cuda::ptx::mbarrier_init(&fullBarrier[idx], 5 * 32);
            cuda::ptx::mbarrier_init(&emptyBarrier[idx], 5 * 32);
        }
    }
    __syncthreads();

    alignas(128) __shared__ __half lhsShared[QUEUE_SIZE][TILE_SIZE][TILE_SIZE];
    alignas(128) __shared__ __half rhsShared[QUEUE_SIZE][TILE_SIZE][TILE_SIZE];
    size_t numTiles = k / TILE_SIZE;
    int32_t warpId = threadIdx.x / 32;

    if (warpId == 4) {
        size_t queueIdx = 0;
        for (size_t tileIdx = 0; tileIdx != numTiles; ++tileIdx) {
            uint64_t phase = cuda::ptx::mbarrier_arrive(&emptyBarrier[queueIdx]);
            while (!cuda::ptx::mbarrier_try_wait(&emptyBarrier[queueIdx], phase)) { /* spin */ };
            if (threadIdx.x % 32 == 0) {
                int32_t lhsCoord[2] = {(int32_t)TILE_SIZE * (int32_t)tileIdx, (int32_t)TILE_SIZE * (int32_t)blockIdx.x};
                cuda::ptx::cp_async_bulk_tensor(cuda::ptx::space_shared_t{}, cuda::ptx::space_global_t{},
                    &lhsShared[queueIdx][0][0], &lhsTensorMap, lhsCoord, &fullBarrier[queueIdx]);
                cuda::ptx::mbarrier_expect_tx(cuda::ptx::sem_relaxed_t{}, cuda::ptx::scope_cta_t{},
                    cuda::ptx::space_shared_t{}, &fullBarrier[queueIdx], TILE_SIZE * TILE_SIZE * sizeof(__half));
                int32_t rhsCoord[2] = {(int32_t)TILE_SIZE * (int32_t)blockIdx.y, (int32_t)TILE_SIZE * (int32_t)tileIdx};
                cuda::ptx::cp_async_bulk_tensor(cuda::ptx::space_shared_t{}, cuda::ptx::space_global_t{},
                    &rhsShared[queueIdx][0][0], &rhsTensorMap, rhsCoord, &fullBarrier[queueIdx]);
                cuda::ptx::mbarrier_expect_tx(cuda::ptx::sem_relaxed_t{}, cuda::ptx::scope_cta_t{},
                    cuda::ptx::space_shared_t{}, &fullBarrier[queueIdx], TILE_SIZE * TILE_SIZE * sizeof(__half));
            }
            cuda::ptx::mbarrier_arrive(&fullBarrier[queueIdx]);
            queueIdx = (queueIdx + 1) % QUEUE_SIZE;
        }
    } else {
        for (size_t idx = 0; idx != QUEUE_SIZE; ++idx) { cuda::ptx::mbarrier_arrive(&emptyBarrier[idx]); }

        uint32_t accFragment[4] = {0, 0, 0, 0};
        int32_t warpIdx = warpId / 2;
        int32_t warpIdY = warpId % 2;
        auto cornerX = blockIdx.x * TILE_SIZE + warpIdx * TILE_SIZE / 2;
        auto cornerY = blockIdx.y * TILE_SIZE + warpIdY * TILE_SIZE / 2;
        size_t queueIdx = 0;

        for (size_t tileIdx = 0; tileIdx != numTiles; ++tileIdx) {
            uint64_t phase = cuda::ptx::mbarrier_arrive(&fullBarrier[queueIdx]);
            while (!cuda::ptx::mbarrier_try_wait(&fullBarrier[queueIdx], phase)) { /* spin */ };
            uint32_t lhsFragment[8];
```

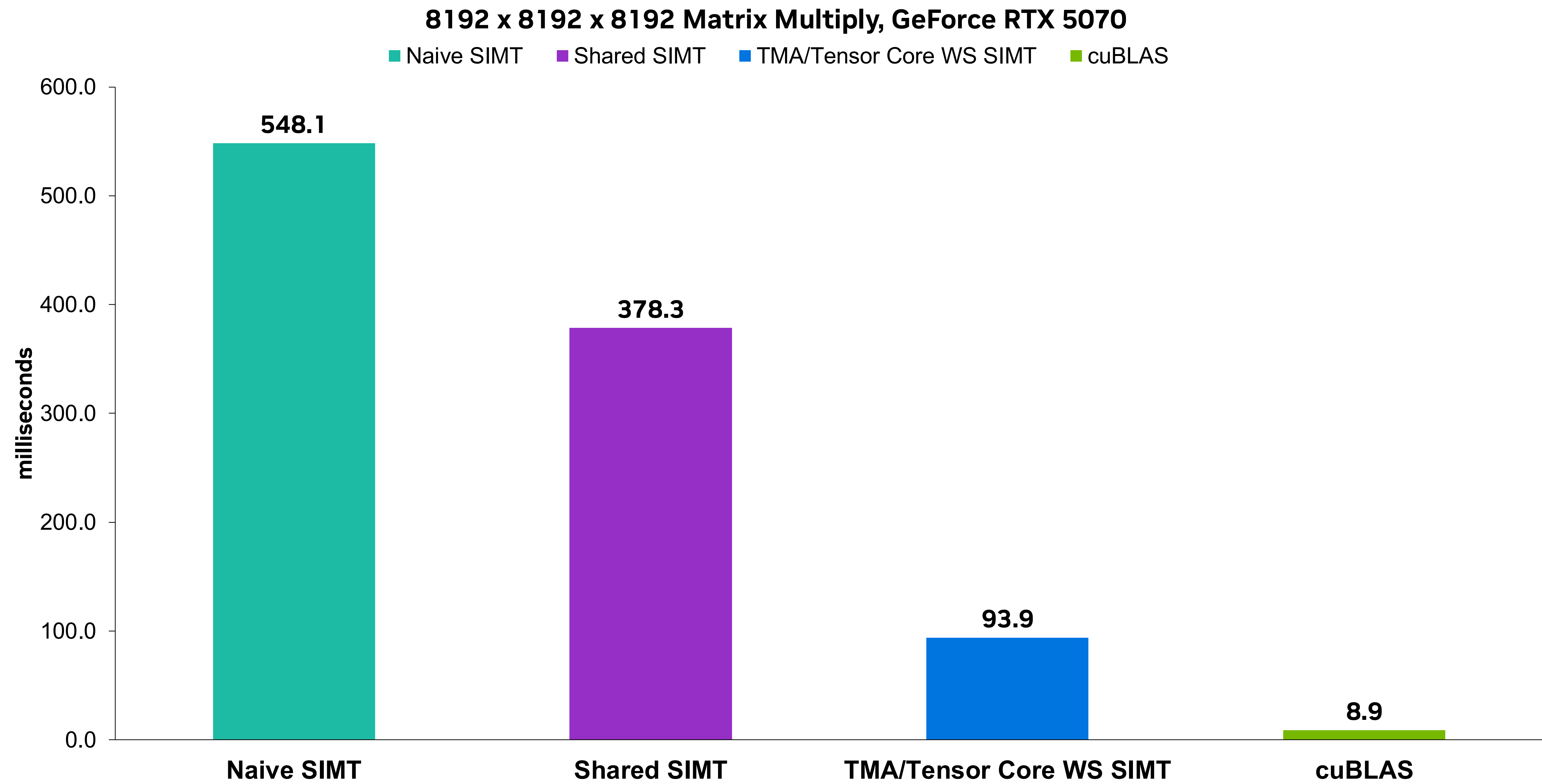
```
            uint32_t rhsFragment[8];
            for (size_t subTileIdx = 0; subTileIdx != 2; ++subTileIdx) {
                __half* lhsCorner = &lhsShared[queueIdx][0][0]
                    + (TILE_SIZE * TILE_SIZE / 2) * warpIdx + subTileIdx * TILE_SIZE / 2;
                __half* rhsCorner = &rhsShared[queueIdx][0][0]
                    + (TILE_SIZE * TILE_SIZE / 2) * subTileIdx + warpIdY * TILE_SIZE / 2;
                asm volatile (
                    "mma.load.a.sync.aligned.row.m16n16k16.shared::cta.f16 {%0, %1, %2, %3, %4, %5, %6, %7}, [%20], %22;"
                    "mma.load.b.sync.aligned.row.m16n16k16.shared::cta.f16 {%8, %9, %10, %11, %12, %13, %14, %15}, [%21], %22;"
                    "mma.mma.sync.aligned.row.row.m16n16k16.f16.f16 {%16, %17, %18, %19}, "
                    "{%0, %1, %2, %3, %4, %5, %6, %7}, {%8, %9, %10, %11, %12, %13, %14, %15}, {%16, %17, %18, %19};"
                    : "=r"(lhsFragment[0]), "=r"(lhsFragment[1]), "=r"(lhsFragment[2]), "=r"(lhsFragment[3]),
                      "=r"(lhsFragment[4]), "=r"(lhsFragment[5]), "=r"(lhsFragment[6]), "=r"(lhsFragment[7]),
                      "=r"(rhsFragment[0]), "=r"(rhsFragment[1]), "=r"(rhsFragment[2]), "=r"(rhsFragment[3]),
                      "=r"(rhsFragment[4]), "=r"(rhsFragment[5]), "=r"(rhsFragment[6]), "=r"(rhsFragment[7]),
                      "+r"(accFragment[0]), "+r"(accFragment[1]), "+r"(accFragment[2]), "+r"(accFragment[3])
                    : "1"(lhsCorner), "1"(rhsCorner), "r"(int32_t(TILE_SIZE)) : "memory");
            }
            cuda::ptx::mbarrier_arrive(&emptyBarrier[queueIdx]);
            queueIdx = (queueIdx + 1) % QUEUE_SIZE;
        }

        __half* gOut = &out(cornerX, cornerY);
        asm volatile (
            "mma.store.d.sync.aligned.row.m16n16k16.global.f16 [%4], {%0, %1, %2, %3}, %5;"
            :: "r"(accFragment[0]), "r"(accFragment[1]), "r"(accFragment[2]), "r"(accFragment[3]),
              "1"(gOut), "r"(int32_t(n)) : "memory");
    }
}

void launch_mma_simt_tensor_cores_ws(cudaStream_t stream, __half const* lhs, __half const* rhs,
    size_t m, size_t n, size_t k, __half* out) {
    constexpr size_t TILE_SIZE = 2 * 16;
    CUtensorMap lhsTensorMap, rhsTensorMap;
    {
        uint32_t lhsRank = 2;
        uint64_t lhsSize[2] = {k, m};
        uint64_t lhsStride[1] = {k * sizeof(__half)};
        uint32_t lhsTileSize[2] = {TILE_SIZE, TILE_SIZE};
        uint32_t lhsElemStride[2] = {1, 1};
        checkDriverError(cuTensorMapEncodeTiled(&lhsTensorMap, CU_TENSOR_MAP_DATA_TYPE_FLOAT16, lhsRank, (void*)lhs,
            &lhsSize[0], &lhsStride[0], &lhsTileSize[0], &lhsElemStride[0], CU_TENSOR_MAP_INTERLEAVE_NONE,
            CU_TENSOR_MAP_SWIZZLE_NONE, CU_TENSOR_MAP_L2_PROMOTION_NONE, CU_TENSOR_MAP_FLOAT_OOB_FILL_NONE));
    }
    {
        uint32_t rhsRank = 2;
        uint64_t rhsSize[2] = {n, k};
        uint64_t rhsStride[1] = {n * sizeof(__half)};
        uint32_t rhsTileSize[2] = {TILE_SIZE, TILE_SIZE};
        uint32_t rhsElemStride[2] = {1, 1};
        checkDriverError(cuTensorMapEncodeTiled(&rhsTensorMap, CU_TENSOR_MAP_DATA_TYPE_FLOAT16, rhsRank, (void*)rhs,
            &rhsSize[0], &rhsStride[0], &rhsTileSize[0], &rhsElemStride[0], CU_TENSOR_MAP_INTERLEAVE_NONE,
            CU_TENSOR_MAP_SWIZZLE_NONE, CU_TENSOR_MAP_L2_PROMOTION_NONE, CU_TENSOR_MAP_FLOAT_OOB_FILL_NONE));
    }

    dim3 numBlocks(1 + (m - 1) / TILE_SIZE, 1 + (n - 1) / TILE_SIZE);
    mma_simt_tensor_cores_ws<TILE_SIZE><<<numBlocks, 32 * 5, 0, stream>>>(&lhs, &rhs, &lhsTensorMap, &rhsTensorMap,
        m, n, k, out);
}
```

SIMT: Handwritten TMA/Tensor Core Performance



Tile Programming: From SIMT to SIMD

Algorithm design using Tile C++

- Partition the data set into **blocks** which can be processed independently
- Within a **block**, use **SIMD** operations on **arrays** to process data in bulk

One Logical Thread Per Block

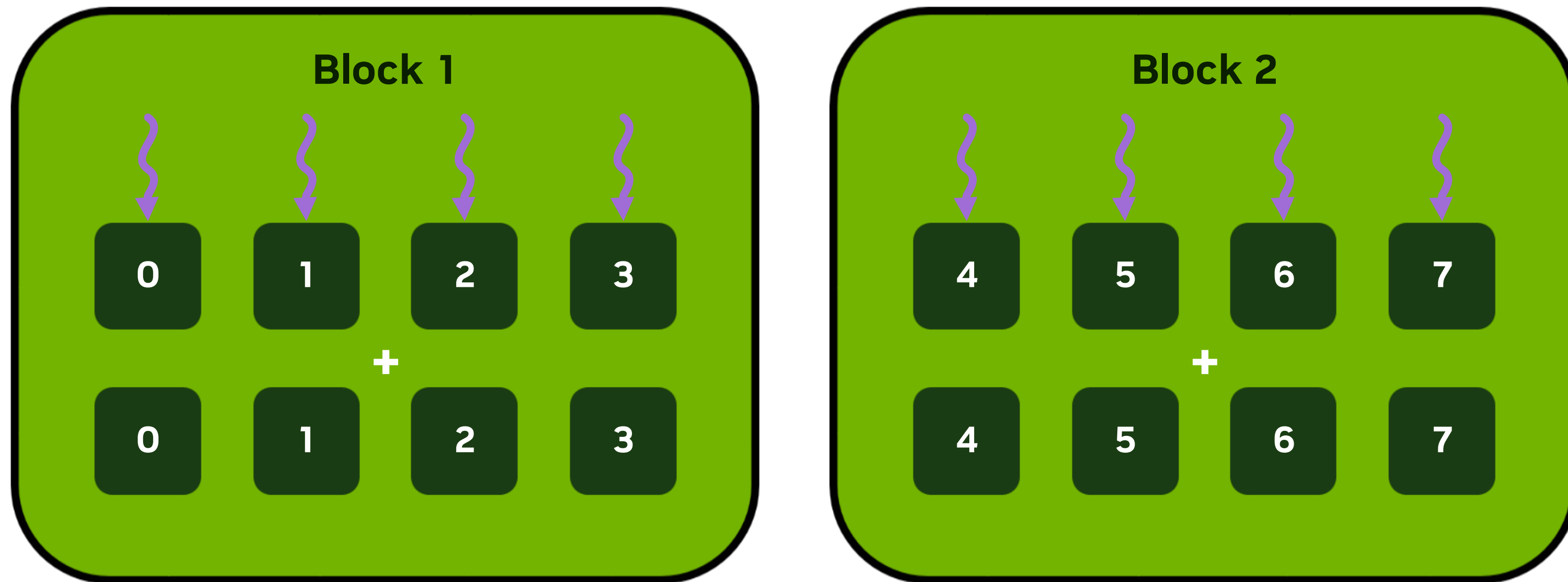
Compiler Handles:

Thread to data assignment
Data movement/caching
Pipelining & warp specialization

Tile Programming

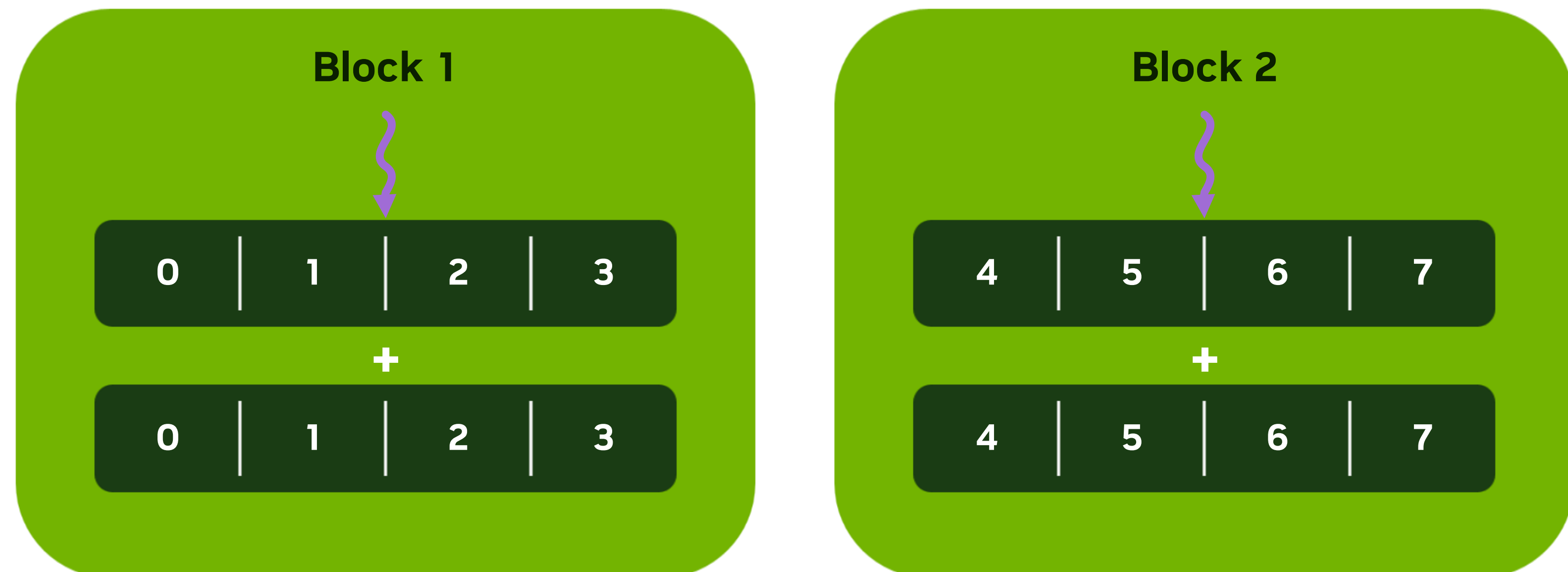
From SIMT to SIMD

SIMT



```
// Blocks = 2      Threads Per Block = 4
__device__ float simtAdd(float a, float b) {
    return a + b;
}
```

Tile



```
// Blocks = 2      Threads Per Block = 1
using tile4 = ct::tile<float, ct::shape<4>>;

__tile__ tile4 tileAdd(tile4 a, tile4 b) {
    return a + b;
}
```

The `cuda::tiles::tile` data type

Similar to a SIMD Register

```
namespace ct = cuda::tiles;

using tile16 =
    ct::tile<float, ct::shape<16>>>;

__tile__ tile16 hypot(tile16 a, tile16 b)
{
    return ct::sqrt(a * a + b * b);
}
```

- Compile time N-D shape
- Value semantics
- Elementwise operations
- Thread to value association determined by the compiler

Example Thread Schedule

Tile

```
using tile16 =  
    ct::tile<float,ct::shape<16>>;  
  
{  
    tile16 a = ...;  
    tile16 b = ...;  
    tile16 c = ct::sqrt(a * a + b * b);  
}
```

SIMT

```
{  
    float a = ...;  
    float b = ...;  
    float c = sqrt(a * a + b * b);  
}
```

Registers, 1 element per thread

Example Thread Schedule

Tile

```
using tile16 =
    ct::tile<float,ct::shape<16>>;

{
    tile16 a = ...;
    tile16 b = ...;
    tile16 c = ct::sqrt(a * a + b * b);
}
```

SIMT

```
{
    float a0 = ...;
    float a1 = ...;
    float b0 = ...;
    float b1 = ...;
    float c0 = sqrt(a0 * a0 + b0 * b0);
    float c1 = sqrt(a1 * a1 + b1 * b1);
}
```

Registers, 2 elements per thread

Example Thread Schedule

Tile

```
using tile16 =
    ct::tile<float, ct::shape<16>>;

{
    tile16 a = ...;
    tile16 b = ...;
    tile16 c = ct::sqrt(a * a + b * b);
}
```

SIMT

```
{
    __shared__ float a[16], b[16], c[16];

    int x = threadIdx.x;

    c[x] = sqrt(a[x] * a[x] + b[x] * b[x]);
}
```

Shared Memory, 1 element per thread

Example Thread Schedule

Tile

```
using tile16 =
    ct::tile<float,ct::shape<16>>;

{
    tile16 a = ...;
    tile16 b = ...;
    tile16 c = ct::sqrt(a * a + b * b);
}
```

SIMT

```
{
    __shared__ float a[16], b[16], c[16];

    int x0 = 2 * threadIdx.x;
    int x1 = 2 * threadIdx.x + 1;

    c[x0] = sqrt(a[x0] * a[x0] + b[x0] * b[x0]);
    c[x1] = sqrt(a[x1] * a[x1] + b[x1] * b[x1]);
}
```

Shared Memory, 2 elements per thread

Tile C++: Matrix Multiply

```
__tile_global__ void mma_tile_kernel(
    const __half* __restrict__ aPtr,
    const __half* __restrict__ bPtr,
    int M, int N, int K,
    __half* __restrict__ cPtr) {

    namespace ct = cuda::tiles;
    using namespace ct::literals;

    aPtr = ct::assume_aligned(aPtr, 16_ic);
    bPtr = ct::assume_aligned(bPtr, 16_ic);
    cPtr = ct::assume_aligned(cPtr, 16_ic);

    M = ct::assume_divisible(M, 16_ic);
    N = ct::assume_divisible(N, 16_ic);
    K = ct::assume_divisible(K, 16_ic);

    using MxN = ct::shape<TILE_M, TILE_N>;
    using MxK = ct::shape<TILE_M, TILE_K>;
    using KxN = ct::shape<TILE_K, TILE_N>;
```

```
    auto A = ct::tensor_span{aPtr, ct::extents{M, K}};
    auto B = ct::tensor_span{bPtr, ct::extents{K, N}};
    auto C = ct::tensor_span{cPtr, ct::extents{M, N}};

    auto aView = ct::partition_view{A, MxK{}};
    auto bView = ct::partition_view{B, KxN{}};
    auto cView = ct::partition_view{C, MxN{}};

    auto [bidx, bidy, dummy] = ct::bid();

    auto acc = ct::zeros<ct::tile<__half, MxN>>();
    int numTiles = 1 + (K - 1) / TILE_K;

    for (auto idx : ct::irange(0, numTiles)) {
        auto aTile = aView.load_masked(bidx, idx);
        auto bTile = bView.load_masked(idx, bidy);
        acc = ct::mma(aTile, bTile, acc);
    }

    cView.store_masked(acc, bidx, bidy);
}
```

Tile C++: Matrix Multiply

```
auto A = ct::tensor_span{aPtr, ct::extents{M, K}};
auto B = ct::tensor_span{bPtr, ct::extents{K, N}};
auto C = ct::tensor_span{cPtr, ct::extents{M, N}};

auto aView = ct::partition_view{A, MxK{}};
auto bView = ct::partition_view{B, KxN{}};
auto cView = ct::partition_view{C, MxN{}};

auto [bidx, bidy, dummy] = ct::bid();

auto acc = ct::zeros<ct::tile<__half, MxN>>();
int numTiles = 1 + (K - 1) / TILE_K;

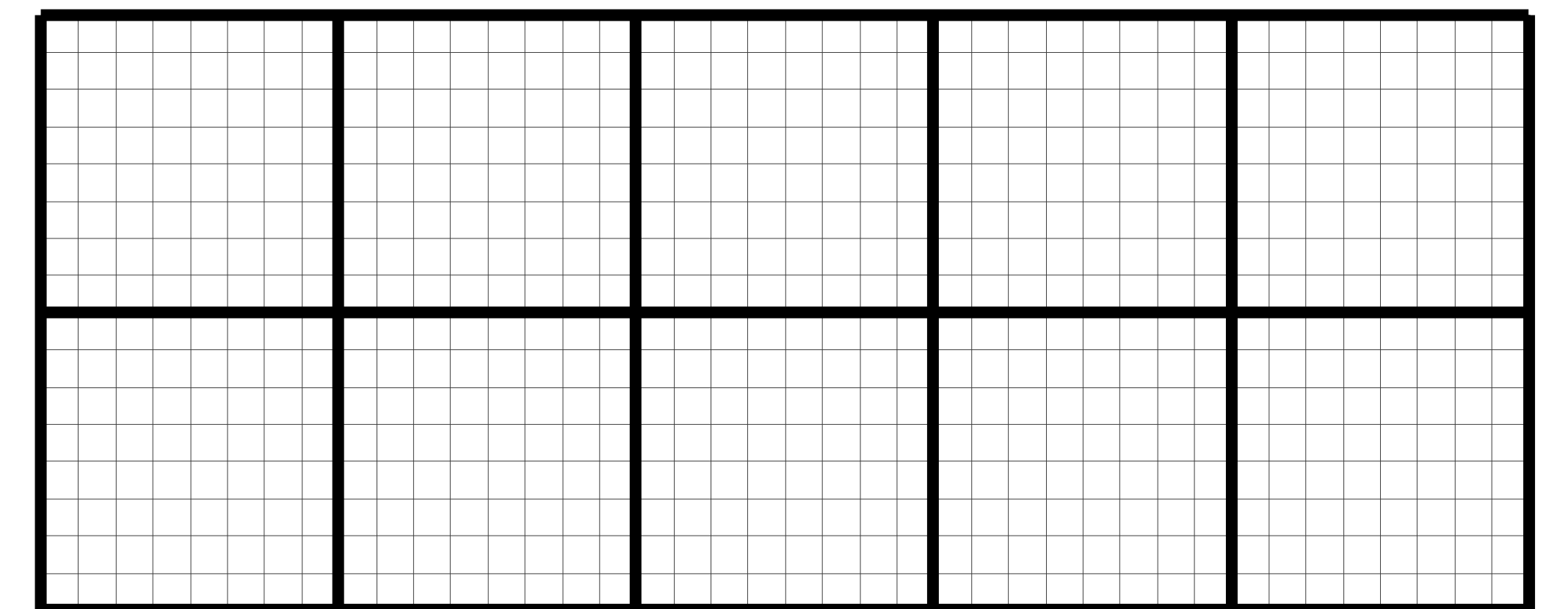
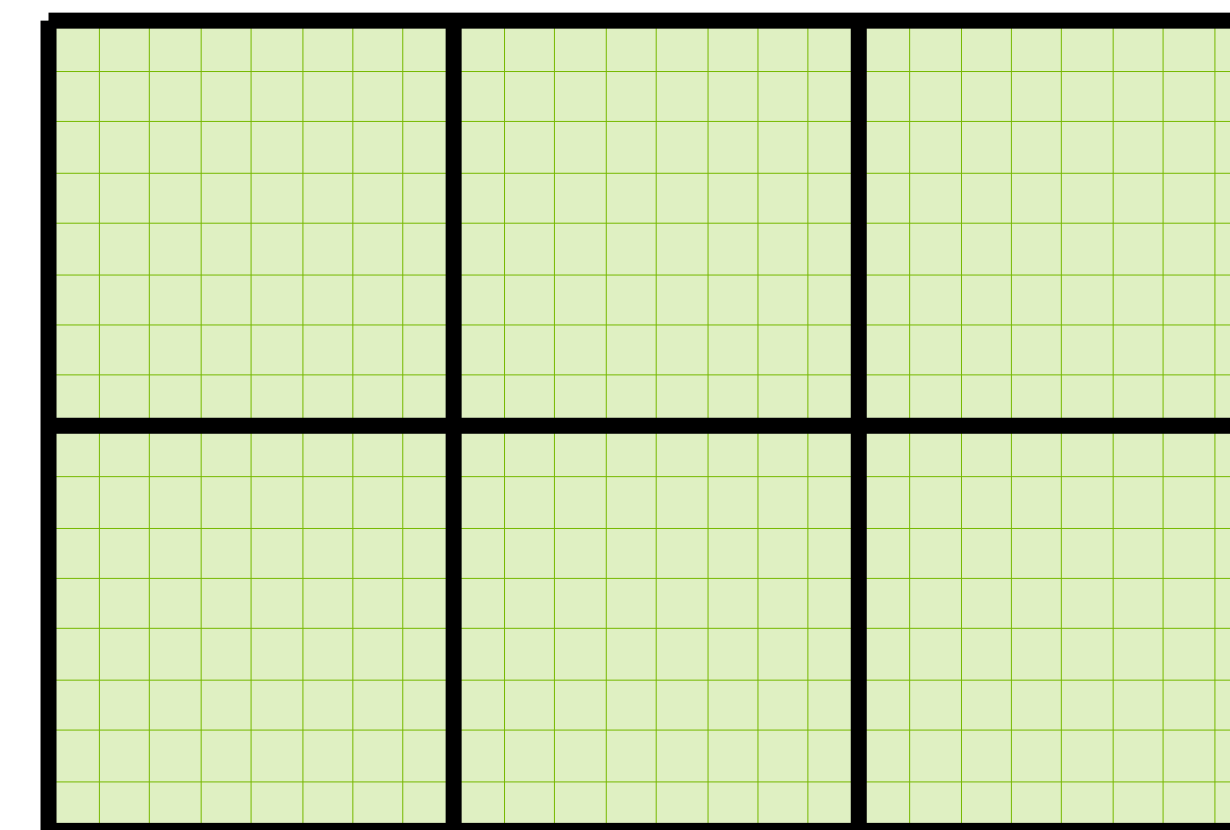
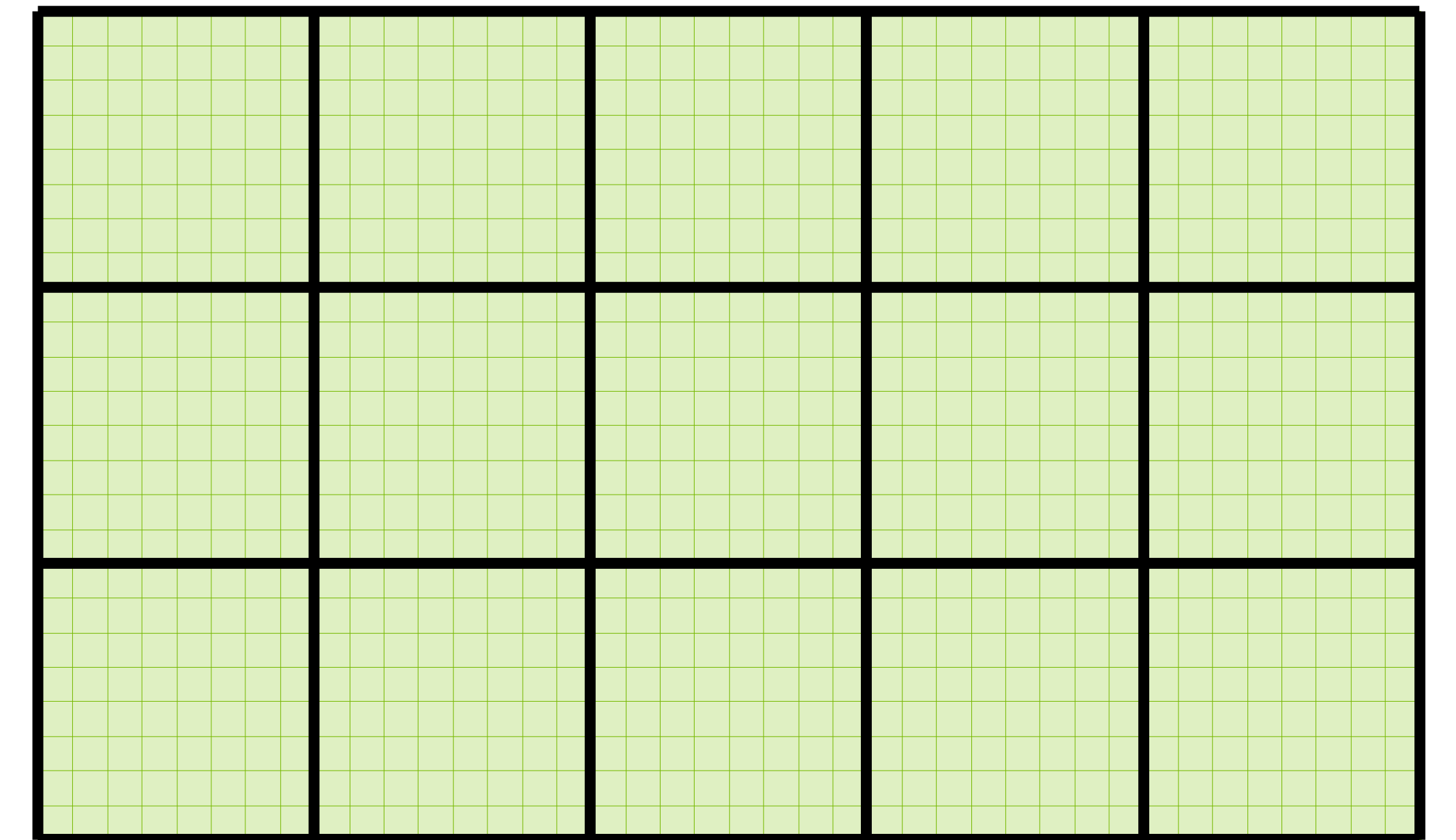
for (auto idx : ct::irange(0, numTiles)) {
    auto aTile = aView.load_masked(bidx, idx);
    auto bTile = bView.load_masked(idx, bidy);
    acc = ct::mma(aTile, bTile, acc);
}

cView.store_masked(acc, bidx, bidy);
}
```

Tile C++: Matrix Multiply

```
auto A = ct::tensor_span{aPtr, ct::extents{M, K}};  
auto B = ct::tensor_span{bPtr, ct::extents{K, N}};  
auto C = ct::tensor_span{cPtr, ct::extents{M, N}};  
  
auto aView = ct::partition_view{A, MxK{}};  
auto bView = ct::partition_view{B, KxN{}};  
auto cView = ct::partition_view{C, MxN{}};
```

```
auto [bidx, bidy, dummy] = ct::bid();  
  
auto acc = ct::zeros<ct::tile<__half, MxN>>();  
int numTiles = 1 + (K - 1) / TILE_K;  
  
for (auto idx : ct::irange(0, numTiles)) {  
    auto aTile = aView.load_masked(bidx, idx);  
    auto bTile = bView.load_masked(idx, bidy);  
    acc = ct::mma(aTile, bTile, acc);  
}  
  
cView.store_masked(acc, bidx, bidy);  
}
```



Tile C++: Matrix Multiply

```
auto A = ct::tensor_span{aPtr, ct::extents{M, K}};  
auto B = ct::tensor_span{bPtr, ct::extents{K, N}};  
auto C = ct::tensor_span{cPtr, ct::extents{M, N}};
```

```
auto aView = ct::partition_view{A, MxK{}};  
auto bView = ct::partition_view{B, KxN{}};  
auto cView = ct::partition_view{C, MxN{}};
```

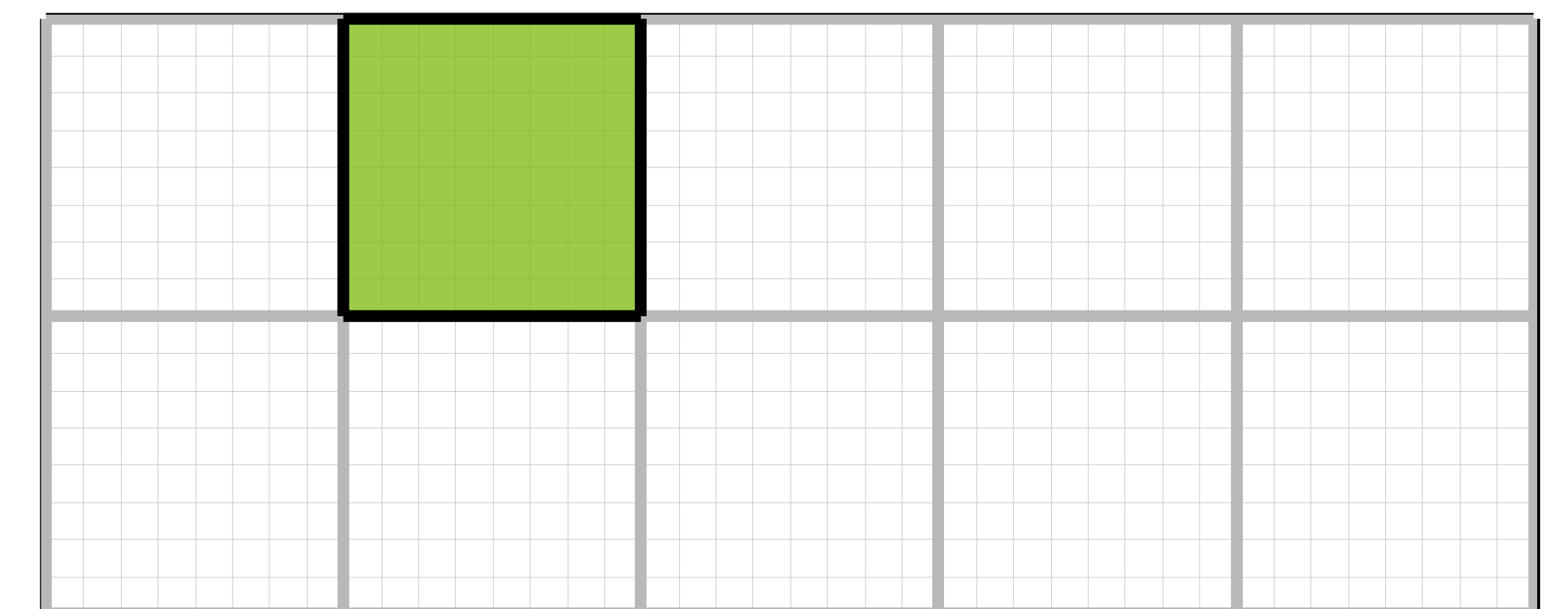
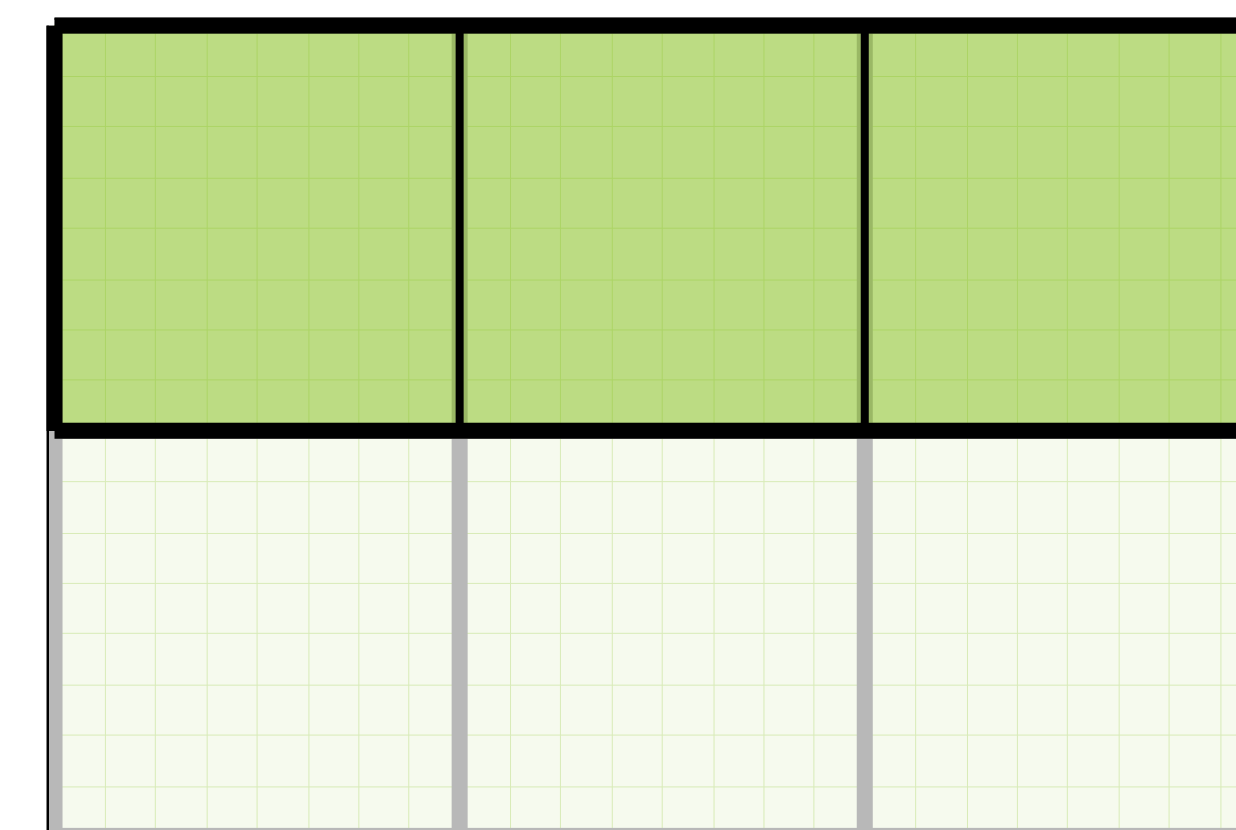
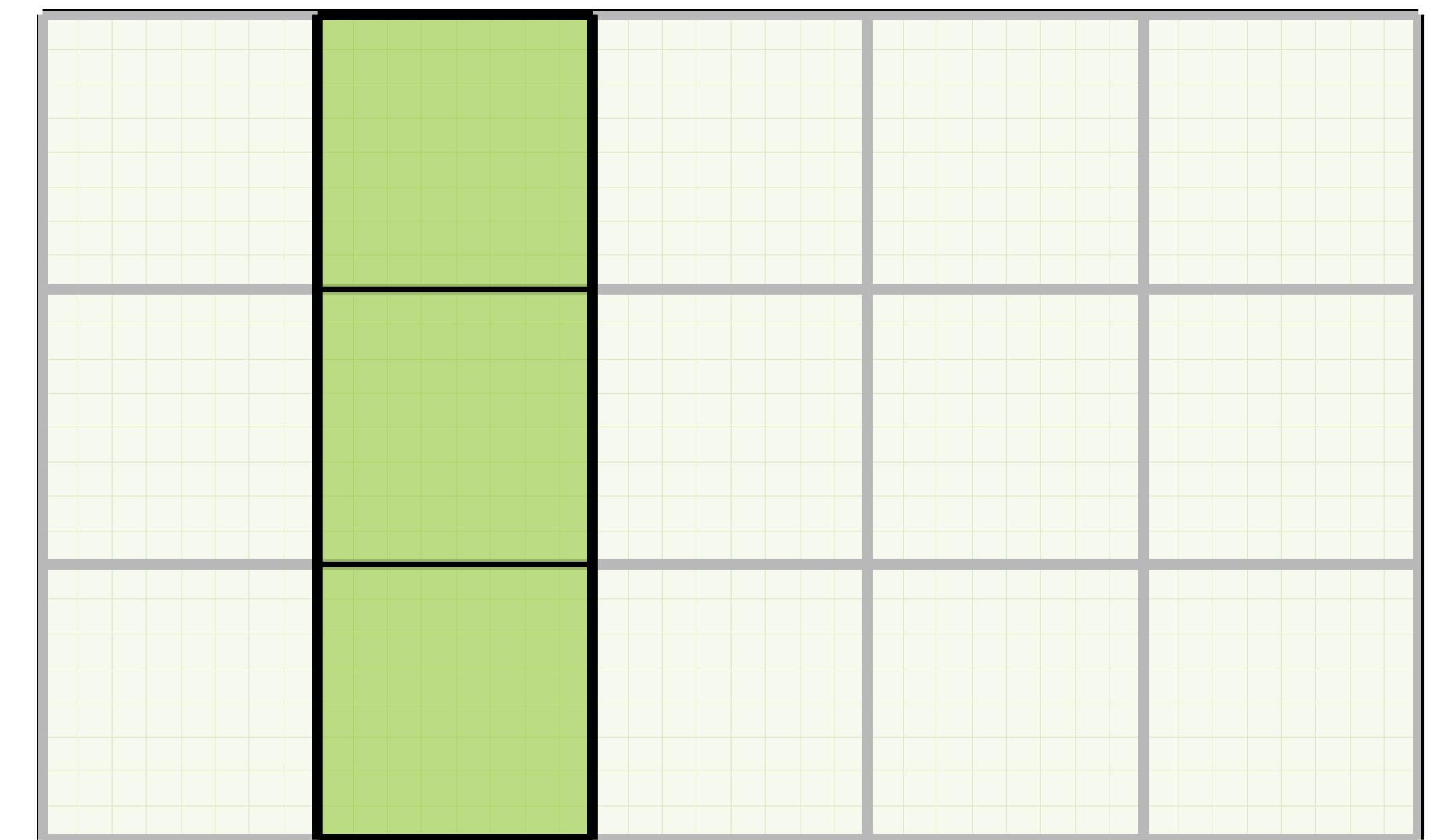
```
auto [bidx, bidy, dummy] = ct::bid();
```

```
auto acc = ct::zeros<ct::tile<__half, MxN>>();  
int numTiles = 1 + (K - 1) / TILE_K;
```

```
for (auto idx : ct::irange(0, numTiles)) {  
    auto aTile = aView.load_masked(bidx, idx);  
    auto bTile = bView.load_masked(idx, bidy);  
    acc = ct::mma(aTile, bTile, acc);  
}
```

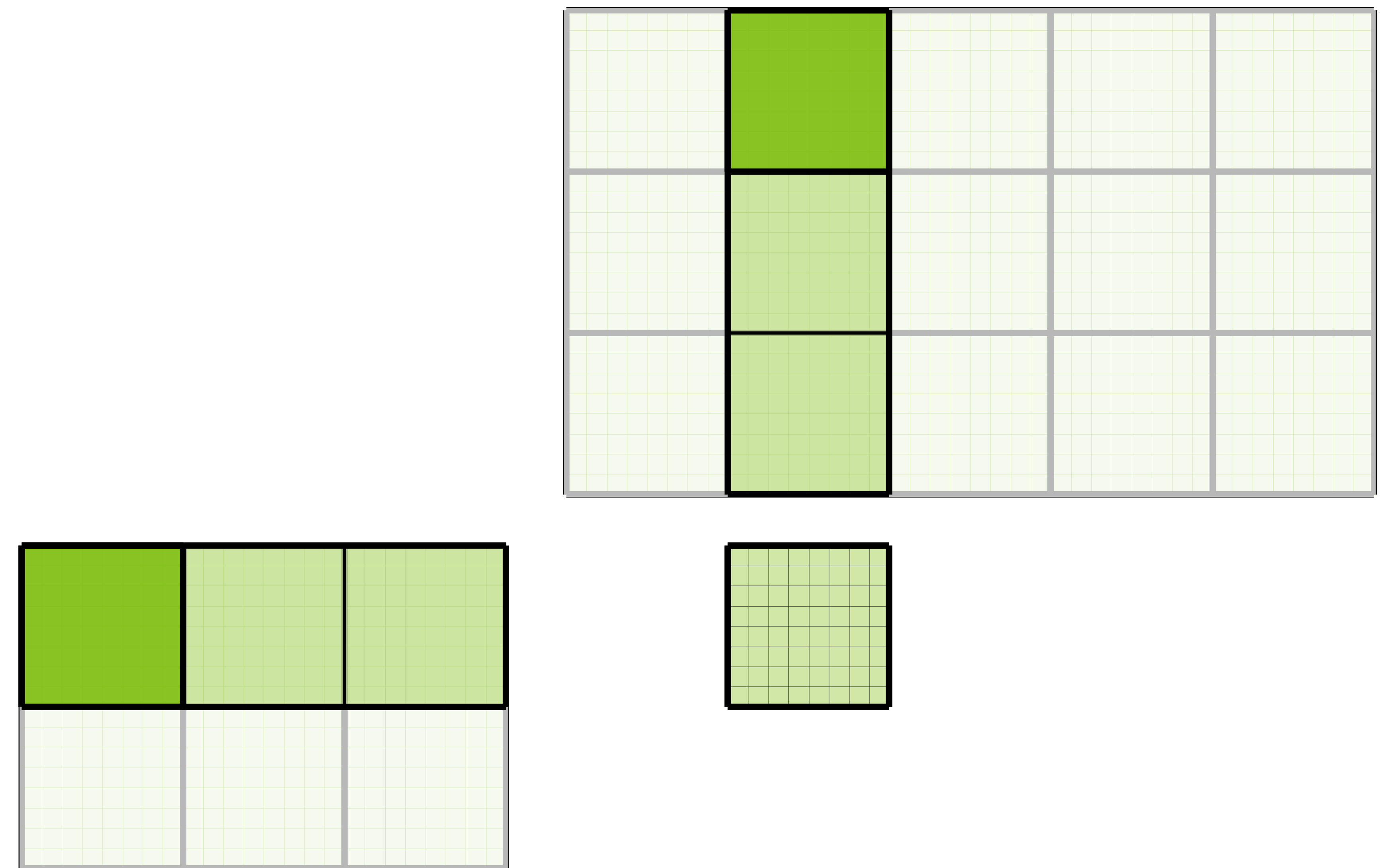
```
cView.store_masked(acc, bidx, bidy);
```

```
}
```



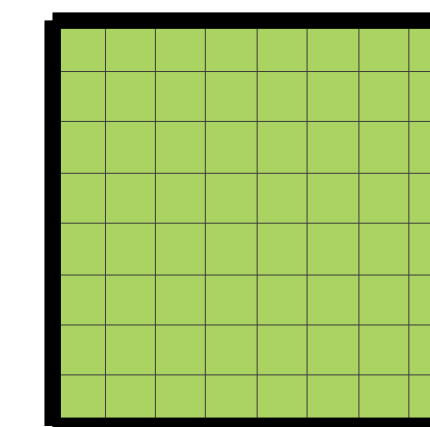
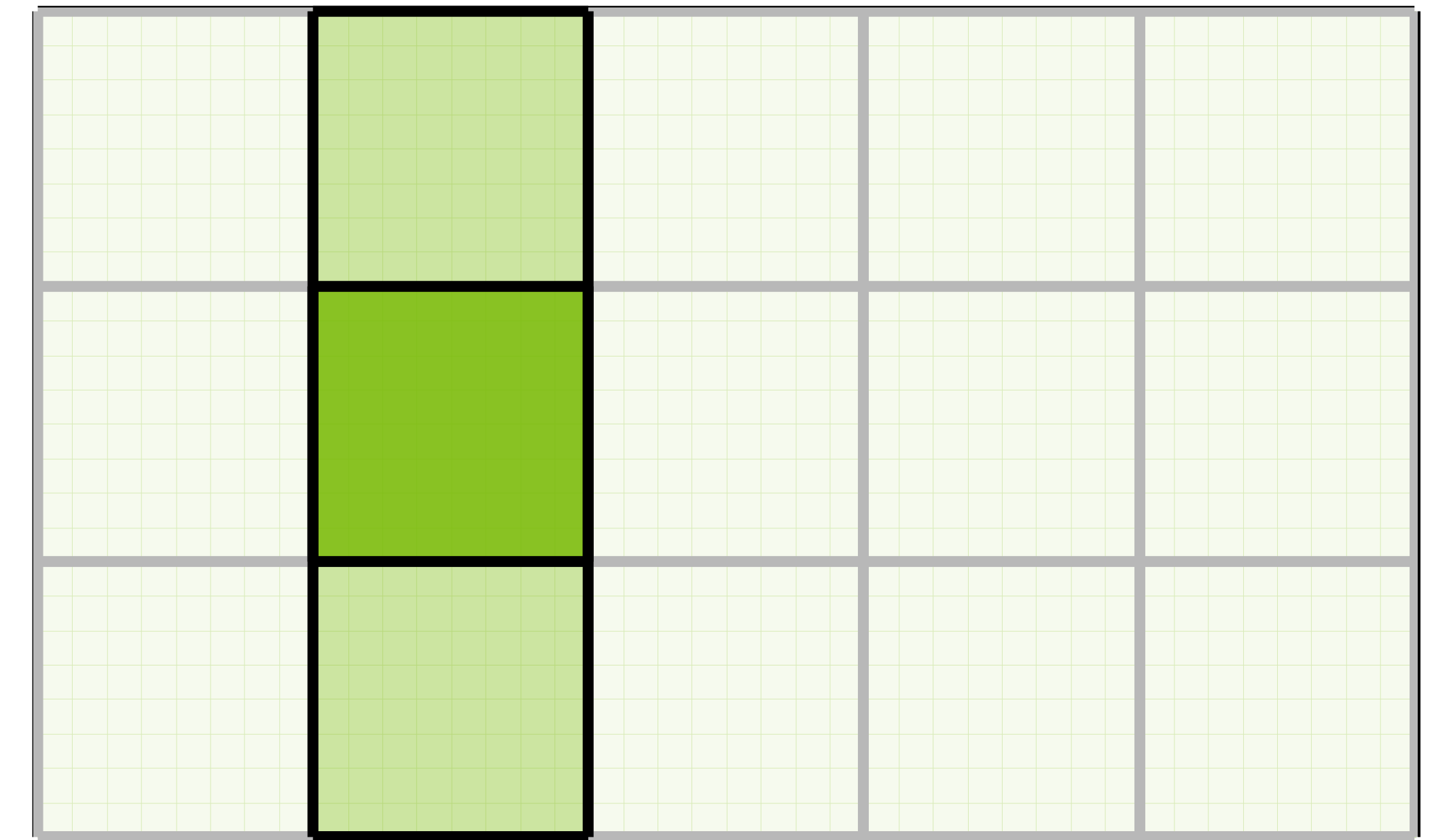
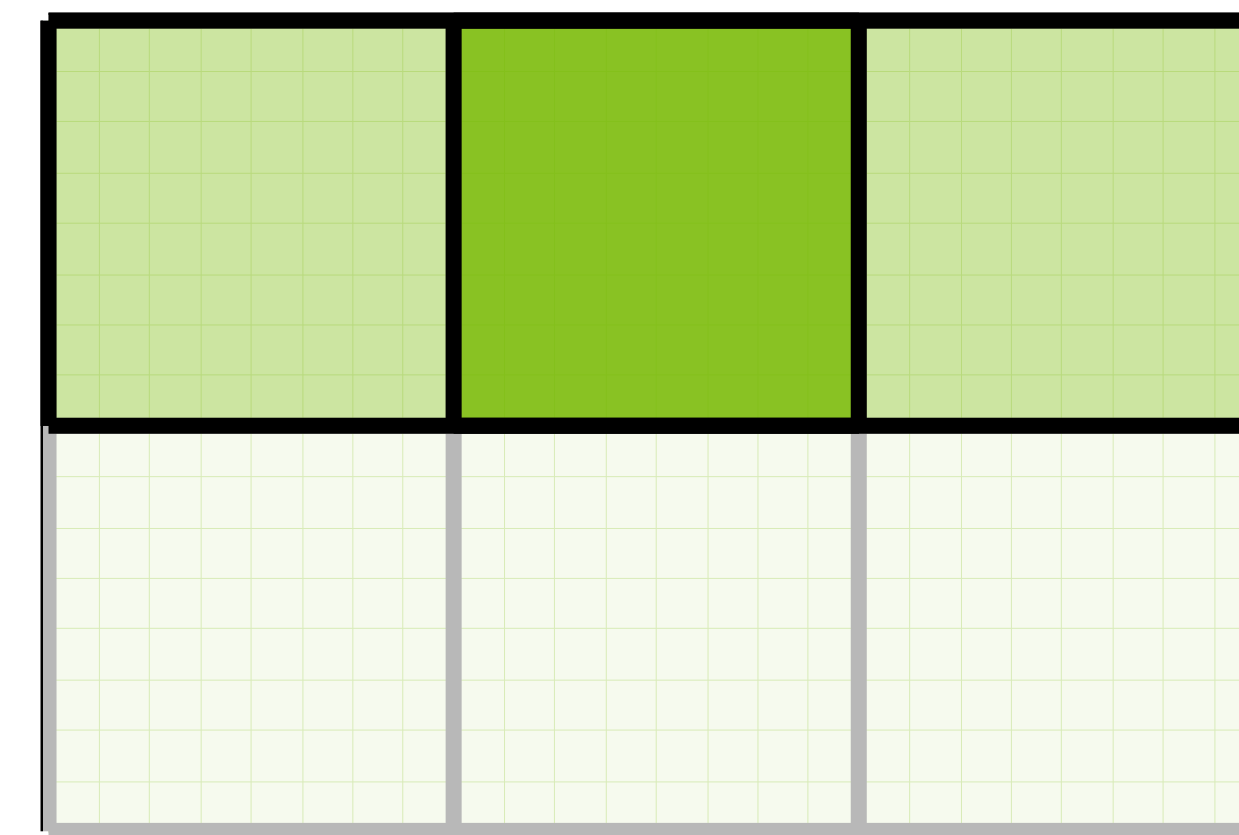
Tile C++: Matrix Multiply

```
auto A = ct::tensor_span{aPtr, ct::extents{M, K}};  
auto B = ct::tensor_span{bPtr, ct::extents{K, N}};  
auto C = ct::tensor_span{cPtr, ct::extents{M, N}};  
  
auto aView = ct::partition_view{A, MxK{}};  
auto bView = ct::partition_view{B, KxN{}};  
auto cView = ct::partition_view{C, MxN{}};  
  
auto [bidx, bidy, dummy] = ct::bid();  
  
auto acc = ct::zeros<ct::tile<__half, MxN>>();  
int numTiles = 1 + (K - 1) / TILE_K;  
  
for (auto idx : ct::irange(0, numTiles)) {  
    auto aTile = aView.load_masked(bidx, idx);  
    auto bTile = bView.load_masked(idx, bidy);  
    acc = ct::mma(aTile, bTile, acc);  
}  
  
cView.store_masked(acc, bidx, bidy);  
}
```



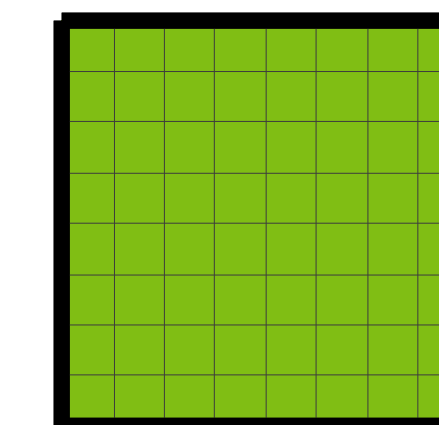
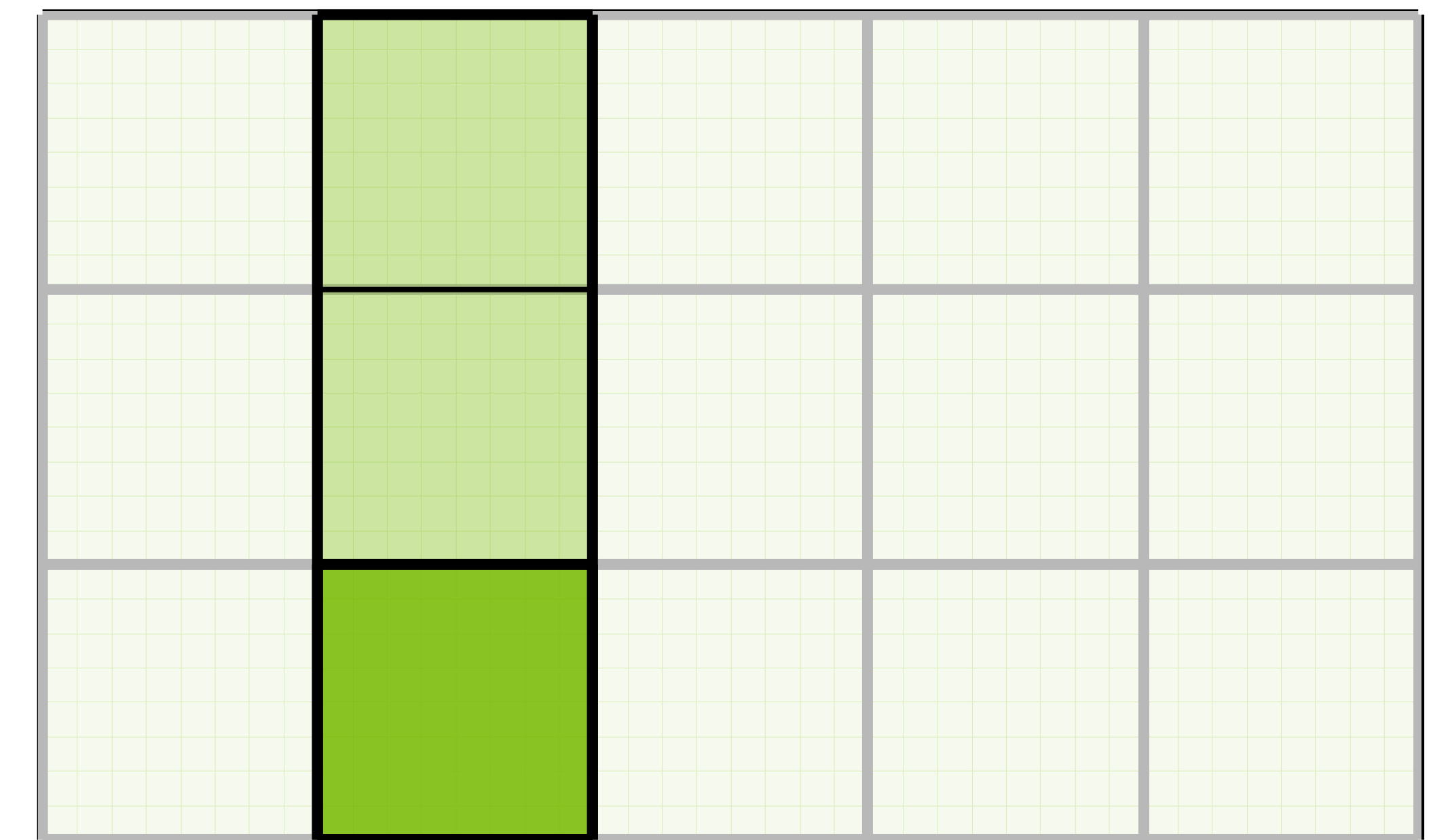
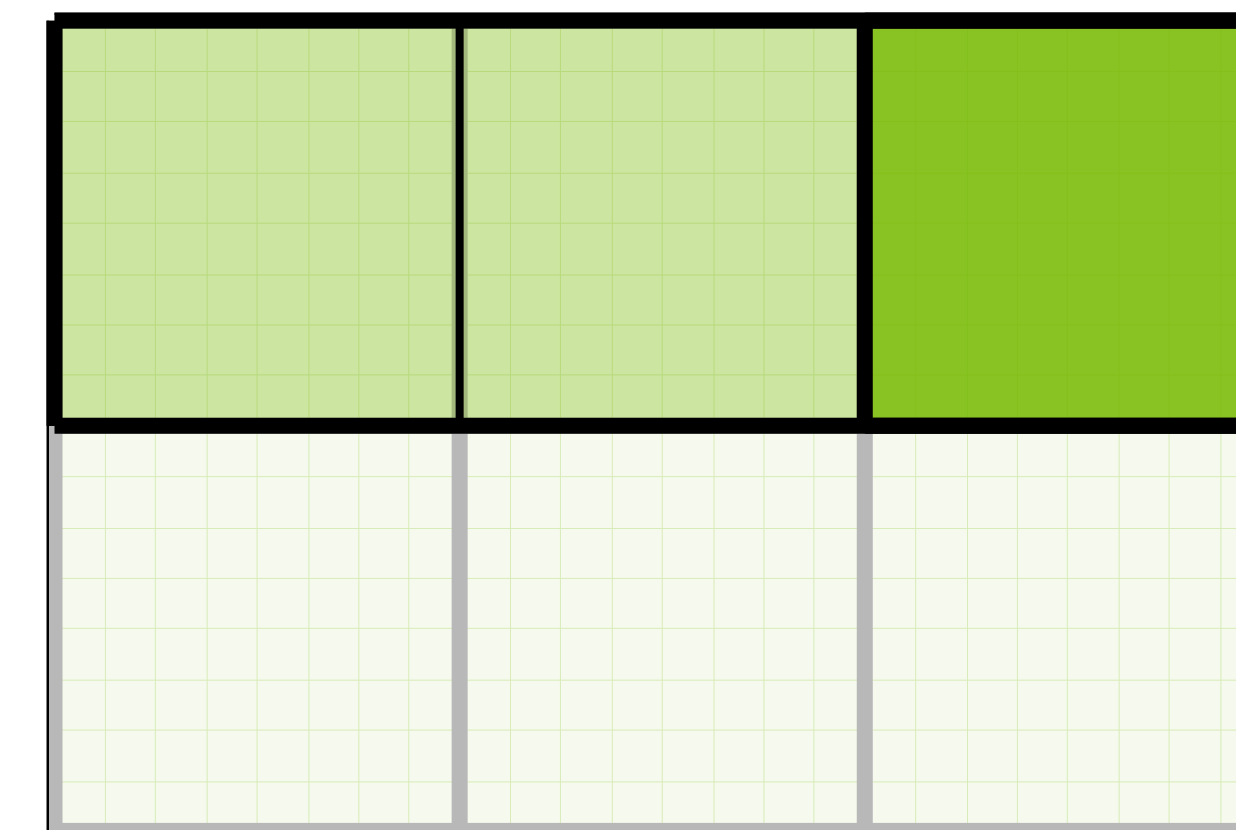
Tile C++: Matrix Multiply

```
auto A = ct::tensor_span{aPtr, ct::extents{M, K}};  
auto B = ct::tensor_span{bPtr, ct::extents{K, N}};  
auto C = ct::tensor_span{cPtr, ct::extents{M, N}};  
  
auto aView = ct::partition_view{A, MxK{}};  
auto bView = ct::partition_view{B, KxN{}};  
auto cView = ct::partition_view{C, MxN{}};  
  
auto [bidx, bidy, dummy] = ct::bid();  
  
auto acc = ct::zeros<ct::tile<__half, MxN>>();  
int numTiles = 1 + (K - 1) / TILE_K;  
  
for (auto idx : ct::irange(0, numTiles)) {  
    auto aTile = aView.load_masked(bidx, idx);  
    auto bTile = bView.load_masked(idx, bidy);  
    acc = ct::mma(aTile, bTile, acc);  
}  
  
cView.store_masked(acc, bidx, bidy);  
}
```



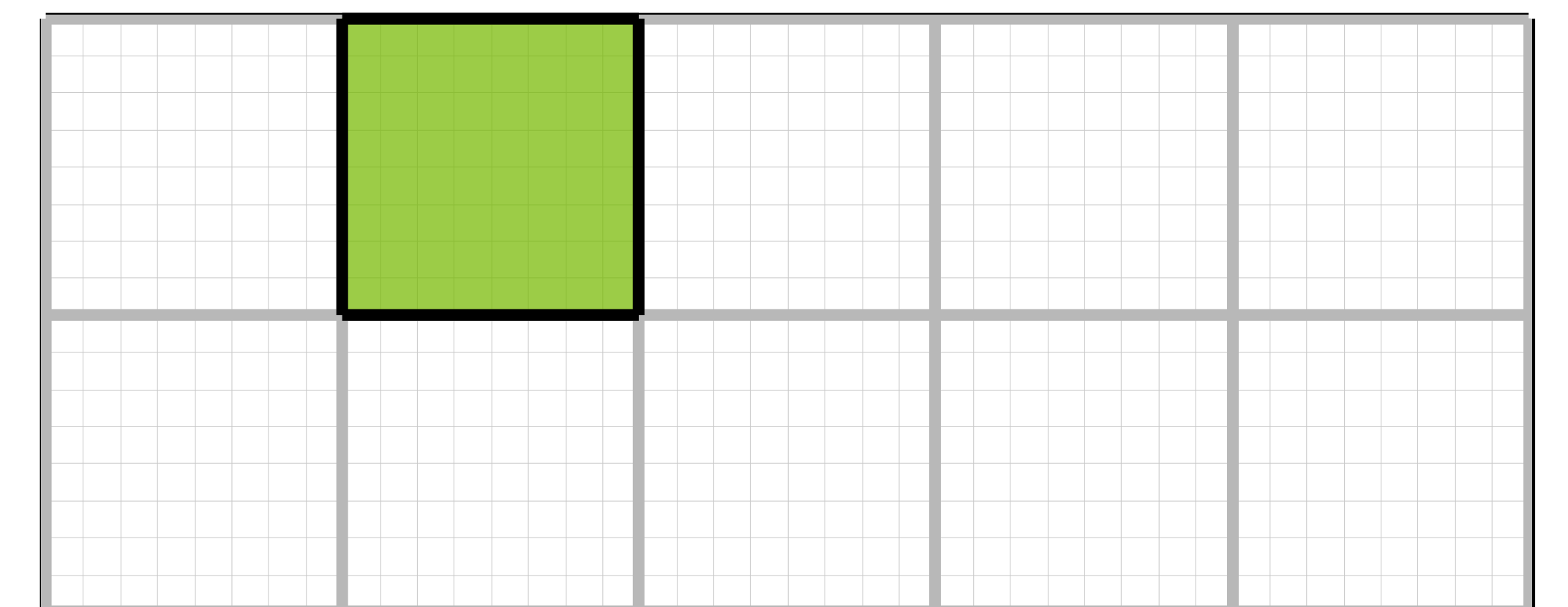
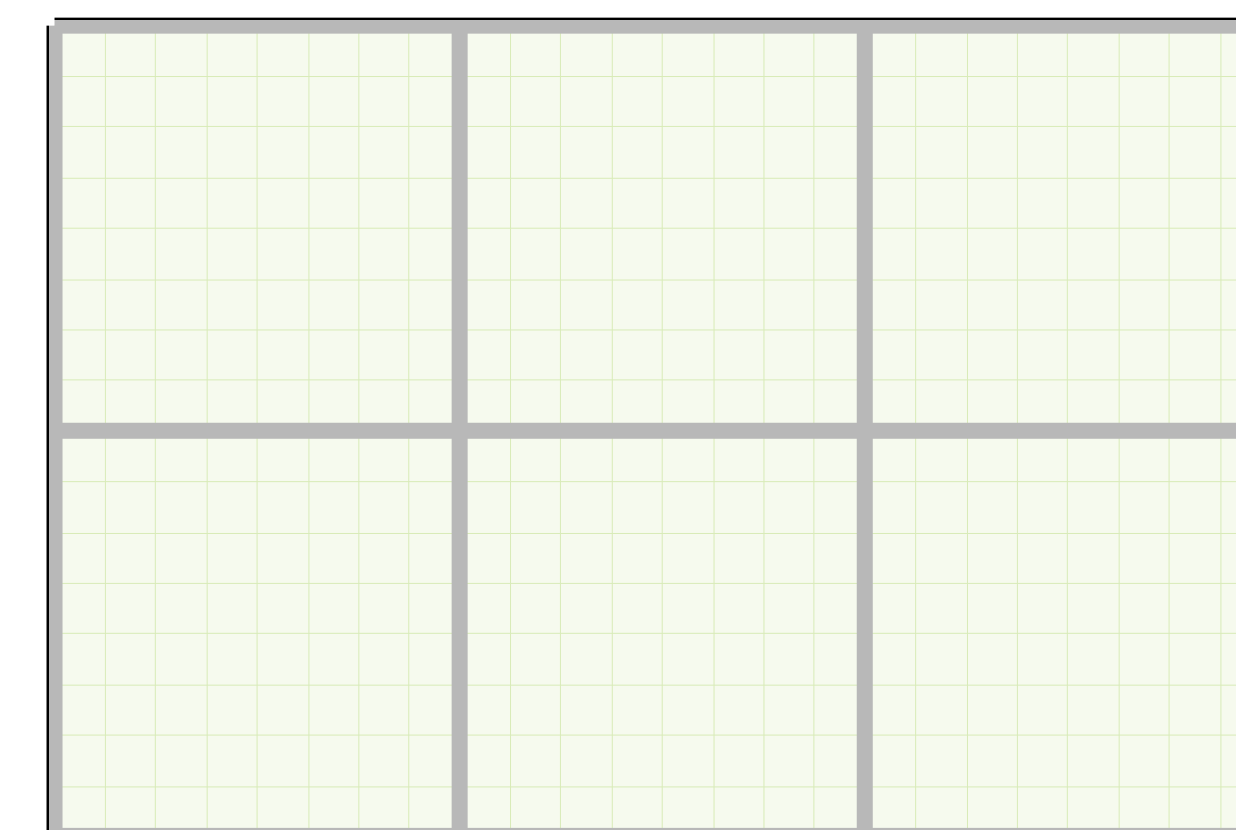
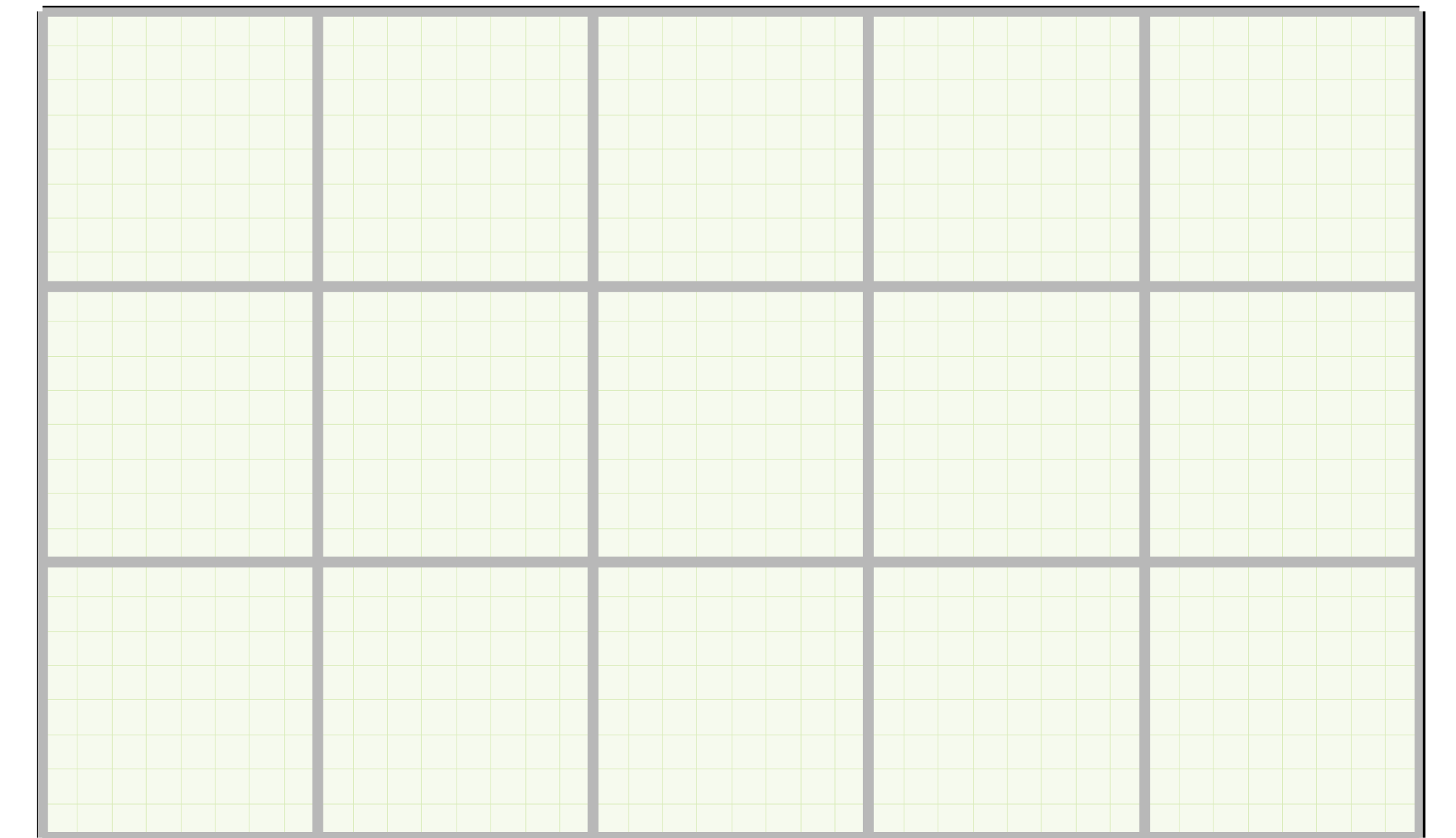
Tile C++: Matrix Multiply

```
auto A = ct::tensor_span{aPtr, ct::extents{M, K}};  
auto B = ct::tensor_span{bPtr, ct::extents{K, N}};  
auto C = ct::tensor_span{cPtr, ct::extents{M, N}};  
  
auto aView = ct::partition_view{A, MxK{}};  
auto bView = ct::partition_view{B, KxN{}};  
auto cView = ct::partition_view{C, MxN{}};  
  
auto [bidx, bidy, dummy] = ct::bid();  
  
auto acc = ct::zeros<ct::tile<__half, MxN>>();  
int numTiles = 1 + (K - 1) / TILE_K;  
  
for (auto idx : ct::irange(0, numTiles)) {  
    auto aTile = aView.load_masked(bidx, idx);  
    auto bTile = bView.load_masked(idx, bidy);  
    acc = ct::mma(aTile, bTile, acc);  
}  
  
cView.store_masked(acc, bidx, bidy);  
}
```



Tile C++: Matrix Multiply

```
auto A = ct::tensor_span{aPtr, ct::extents{M, K}};  
auto B = ct::tensor_span{bPtr, ct::extents{K, N}};  
auto C = ct::tensor_span{cPtr, ct::extents{M, N}};  
  
auto aView = ct::partition_view{A, MxK{}};  
auto bView = ct::partition_view{B, KxN{}};  
auto cView = ct::partition_view{C, MxN{}};  
  
auto [bidx, bidy, dummy] = ct::bid();  
  
auto acc = ct::zeros<ct::tile<__half, MxN>>();  
int numTiles = 1 + (K - 1) / TILE_K;  
  
for (auto idx : ct::irange(0, numTiles)) {  
    auto aTile = aView.load_masked(bidx, idx);  
    auto bTile = bView.load_masked(idx, bidy);  
    acc = ct::mma(aTile, bTile, acc);  
}  
  
cView.store_masked(acc, bidx, bidy);  
}
```



Tile C++: Matrix Multiply

```
auto A = ct::tensor_span{aPtr, ct::extents{M, K}};  
auto B = ct::tensor_span{bPtr, ct::extents{K, N}};  
auto C = ct::tensor_span{cPtr, ct::extents{M, N}};  
  
auto aView = ct::partition_view{A, MxK{}};  
auto bView = ct::partition_view{B, KxN{}};  
auto cView = ct::partition_view{C, MxN{}};  
  
auto [bidx, bidy, dummy] = ct::bid();  
  
auto acc = ct::zeros<ct::tile<__half, MxN>>();  
int numTiles = 1 + (K - 1) / TILE_K;  
  
for (auto idx : ct::irange(0, numTiles)) {  
    auto aTile = aView.load_masked(bidx, idx);  
    auto bTile = bView.load_masked(idx, bidy);  
    acc = ct::mma(aTile, bTile, acc);  
}  
  
cView.store_masked(acc, bidx, bidy);  
}
```

- Loop with known iteration space

Tile C++: Matrix Multiply

```
__tile_global__ void mma_tile_kernel(  
    const __half* __restrict__ aPtr,  
    const __half* __restrict__ bPtr,  
    int M, int N, int K,  
    __half* __restrict__ cPtr) {  
  
    namespace ct = cuda::tiles;  
    using namespace ct::literals;  
  
    aPtr = ct::assume_aligned(aPtr, 16_ic);  
    bPtr = ct::assume_aligned(bPtr, 16_ic);  
    cPtr = ct::assume_aligned(cPtr, 16_ic);  
  
    M = ct::assume_divisible(M, 16_ic);  
    N = ct::assume_divisible(N, 16_ic);  
    K = ct::assume_divisible(K, 16_ic);  
  
    using MxN = ct::shape<TILE_M, TILE_N>;  
    using MxK = ct::shape<TILE_M, TILE_K>;  
    using KxN = ct::shape<TILE_K, TILE_N>;
```

Tile C++: Matrix Multiply

```
tile global void mma tile kernel(  
    const __half* __restrict__ aPtr,  
    const __half* __restrict__ bPtr,  
    int M, int N, int K,  
    __half* __restrict__ cPtr) {
```

```
    namespace ct = cuda::tiles;  
    using namespace ct::literals;  
  
    aPtr = ct::assume_aligned(aPtr, 16_ic);  
    bPtr = ct::assume_aligned(bPtr, 16_ic);  
    cPtr = ct::assume_aligned(cPtr, 16_ic);
```

```
    M = ct::assume_divisible(M, 16_ic);  
    N = ct::assume_divisible(N, 16_ic);  
    K = ct::assume_divisible(K, 16_ic);
```

```
    using MxN = ct::shape<TILE_M, TILE_N>;  
    using MxK = ct::shape<TILE_M, TILE_K>;  
    using KxN = ct::shape<TILE_K, TILE_N>;
```

- Declares no overlap of inputs
- Critical for automatic parallelization

Tile C++: Matrix Multiply

```
__tile_global__ void mma_tile_kernel(  
    const __half* __restrict__ aPtr,  
    const __half* __restrict__ bPtr,  
    int M, int N, int K,  
    __half* __restrict__ cPtr) {
```

```
    namespace ct = cuda::tiles;  
    using namespace ct::literals;
```

```
    aPtr = ct::assume_aligned(aPtr, 16_ic);  
    bPtr = ct::assume_aligned(bPtr, 16_ic);  
    cPtr = ct::assume_aligned(cPtr, 16_ic);  
  
    M = ct::assume_divisible(M, 16_ic);  
    N = ct::assume_divisible(N, 16_ic);  
    K = ct::assume_divisible(K, 16_ic);
```

```
    using MxN = ct::shape<TILE_M, TILE_N>;  
    using MxK = ct::shape<TILE_M, TILE_K>;  
    using KxN = ct::shape<TILE_K, TILE_N>;
```

- Declares pointer and size alignment
- Allows TMA hardware to be used for load/store

Tile C++: Matrix Multiply

```
__tile_global__ void mma_tile_kernel(  
    const __half* __restrict__ aPtr,  
    const __half* __restrict__ bPtr,  
    int M, int N, int K,  
    __half* __restrict__ cPtr) {  
  
    namespace ct = cuda::tiles;  
    using namespace ct::literals;  
  
    aPtr = ct::assume_aligned(aPtr, 16_ic);  
    bPtr = ct::assume_aligned(bPtr, 16_ic);  
    cPtr = ct::assume_aligned(cPtr, 16_ic);  
  
    M = ct::assume_divisible(M, 16_ic);  
    N = ct::assume_divisible(N, 16_ic);  
    K = ct::assume_divisible(K, 16_ic);
```

```
using MxN = ct::shape<TILE_M, TILE_N>;  
using MxK = ct::shape<TILE_M, TILE_K>;  
using KxN = ct::shape<TILE_K, TILE_N>;
```

- Sets the tiling size for caching data from global memory
- Profiling should be used to select the concrete tile sizes

Tile C++: Matrix Multiply

```
__tile_global__ void mma_tile_kernel(
    const __half* __restrict__ aPtr,
    const __half* __restrict__ bPtr,
    int M, int N, int K,
    __half* __restrict__ cPtr) {

    namespace ct = cuda::tiles;
    using namespace ct::literals;

    aPtr = ct::assume_aligned(aPtr, 16_ic);
    bPtr = ct::assume_aligned(bPtr, 16_ic);
    cPtr = ct::assume_aligned(cPtr, 16_ic);

    M = ct::assume_divisible(M, 16_ic);
    N = ct::assume_divisible(N, 16_ic);
    K = ct::assume_divisible(K, 16_ic);

    using MxN = ct::shape<TILE_M, TILE_N>;
    using MxK = ct::shape<TILE_M, TILE_K>;
    using KxN = ct::shape<TILE_K, TILE_N>;
```

```
    auto A = ct::tensor_span{aPtr, ct::extents{M, K}};
    auto B = ct::tensor_span{bPtr, ct::extents{K, N}};
    auto C = ct::tensor_span{cPtr, ct::extents{M, N}};

    auto aView = ct::partition_view{A, MxK{}};
    auto bView = ct::partition_view{B, KxN{}};
    auto cView = ct::partition_view{C, MxN{}};

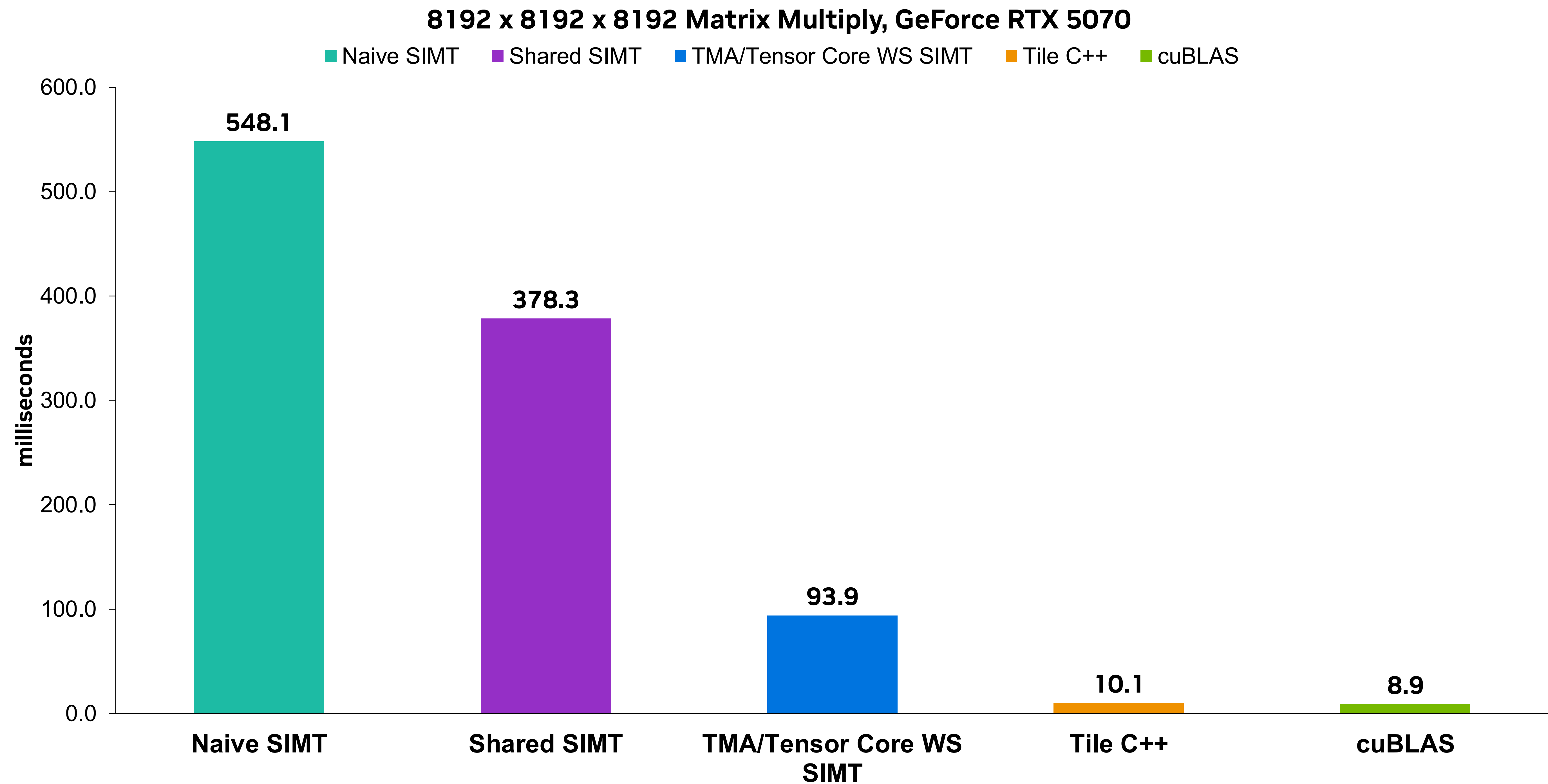
    auto [bidx, bidy, dummy] = ct::bid();

    auto acc = ct::zeros<ct::tile<__half, MxN>>();
    int numTiles = 1 + (K - 1) / TILE_K;

    for (auto idx : ct::irange(0, numTiles)) {
        auto aTile = aView.load_masked(bidx, idx);
        auto bTile = bView.load_masked(idx, bidy);
        acc = ct::mma(aTile, bTile, acc);
    }

    cView.store_masked(acc, bidx, bidy);
}
```

Tile C++: Matrix Multiply Performance



Tile C++ is C++

- Tile C++ is general purpose
- Most C++ language features are supported
 - classes, structs, unions
 - most control flow
 - pointer arithmetic & indirection

CUDA Tile C++

- Available in CUDA Toolkit 13.3
- Ampere (sm_80), r580 driver, C++20 minimum

```
$ nvcc --enable-tile --arch sm_80 --std=c++20 foo.cu
$ ./a.out
Hello, SIMT!
Hello, Tile!
```

```
__global__ void simtHello() {
    printf("Hello, SIMT!\n");
}

__tile_global__ void tileHello() {
    printf("Hello, Tile!\n");
}

int main() {
    simtHello<<<1,1>>>();
    tileHello<<<1,1>>>();
    cudaDeviceSynchronize();
}
```

CUDA Tile C++ - Resources and Q & A

- **Blog** — [High-Performance GPU Kernels in C++ with NVIDIA CUDA Tile](#)
- **Examples** — [NVIDIA/cuda-samples on GitHub](#)
- **Programming Guide** — [2.4 Writing Tile Kernels](#)
- **API Reference** — [CUDA Tile C++ API Reference](#)
- **High-Performance Kernels** — [NVIDIA/TileGym on GitHub](#)